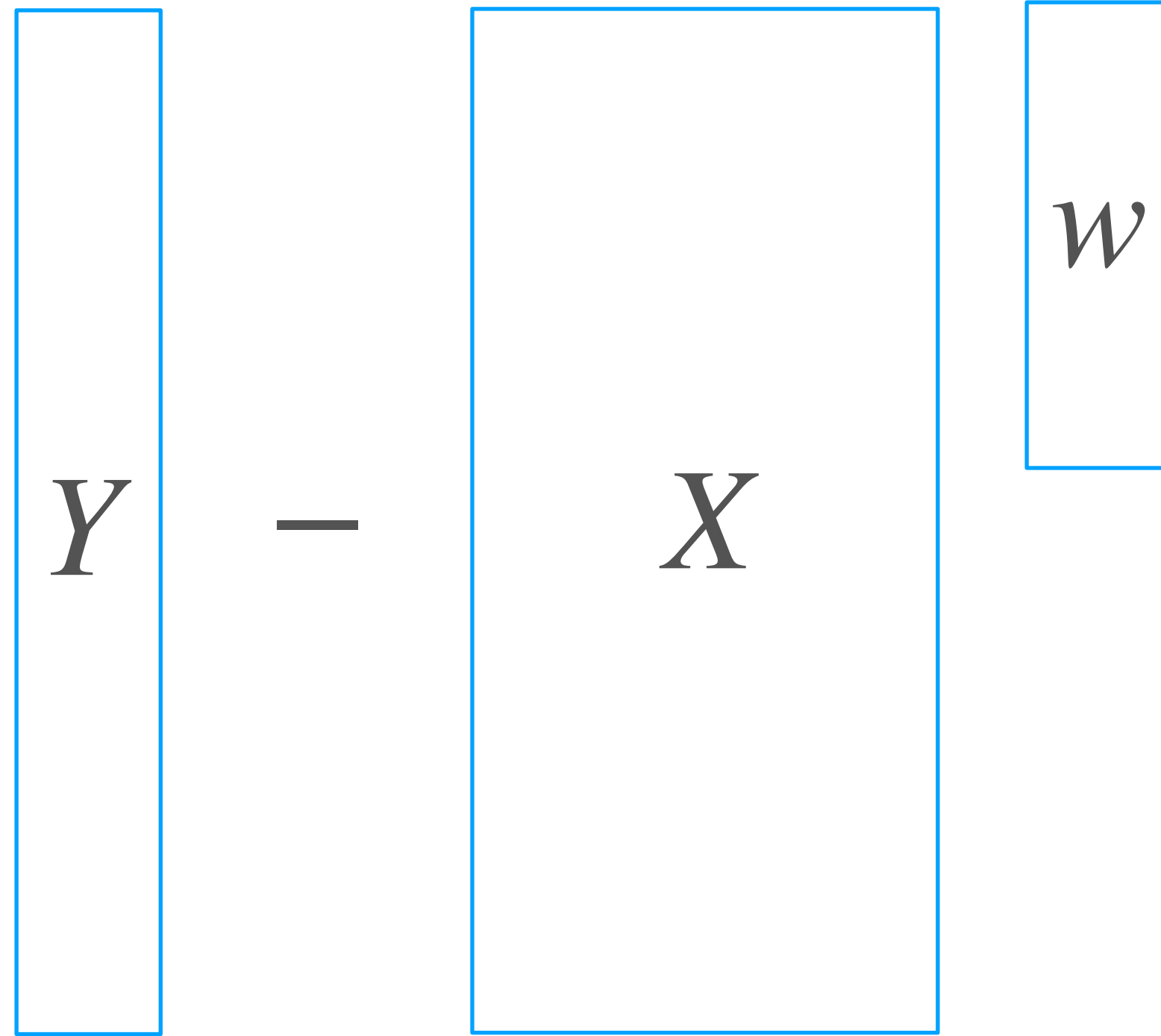


Linear regression



10-701 Introduction to Machine Learning
Geoff Gordon

Gradients



- For any of the solution methods, need gradients

$$\begin{aligned} \blacktriangleright L(w) &= \frac{\lambda}{2} \|w\|^2 + \frac{1}{2} \sum_{t=1}^N (Y^{(t)} - w \cdot X^{(t)})^2 \\ &= \frac{\lambda}{2} \|w\|^2 + \frac{1}{2} \|Y - Xw\|^2 \end{aligned}$$

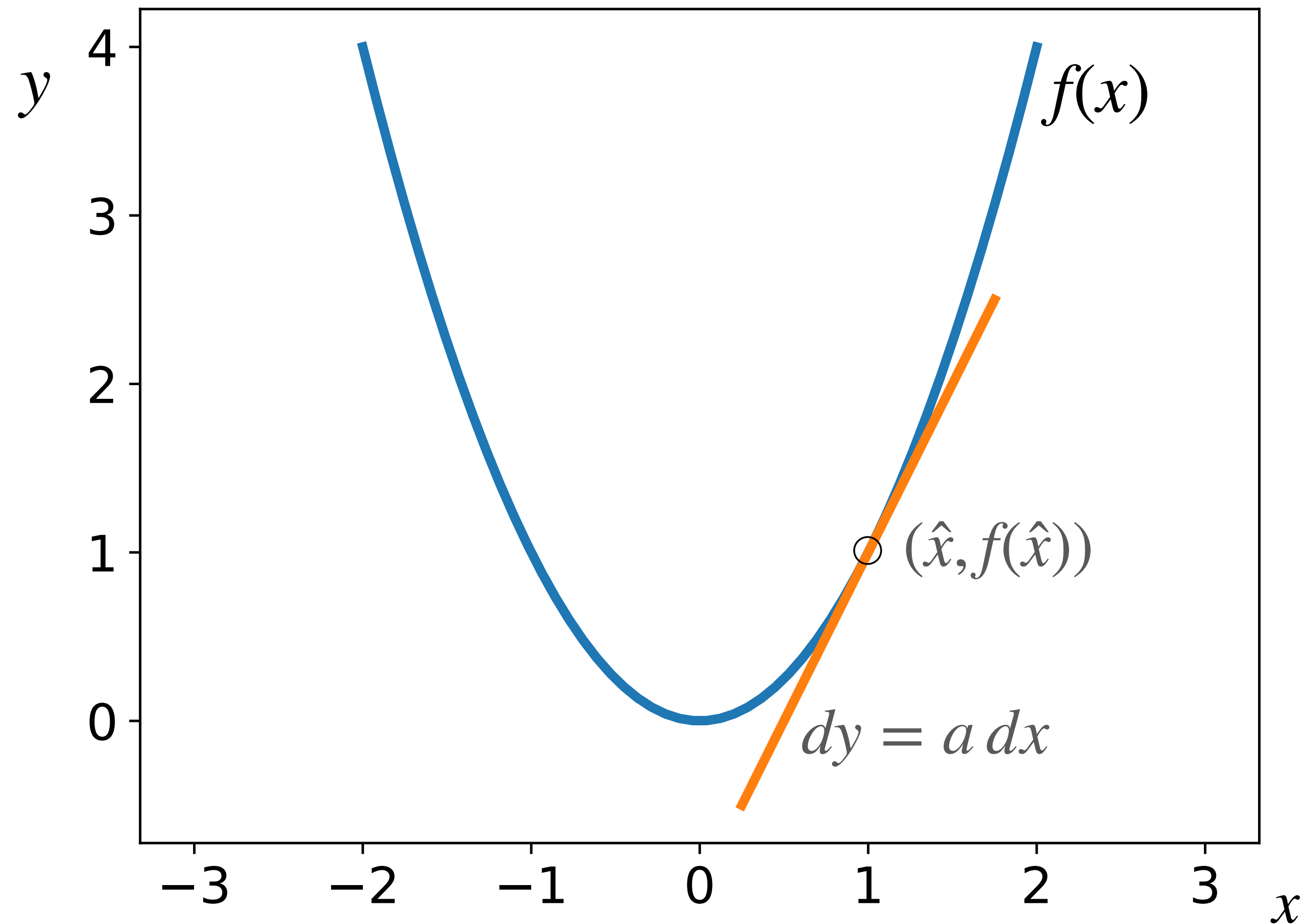
$$\blacktriangleright \nabla L(w) =$$

Derivatives everywhere

- The way we derive many ML methods:
 - ▶ start from objective to minimize (here, sum of squared errors plus optional regularizer)
 - ▶ take a few derivatives ($0 = d \text{ objective} / dw$)
 - ▶ find optimum (e.g., by setting to 0 and solving, or SGD)
 - ▶ done!
- Will continue to be a common pattern
- Review of intuition and tools for high-d derivatives, so we can do it fast

Derivatives notation

dx, dy are called
differentials



$$m = n = 1$$

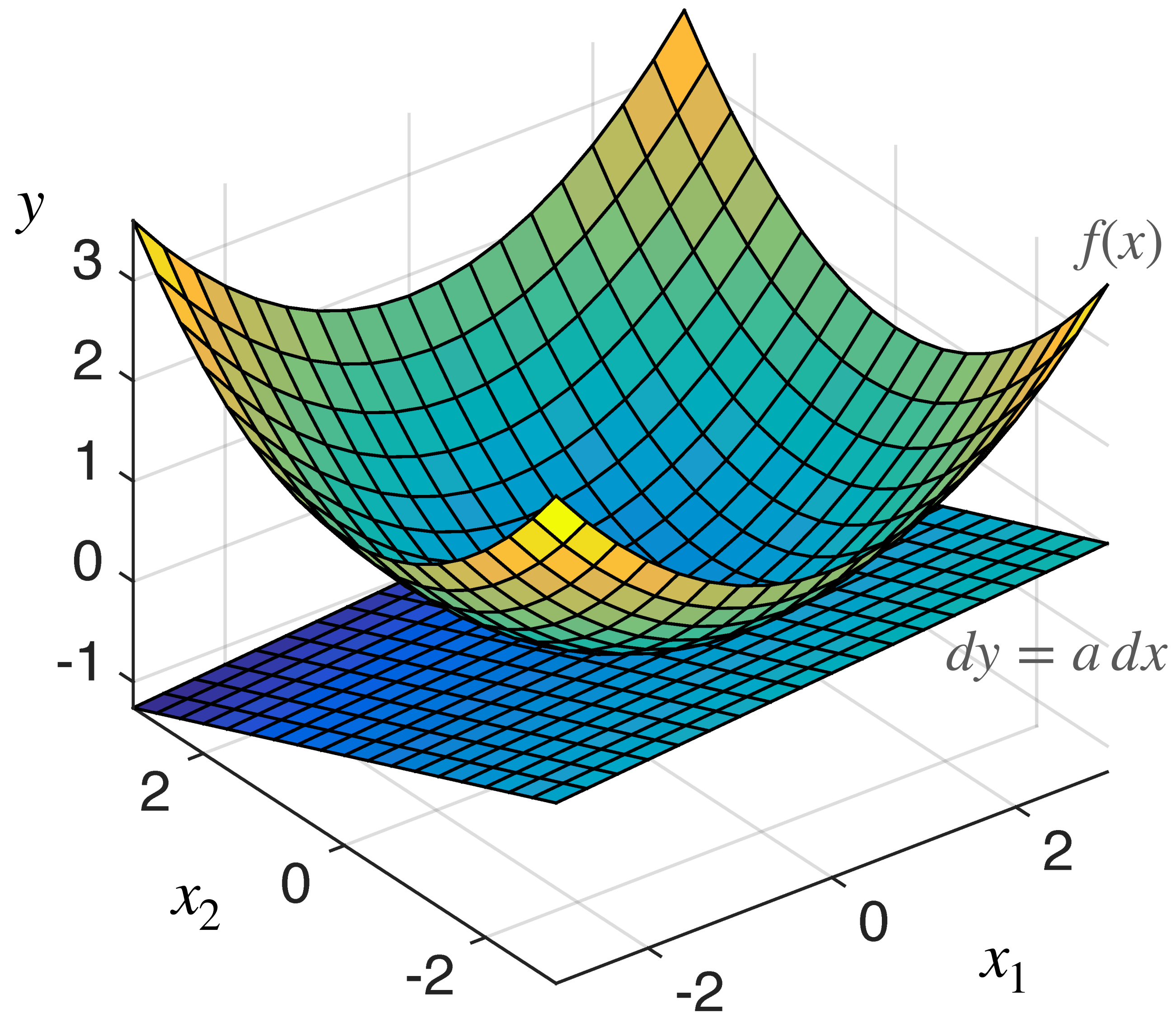
$$dy = y - f(\hat{x})$$
$$dx = x - \hat{x}$$

- Function $y = f(x)$
 - ▶ $x \in \mathbb{R}^n, y \in \mathbb{R}^m, f \in \mathbb{R}^n \rightarrow \mathbb{R}^m$
- Derivative is a *linear fn* that *locally* approximates f near $\hat{x}$
 - ▶ $dy = a dx$, where $a \in \mathbb{R}$ is the derivative

Compare older vs. newer notation

- Older version:
 - ▶ $\frac{dy}{dx}$ is atomic: derivative of y wrt x
 - ▶ dx and dy are meaningless on their own
 - ▶ focused on infinitesimals: change in x or $y \rightarrow 0$
- Newer version:
 - ▶ dx and dy have separate meaning: terms in a linear function (the Taylor expansion)
 - ▶ dx and dy can be any size: linear function is well defined even for large changes in x or y (though accuracy of Taylor approximation can be bad if dx , dy are big)
- Heuristic to convert notation: “divide through” or “multiply through” by dx (only actually makes sense in scalar case, but often succeeds anyway)

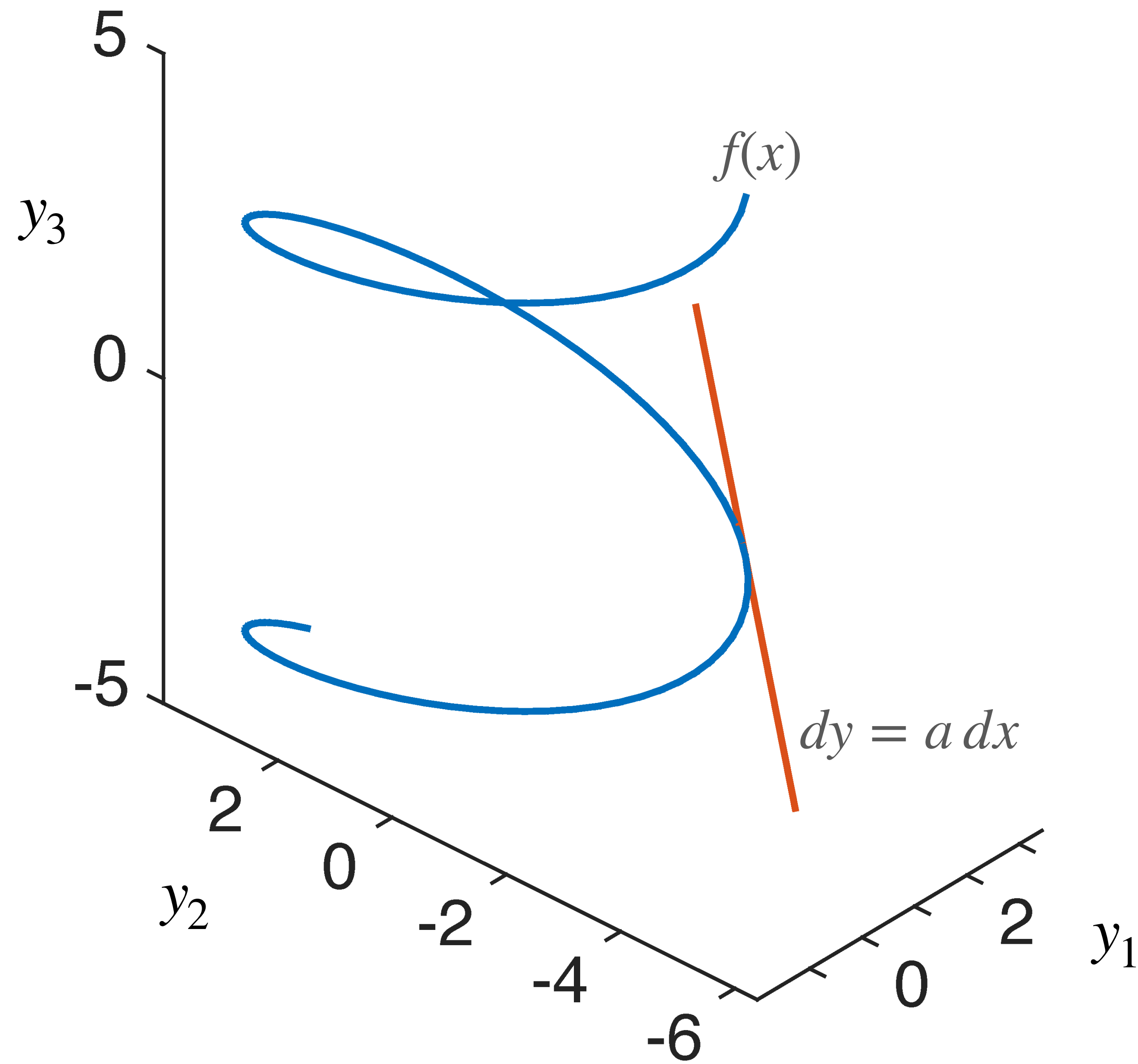
Higher dimensions



$$\mathbb{R}^n \rightarrow \mathbb{R}^m$$
$$m = 1, n = 2$$

Function $y = f(x)$ $dy = a dx$ $a \in \mathbb{R}^{1 \times 2}$

Higher dimensions



$$\mathbb{R}^n \rightarrow \mathbb{R}^m$$
$$m = 3, n = 1$$

Function $y = f(x)$ $dy = a dx$ $a \in \mathbb{R}^{3 \times 1}$

Notation for linear functions

output shape (derivative of)

input shape (with respect to)			
	Scalar	Vector	Matrix
Scalar	$ds = a dt$	$ds = u^T dv$	$ds = \text{tr}(M^T dN)$
Vector	$du = v ds$	$du = M dv$	$\times$
Matrix	$dM = N ds$	$\times$	$\times$

- a, s, t : scalars u, v : column vectors M, N : matrices
- note: $ds = a dt$ is a synonym for $a = \frac{ds}{dt}$, etc.
- $\times$ means no common notation; see torch.einsum

Notation for linear functions

output shape (derivative of)

input shape (with respect to)			
	Scalar	Vector	Matrix
Scalar	$d(\text{slope})/dt$	(gradient)	$ds = \text{tr}(M^T dN)$
Vector	(velocity)	(Jacobian)	$\times$
Matrix	$dM = N ds$	$\times$	$\times$

- a, s, t : scalars u, v : column vectors M, N : matrices
- note: $ds = a dt$ is a synonym for $a = \frac{ds}{dt}$, etc.
- $\times$ means no common notation; see torch.einsum

Inner product

- With scalar output s , for any shape of input X , can write $ds = \langle A, dX \rangle$ where A, X, dX all have same shape
 - ▶ $\langle \cdot, \cdot \rangle$ is ***inner product***, in our case $\sum_{\text{index lists } I} A_I dX_I$
 - ▶ e.g., if A, dX are $m \times n$ matrices, then
$$\langle A, dX \rangle = \sum_{i=1}^m \sum_{j=1}^n A_{ij} dX_{ij}$$
- Expressions on previous slide are equivalent to above
 - ▶ most complex is $\text{tr}(M^\top dN)$
 - ▶ tr is sum of diagonal elements of a square matrix
$$[M^\top dN]_{jk} =$$
 - ▶ sum of diagonal elements means set $j = k$, sum over j :

Shape conventions

- Derivative = coefficient of ds, dt, dv, dN on RHS
- Ambiguity: can write $\langle M, dN \rangle$ or $\text{tr}(M^\top dN)$
 - ▶ is the derivative M or $M^\top$?
- In general, x, y , and derivative $\frac{dy}{dx}$ are **tensors** (multidimensional arrays of numbers)
- Convention: in derivative $\frac{dy}{dx}$
 - ▶ the dimensions of y come first, then x
 - ▶ in same order as dimensions of y and x
- For example: if y is 3×2 and x is $4 \times 6 \times 5$ then $\frac{dy}{dx}$ is $3 \times 2 \times 4 \times 6 \times 5$

Special cases

- If y is a scalar (dimensions $()$), then $\frac{dy}{dx}$ is the same shape as x
 - ▶ derivative of a scalar wrt a 5×3 matrix is a 5×3 matrix
- If x and y are vectors, $\frac{dy}{dx}$ (the Jacobian) is the same shape as a linear function from x to y
 - ▶ derivative of a 3-vector wrt a 6-vector is a 3×6 matrix

Useful identities

- $\text{scalar} = \text{scalar}^\top = \text{tr}(\text{scalar})$

- Trace rotation

- ▶ $\text{tr}(ABC) = \text{tr}(CAB)$ when A, B, C have compatible dimensions

compatible dimensions:

- 2nd dimension of A = 1st dimension of B
- 2nd dimension of B = 1st dimension of C
- 2nd dimension of C = 1st dimension of A

- Transpose-Hadamard

- ▶ $A \circ B = \text{Hadamard (componentwise) product,}$

$$[A \circ B]_{ij} = A_{ij}B_{ij}$$

- ▶ $A^\top(B \circ C) = (A \circ B)^\top C$

Tools for derivatives: linearity, chain, product

- Linearity:
 - ▶ $d(ax + b) = d(ax) + d(b) = a \, dx, \quad d(AX + B) = A \, dX, \dots$
- Chain rule for $f(g(x))$
 - ▶ old notation: $\frac{df}{dx} = \frac{df}{dg} \frac{dg}{dx}$ (“cancel the dg ”)
 - ▶ new: if $df = A \, dg, \, dg = B \, dx$ then $df =$
- Product rule for $f(x) \, g(x)$
 - ▶ $d(fg) = df \, g + f \, dg$
 - ▶ works for any kind of product (matrix product, vector cross product, elementwise (Hadamard) product, vector convolution, Khatri-Rao product ...)
 - ▶ unlike scalar case, order can matter (products are not necessarily commutative)

***Gradient
practice:
Linear
regression
in one slide***

$$L = \|Y - Xw\|^2 = (Y - Xw)^\top (Y - Xw)$$

$$L = Y^\top Y - 2Y^\top Xw + w^\top X^\top Xw$$

Learning objectives— linear regression, derivatives

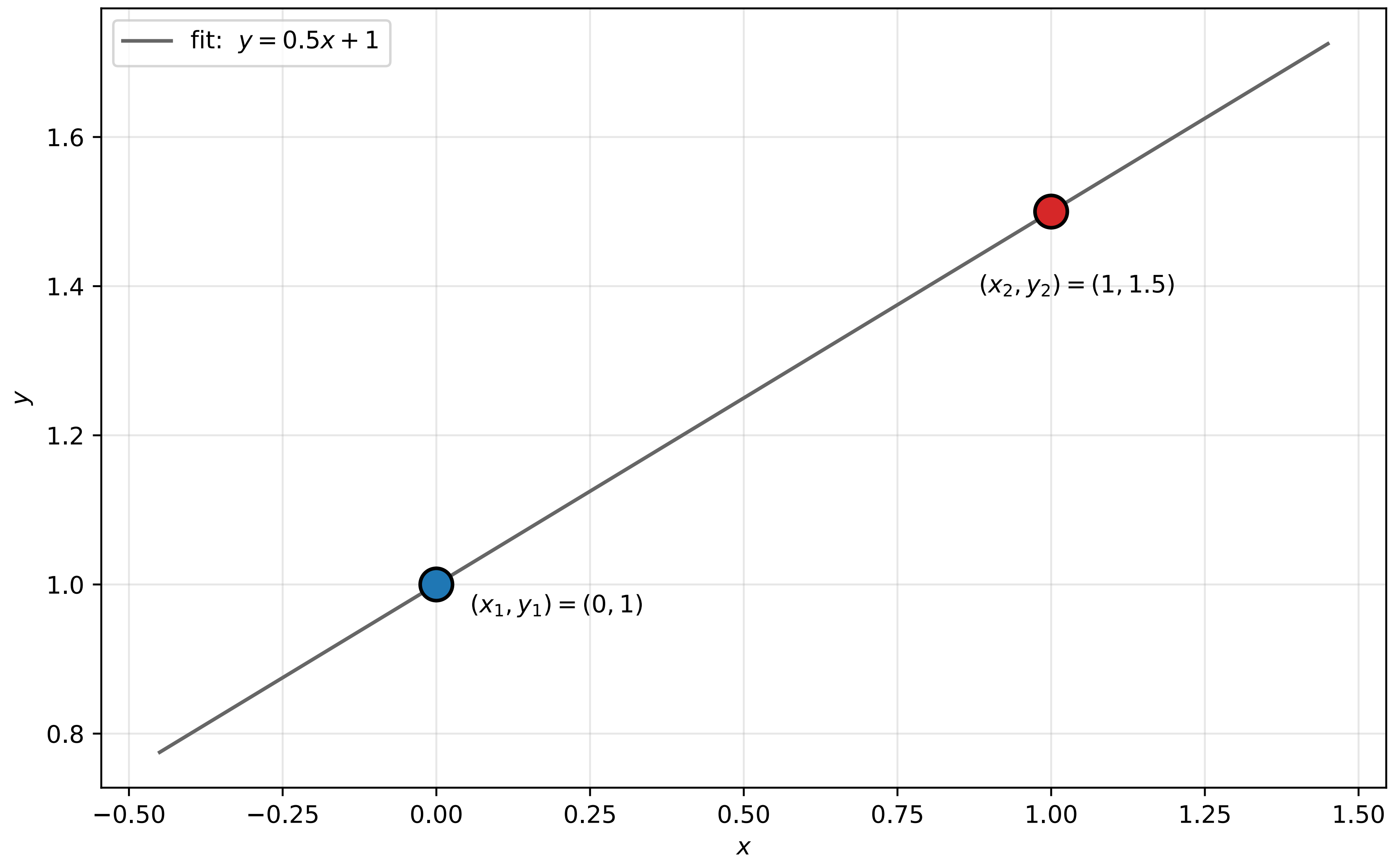
- What is the linear regression model? How can we represent sum of squared errors with matrix notation?
- How should we pick and pre-process features for linear regression?
- What are the effects of common regularizers? How do they affect model complexity and overfitting?
- How can we find gradients of vector, matrix, and tensor functions?
- What is the gradient of the linear regression loss?
- What are the normal equations, and how can we solve them exactly?

Gradient descent

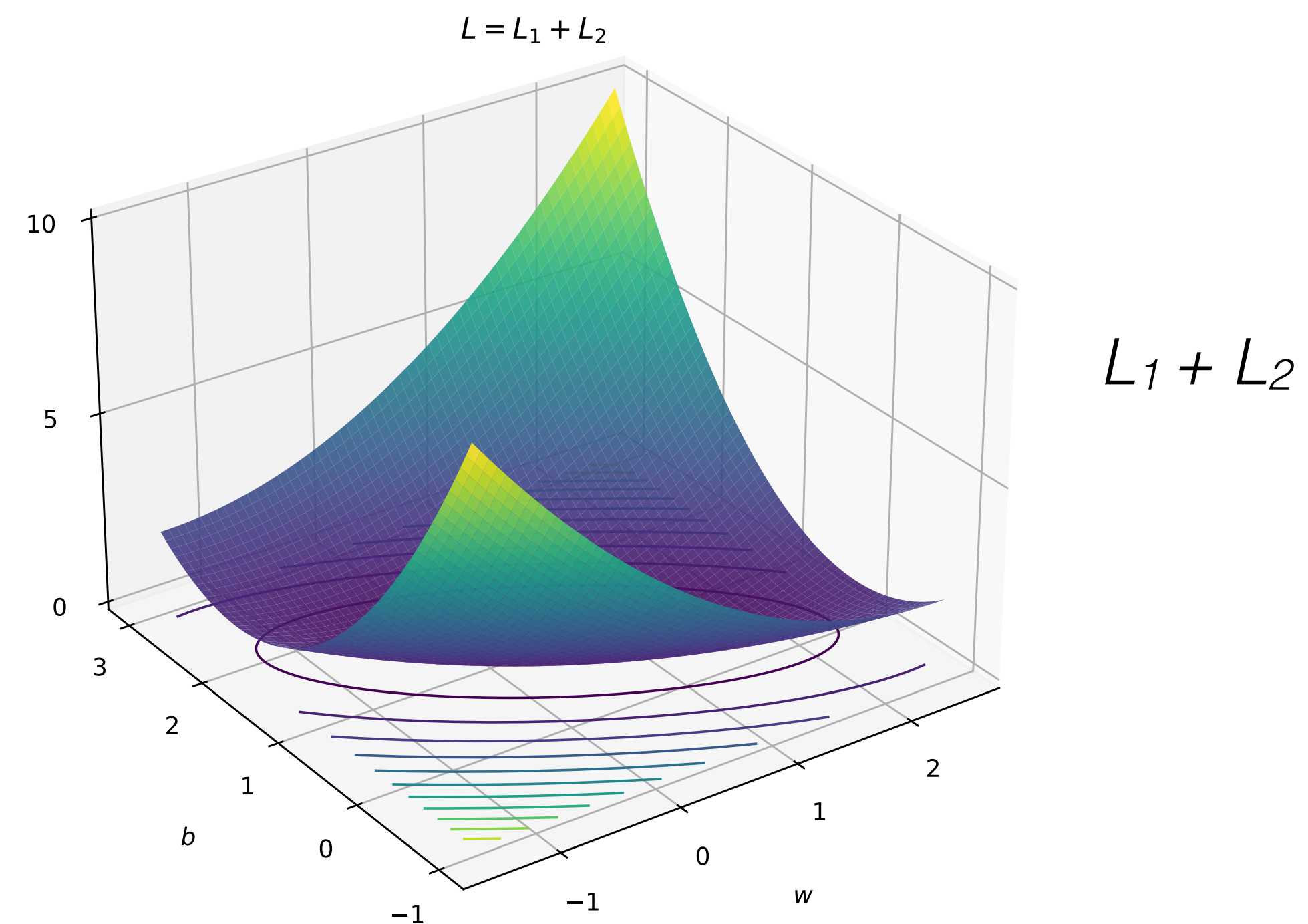
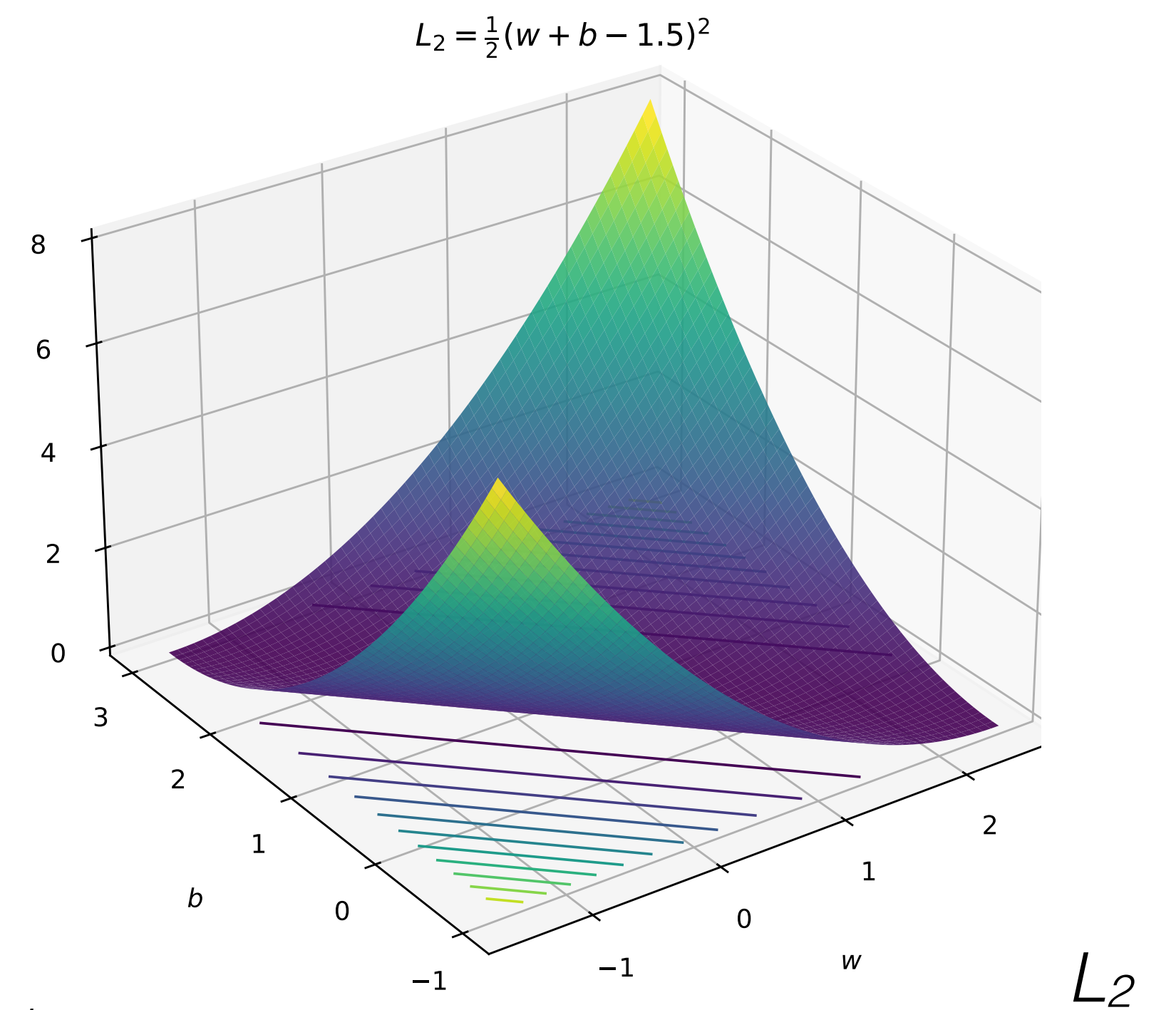
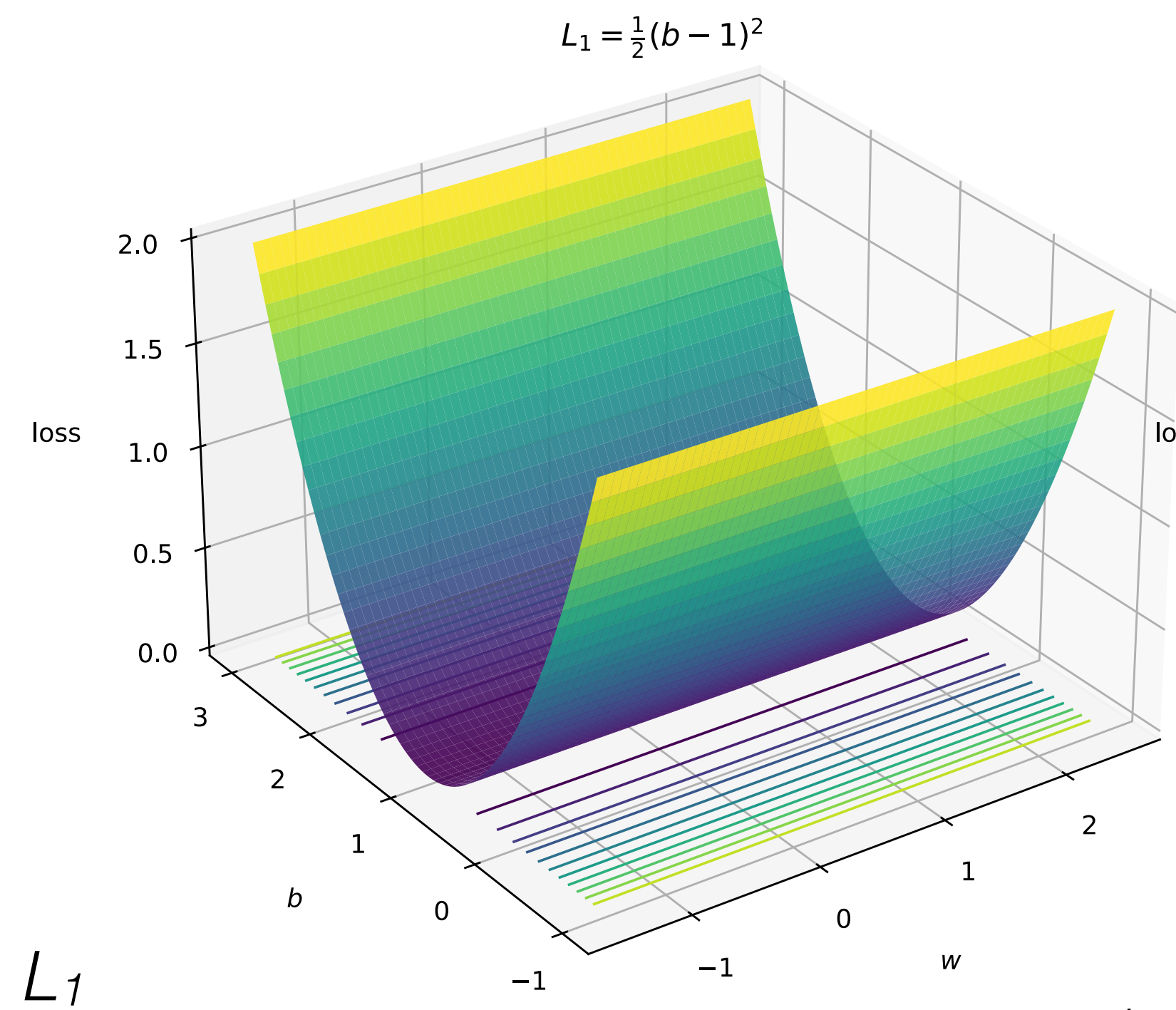


10-701 Introduction to Machine Learning
Geoff Gordon

A very simple regression problem

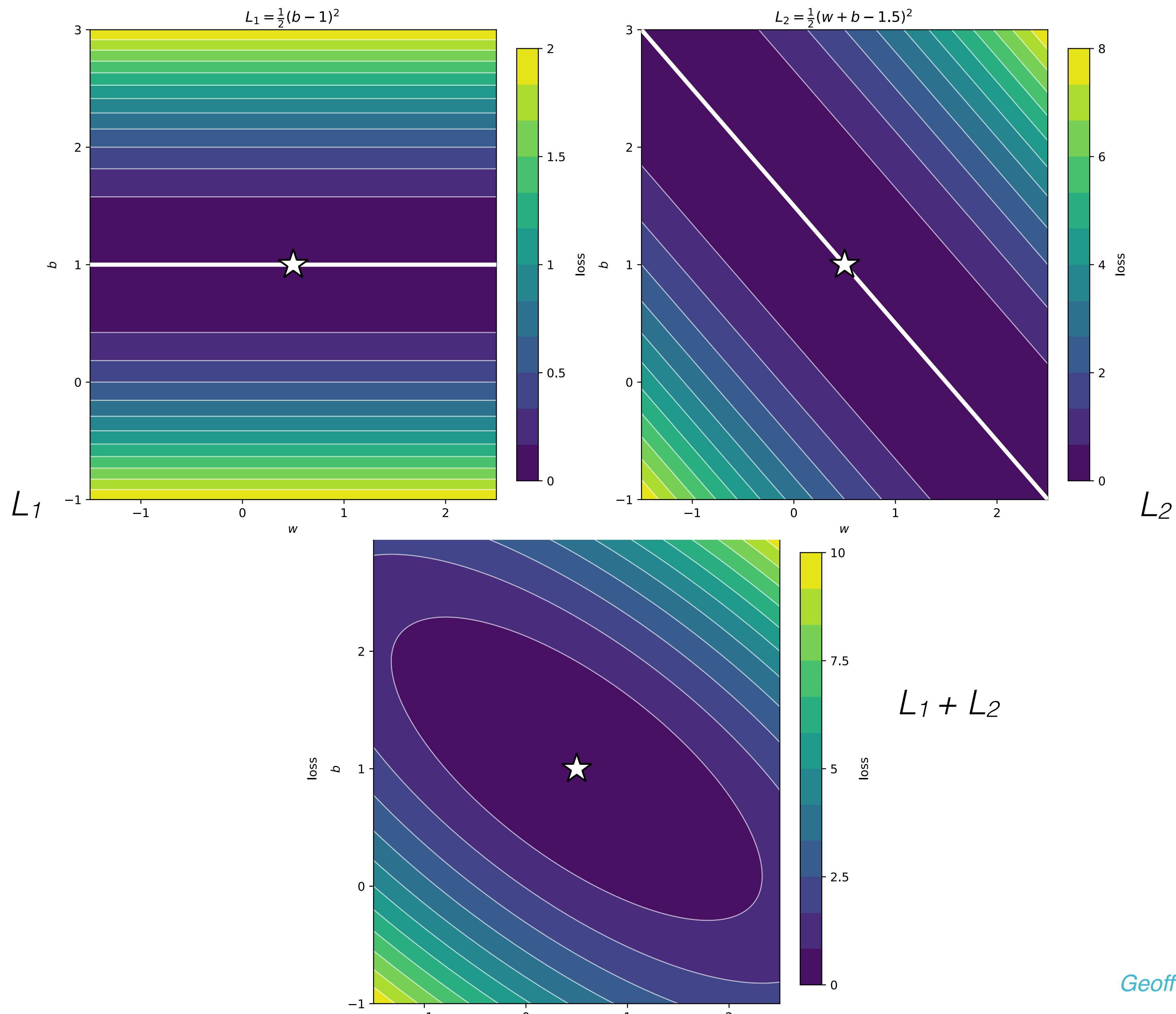


Loss function: sum of squared errors



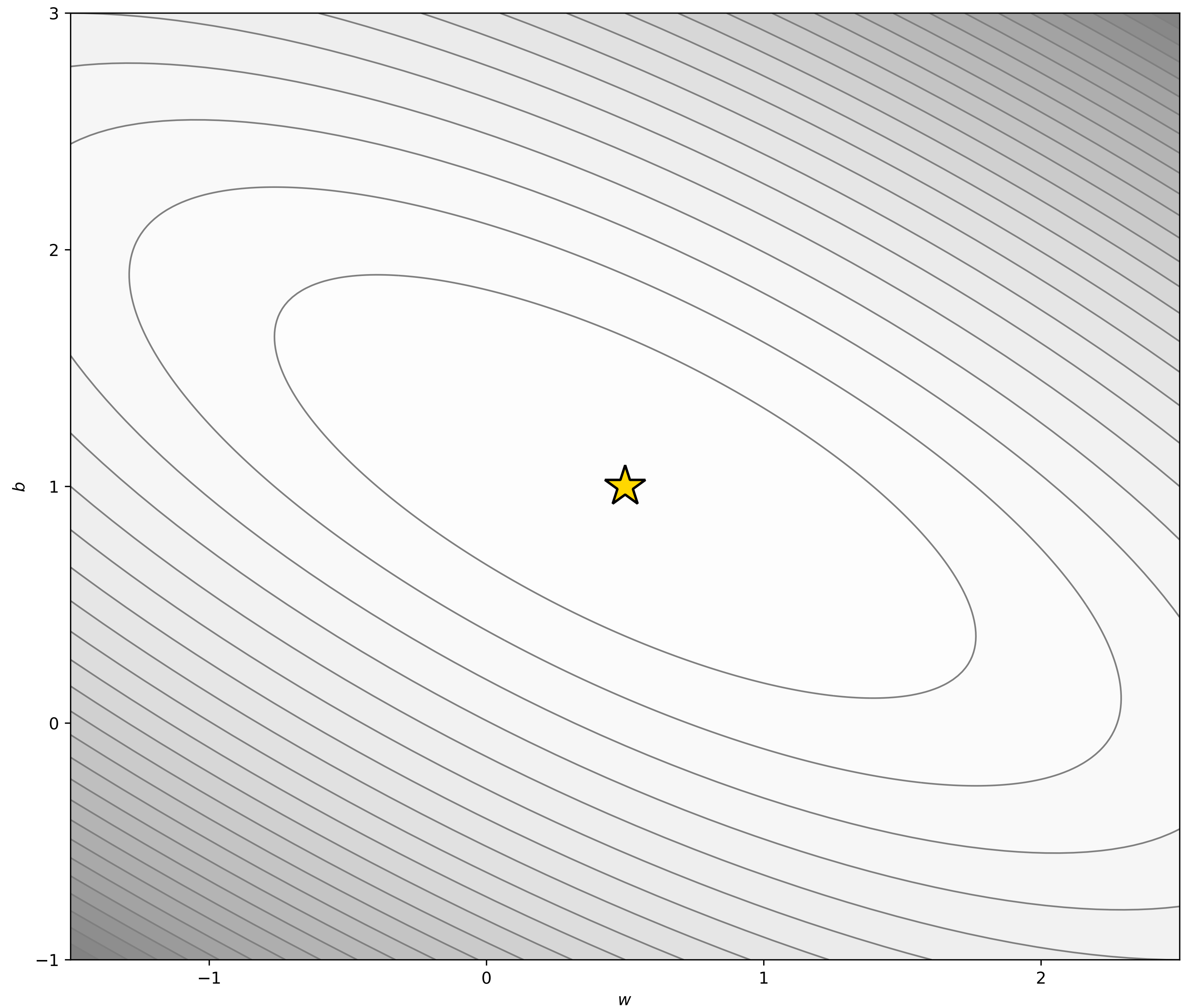
As contour plots

Colors show height
Each contour is equal height



***Find the
optimal w, b***

Gradients $\perp$ contours



Gradient descent

Algorithm: gradient descent

Inputs: loss function $L(\theta)$, initial value $\theta^{(1)}$, step size η

For $i \leftarrow 1, 2, \dots$:

if **converged** break

$g \leftarrow \nabla L(\theta^{(i)})$ $\leftarrow$ for linear/ridge regression, $\lambda w + X^T(Xw - Y)$

$\theta^{(i+1)} \leftarrow \theta^{(i)} - \eta g$

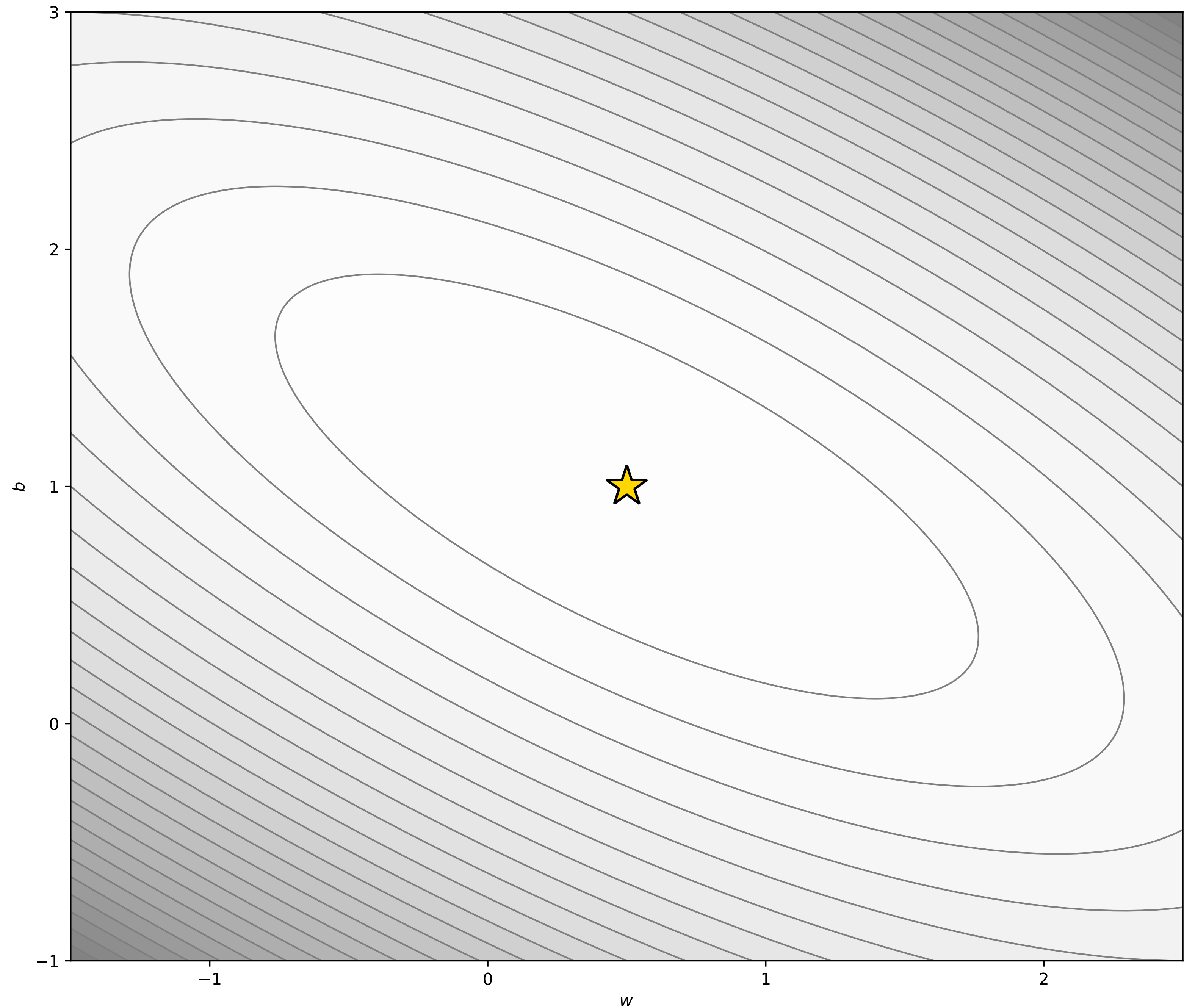
return $\theta^{(i)}$

- Converged: e.g., validation error stops decreasing
- Problem: have to set hyperparameter η
- Variant: $\eta^{(i)}$ (can work better but more hyperparameters)

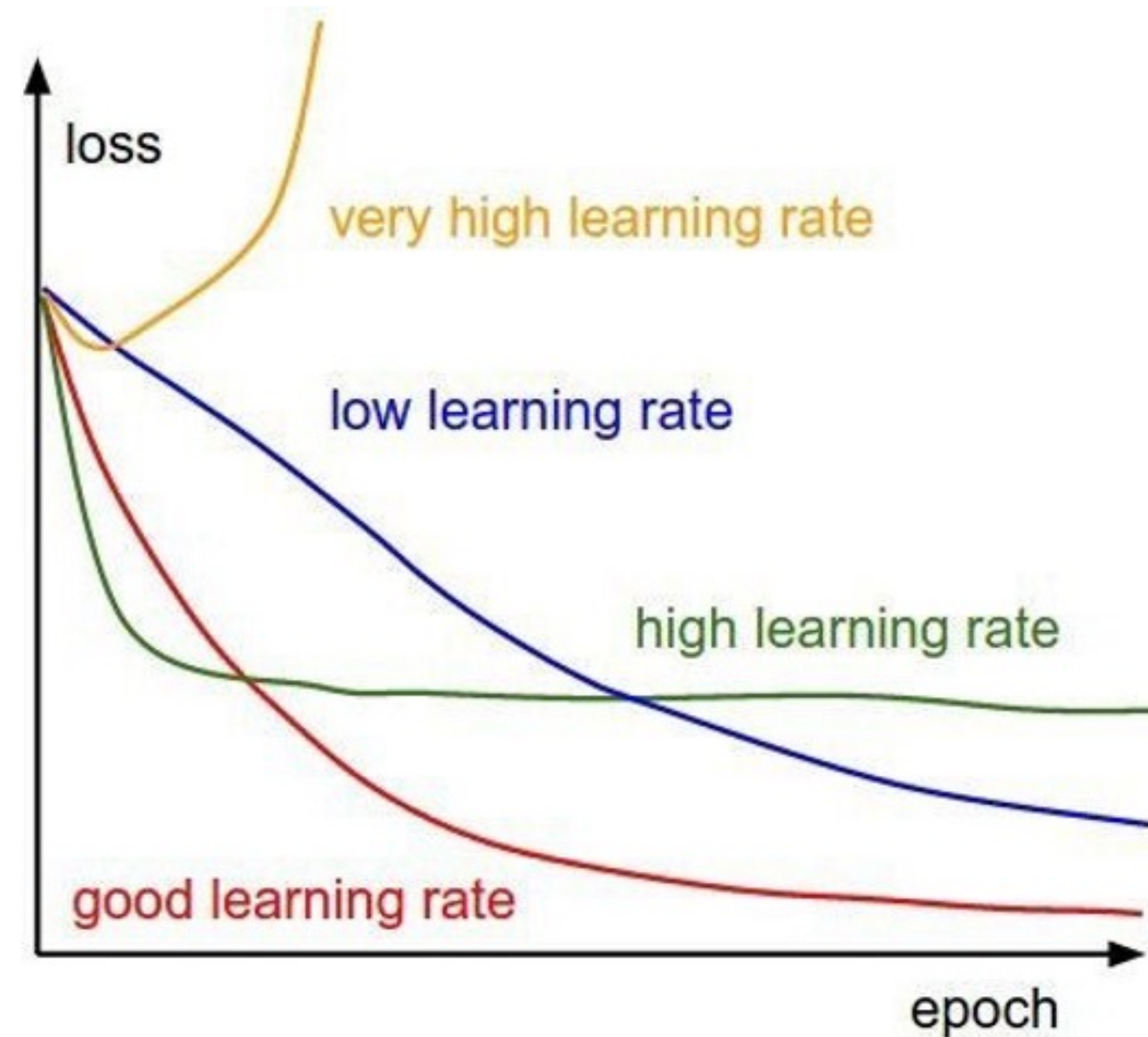
Problem with having to pick learning rate

*Limit on η : **curvature!***

*As long as loss is locally
near linear, we keep
going down; when that
approximation fails we
slow or reverse*



Effect of learning rate



- Learning rate too big: divergence or noisy convergence
- Too small: slow progress
- Can adapt learning rate over time (warmup or anneal) to try to get best of both worlds

Why (not) gradient descent?

- Globally convergent!
- Cost: $O(Nk)$ per iteration (# examples $\times$ # features)
 - ▶ “good enough” answer = not too many iterations (treat as independent of N, k), as long as we can pick η
 - ▶ vs. $O(Nk^2)$ for `linalg.lstsq`
 - ▶ fast for small problems
 - ▶ not fast for large problems, but lower order in k
- Parallelizable: distribute gradient calculation across multiple GPUs or a cluster
- Extensions are available (momentum, preconditioning, parameter server)
- Works for a lot of models beyond just linear regression!

Convexity

- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is convex if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 \leq c \leq 1$$

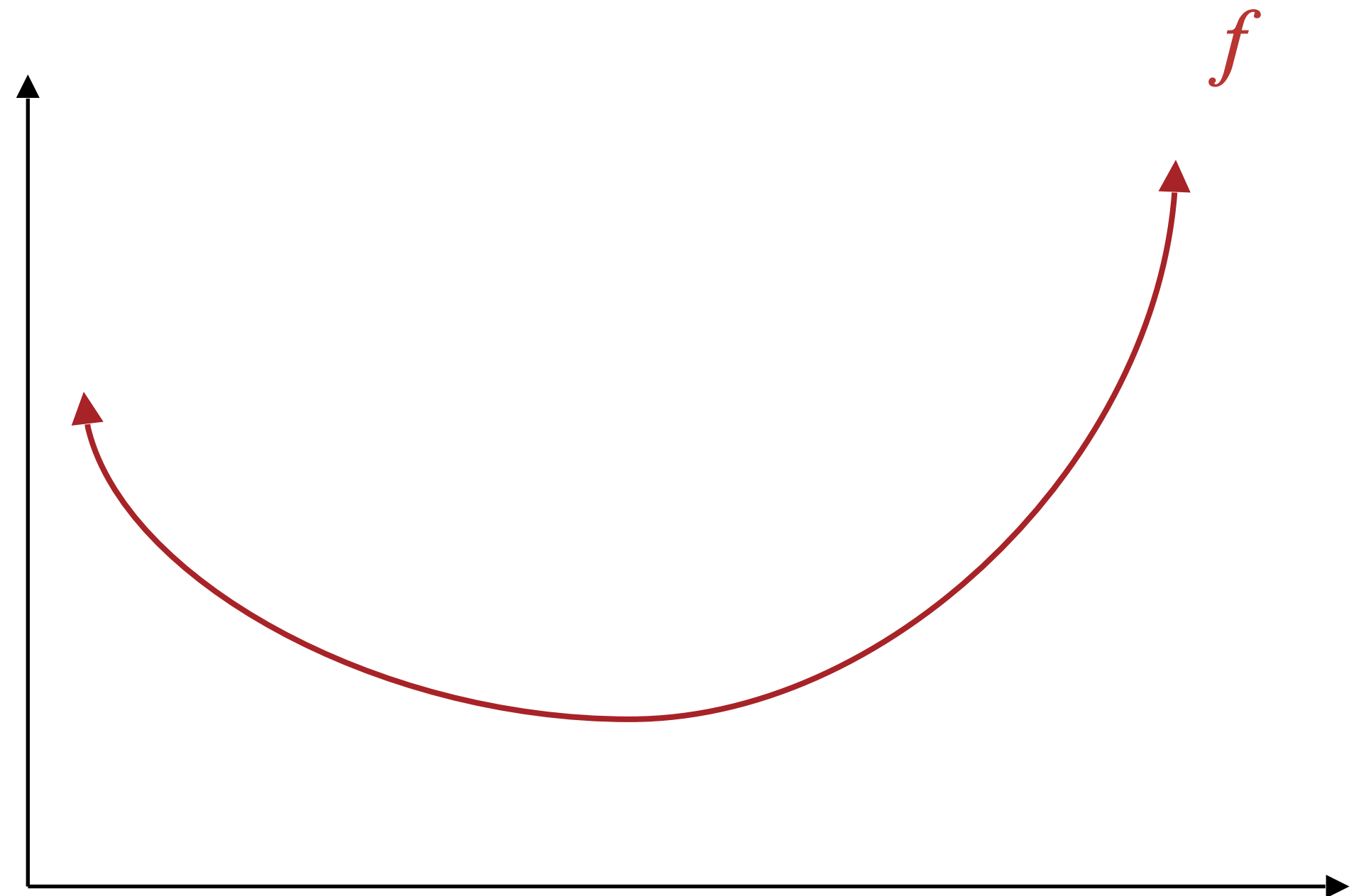
$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)}) \leq cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$

Convexity

- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is convex if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 \leq c \leq 1$$

$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)}) \leq cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$

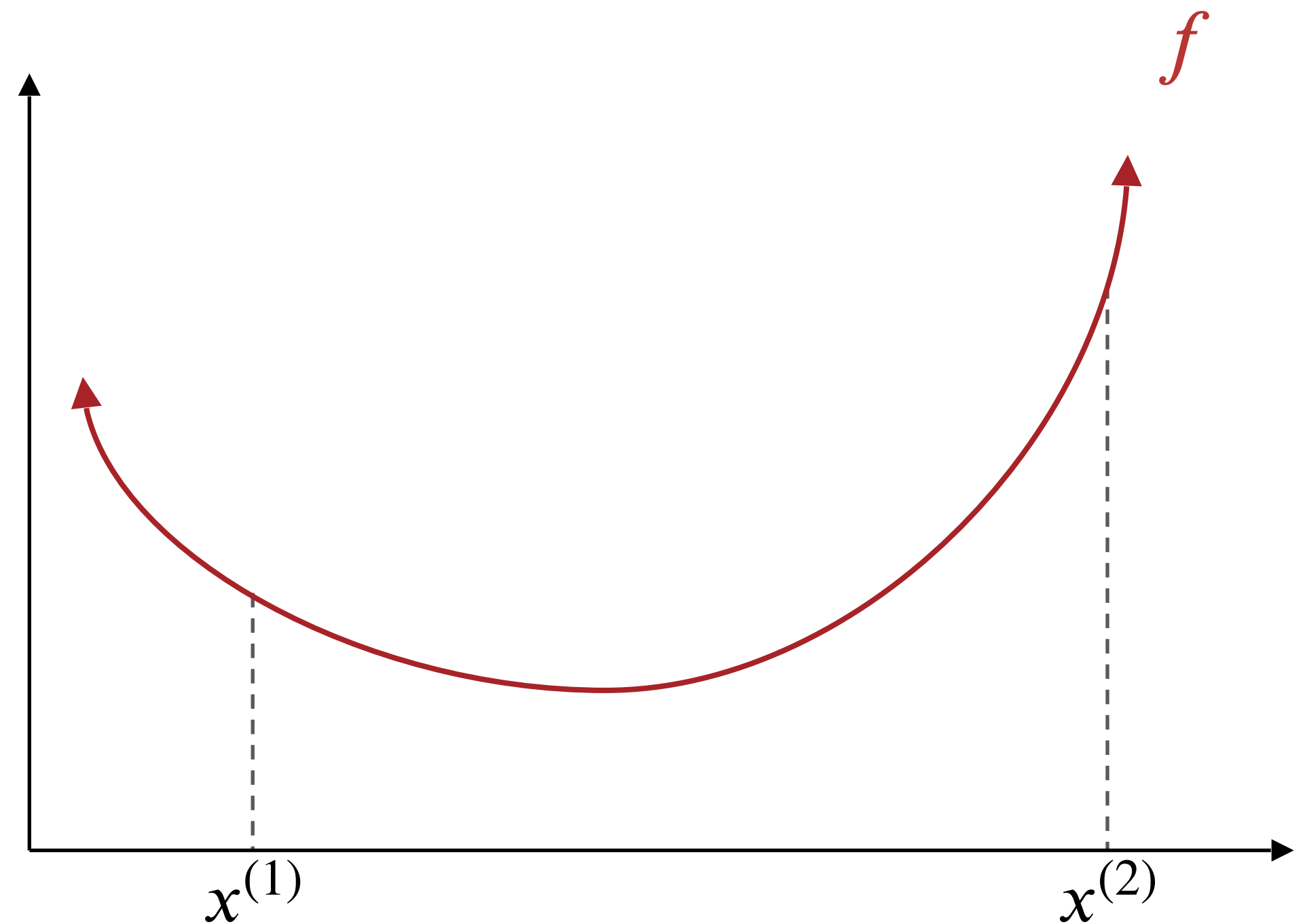


Convexity

- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is convex if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 \leq c \leq 1$$

$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)}) \leq cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$

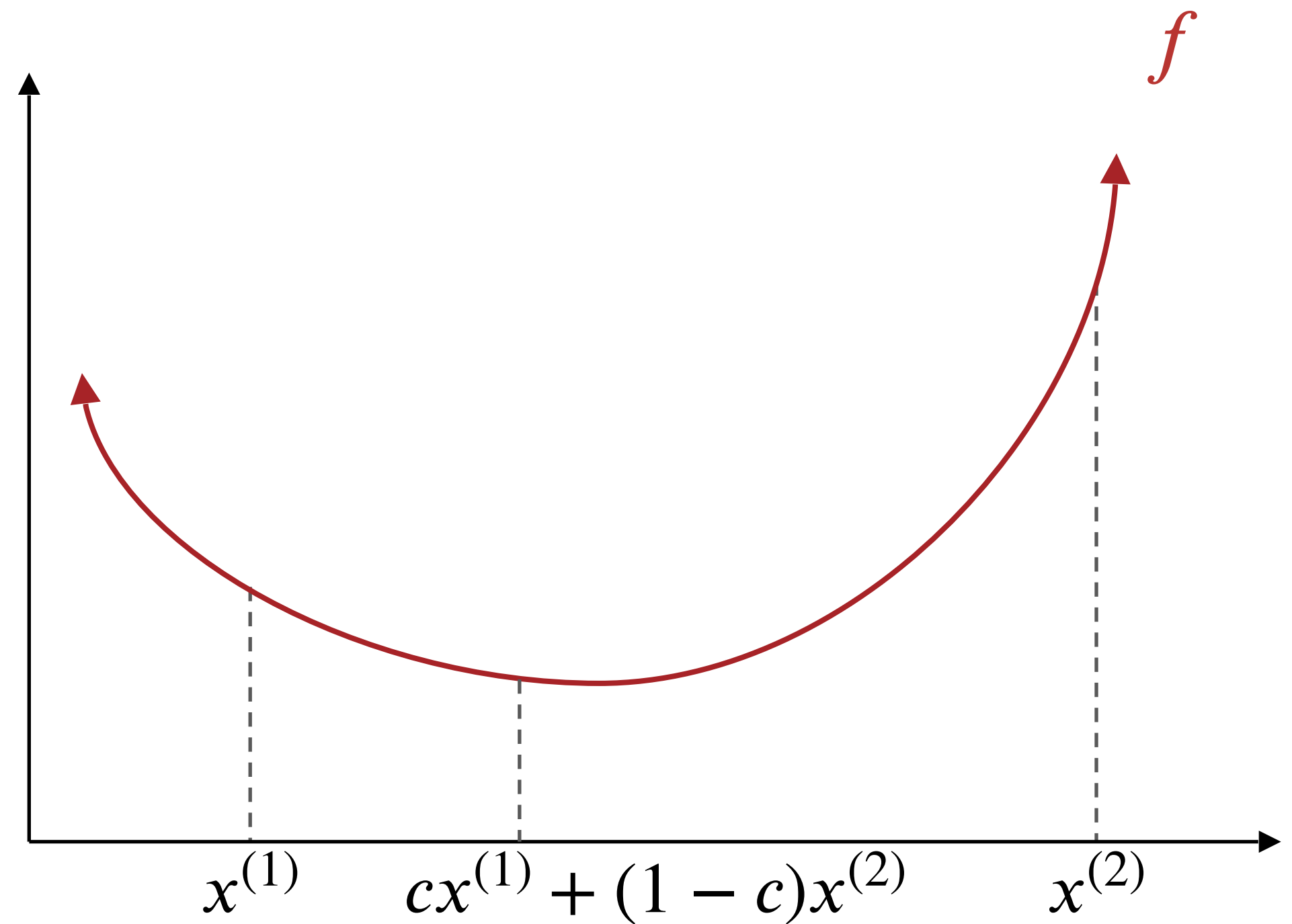


Convexity

- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is convex if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 \leq c \leq 1$$

$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)}) \leq cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$

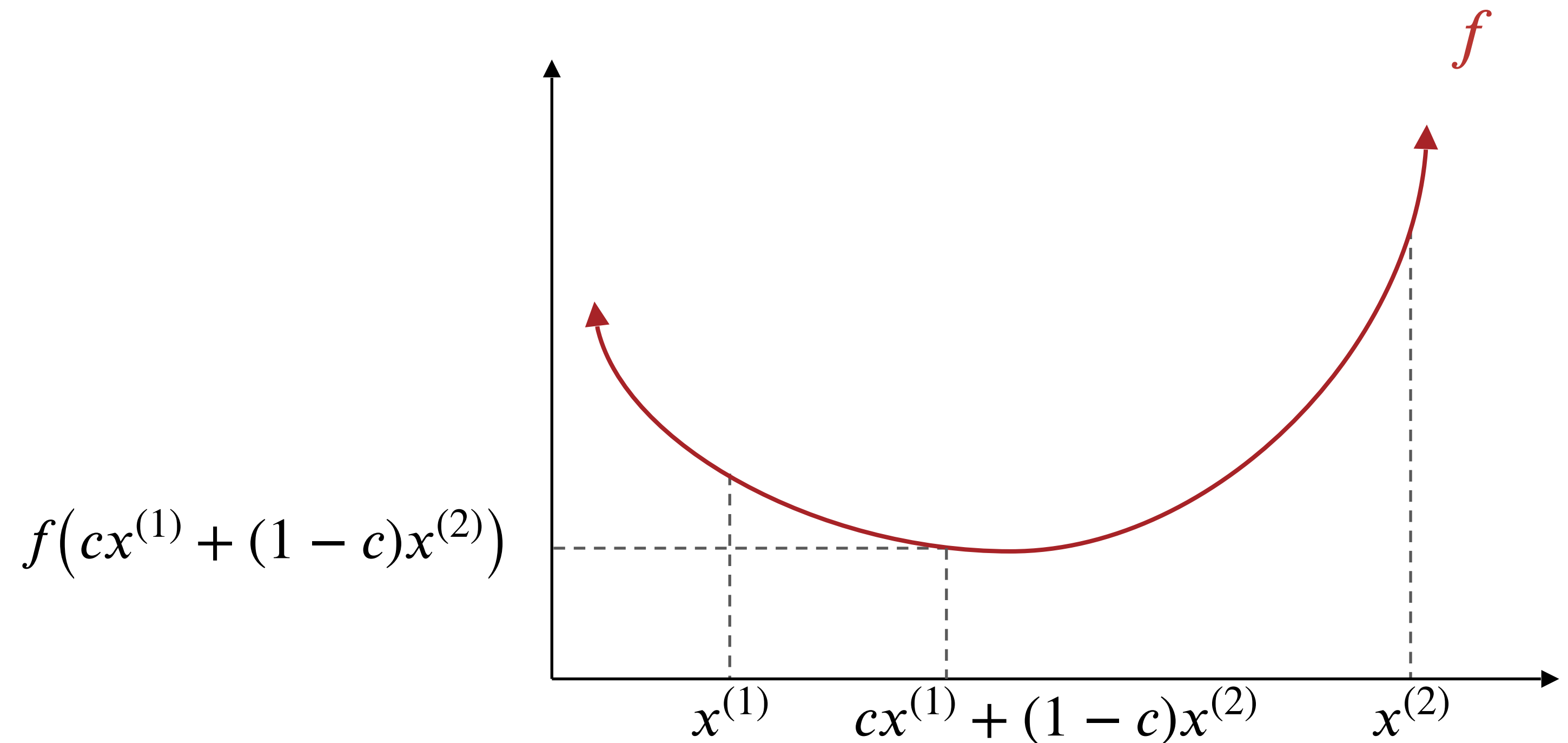


Convexity

- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is convex if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 \leq c \leq 1$$

$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)}) \leq cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$

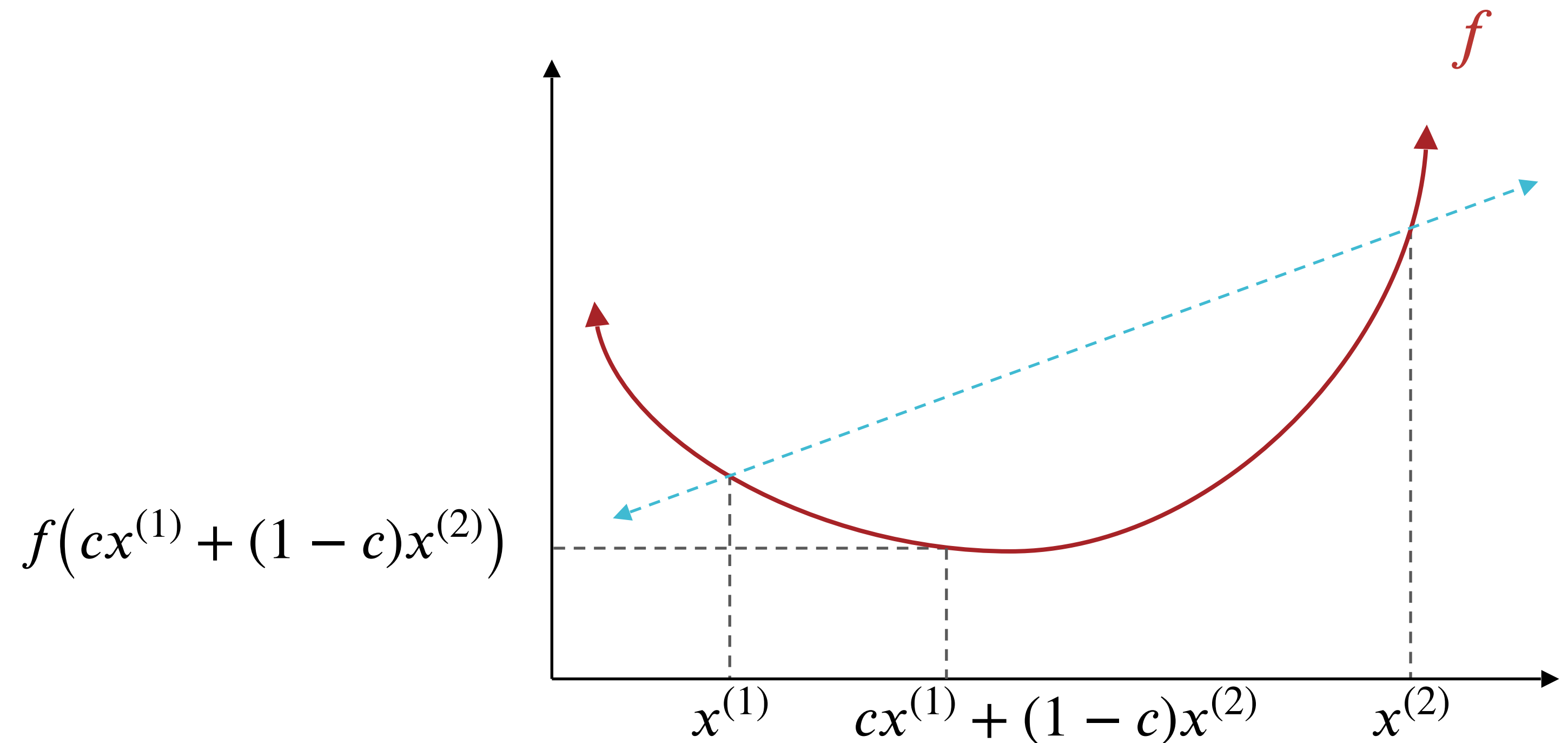


Convexity

- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is convex if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 \leq c \leq 1$$

$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)}) \leq cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$

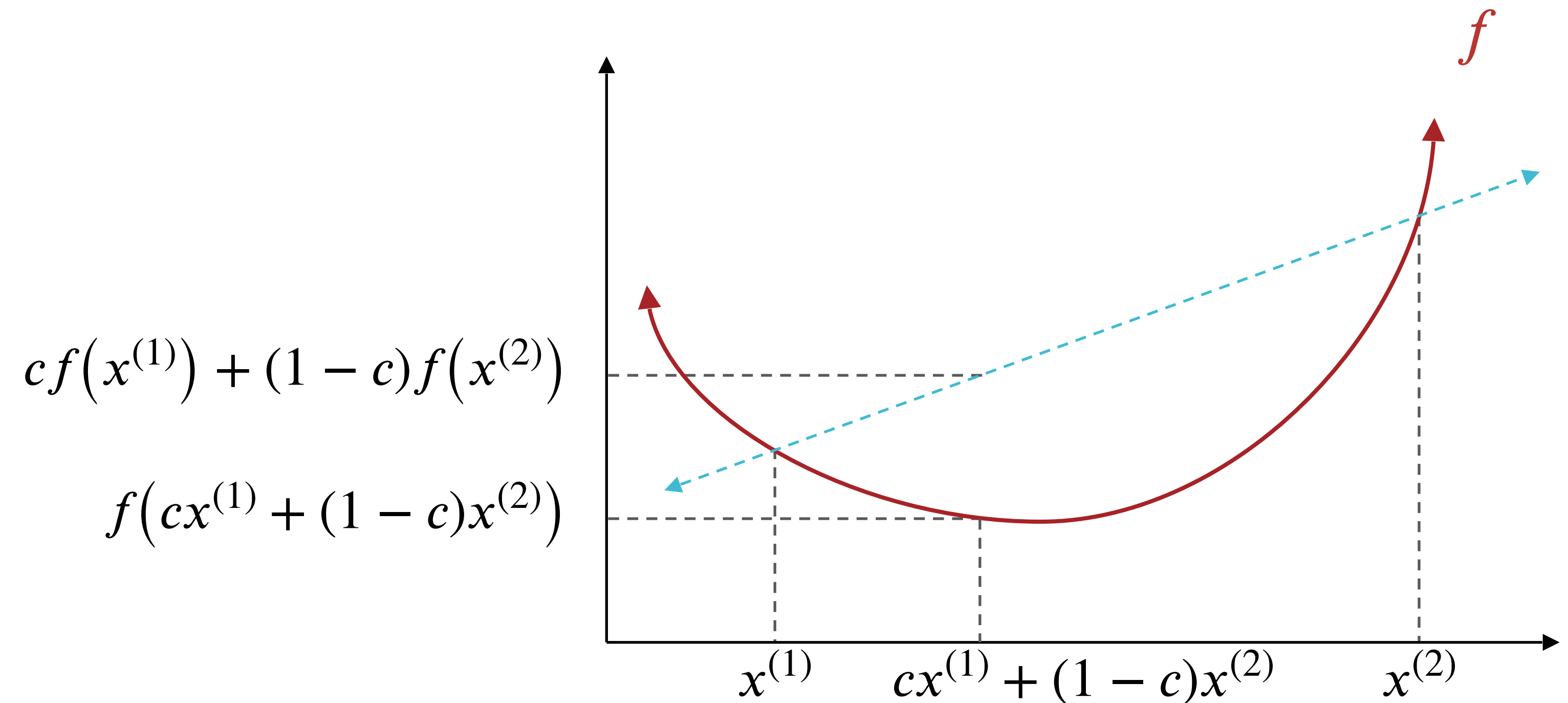


Convexity

- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is convex if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 \leq c \leq 1$$

$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)}) \leq cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$

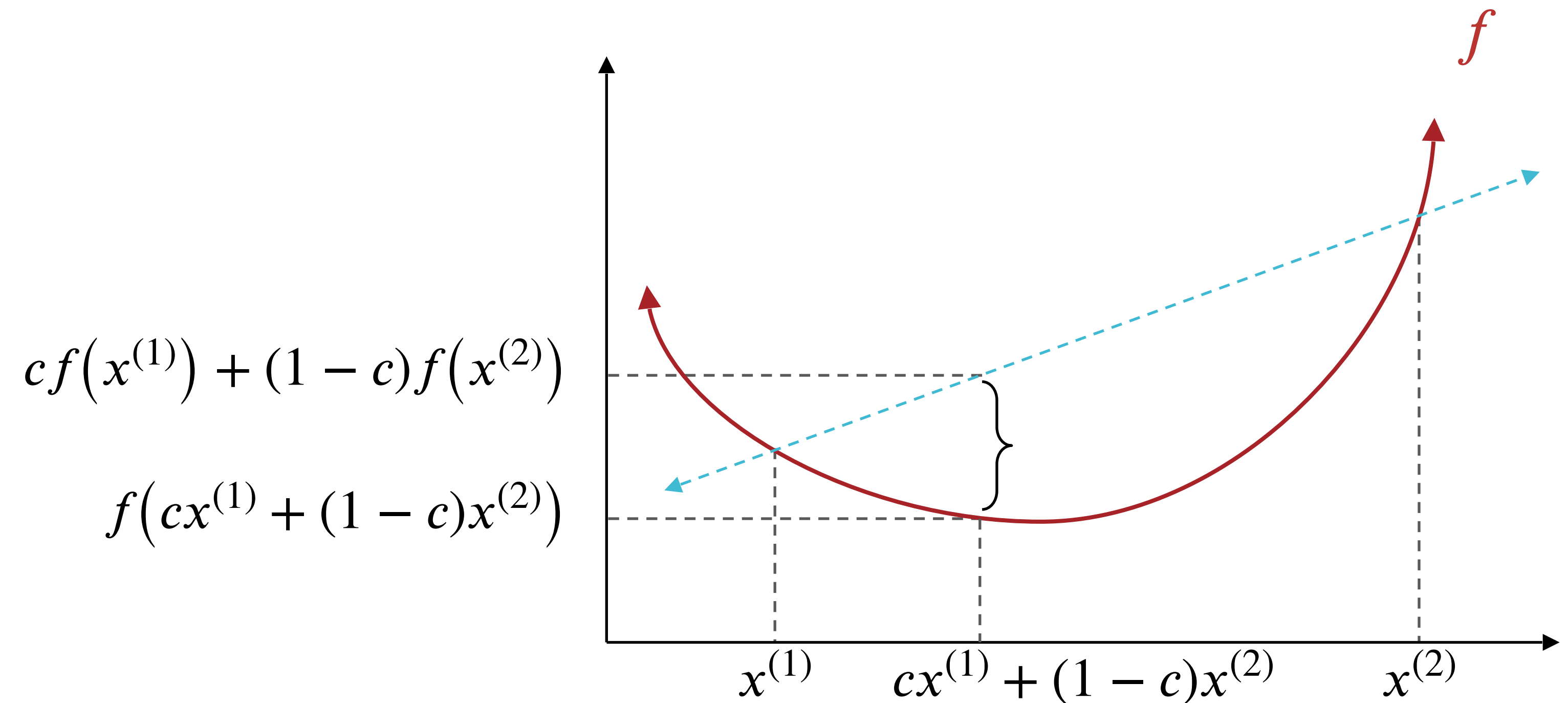


Convexity

- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is convex if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 \leq c \leq 1$$

$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)}) \leq cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$

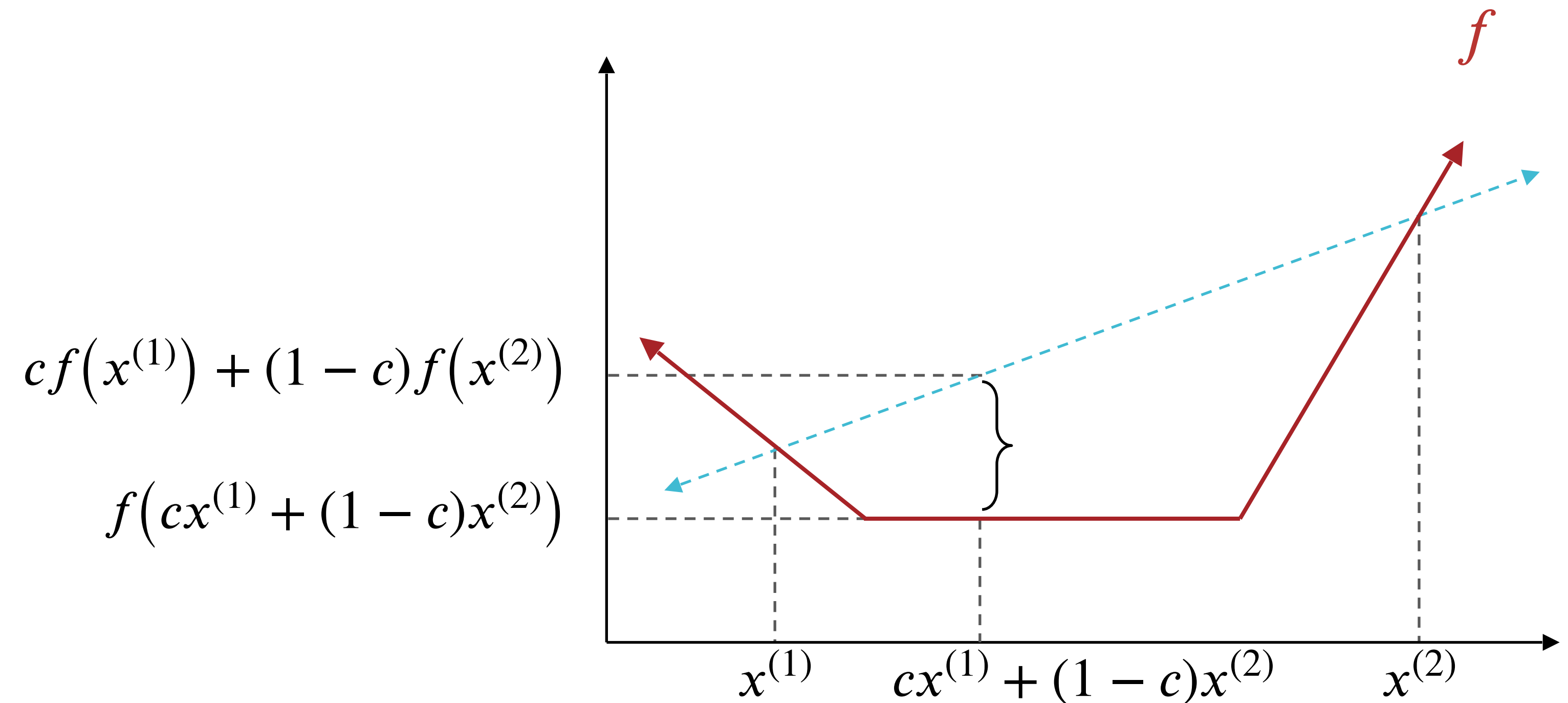


Convexity

- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is convex if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 \leq c \leq 1$$

$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)}) \leq cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$

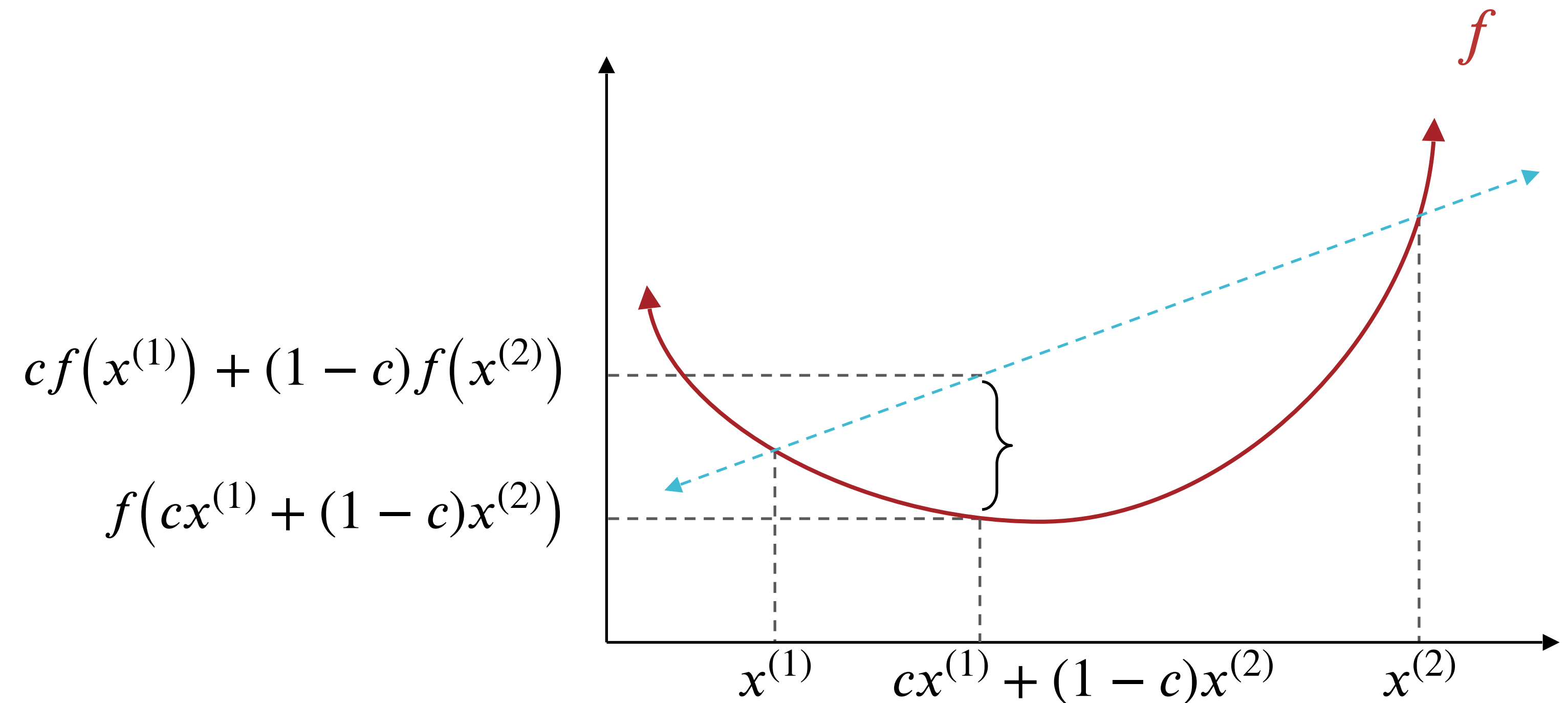


Convexity

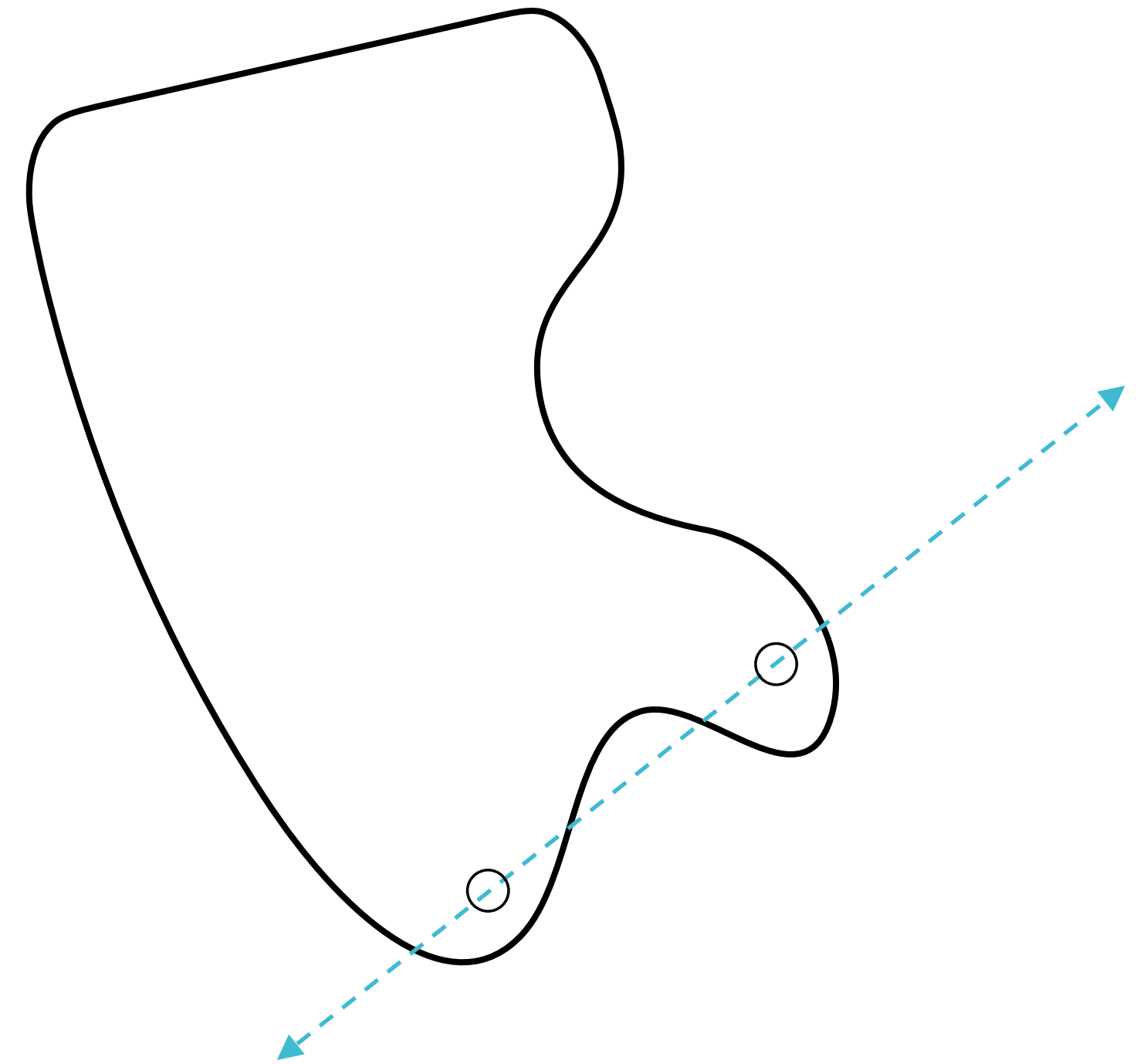
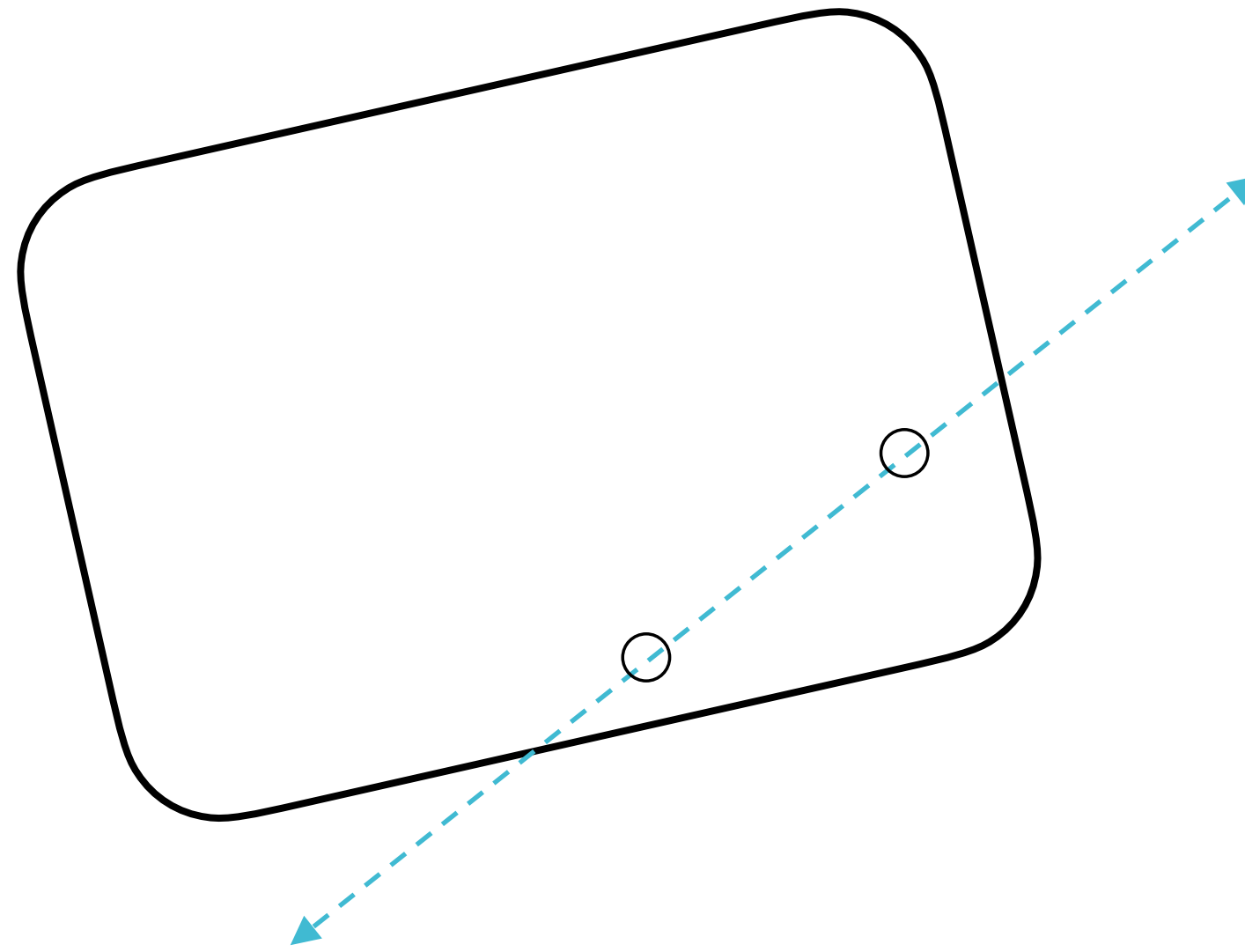
- A function $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is *strictly convex* if

$$\forall \mathbf{x}^{(1)} \in \mathbb{R}^D, \mathbf{x}^{(2)} \in \mathbb{R}^D \text{ and } 0 < c < 1$$

$$f(c\mathbf{x}^{(1)} + (1 - c)\mathbf{x}^{(2)}) < cf(\mathbf{x}^{(1)}) + (1 - c)f(\mathbf{x}^{(2)})$$

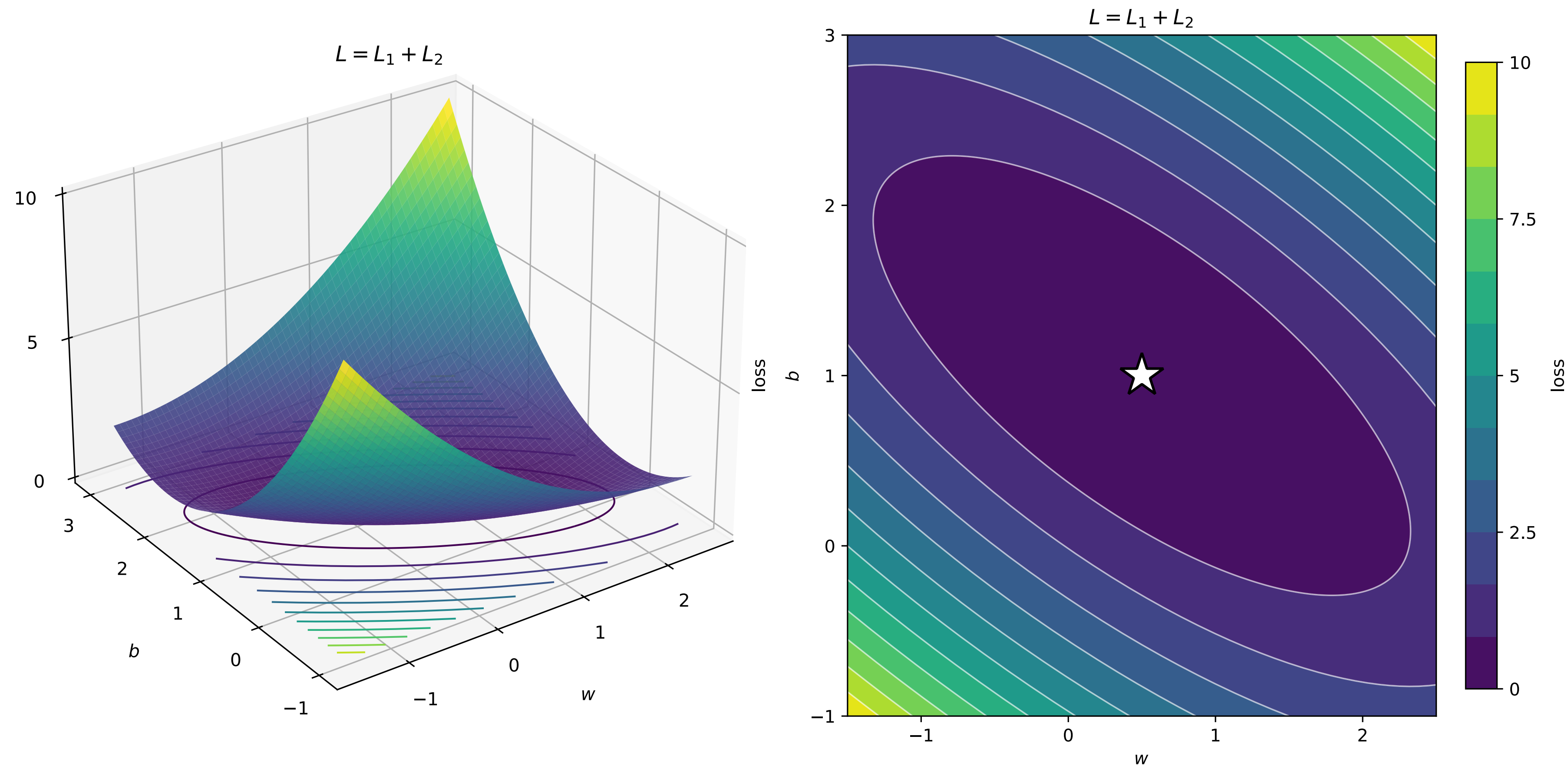


Convex sets



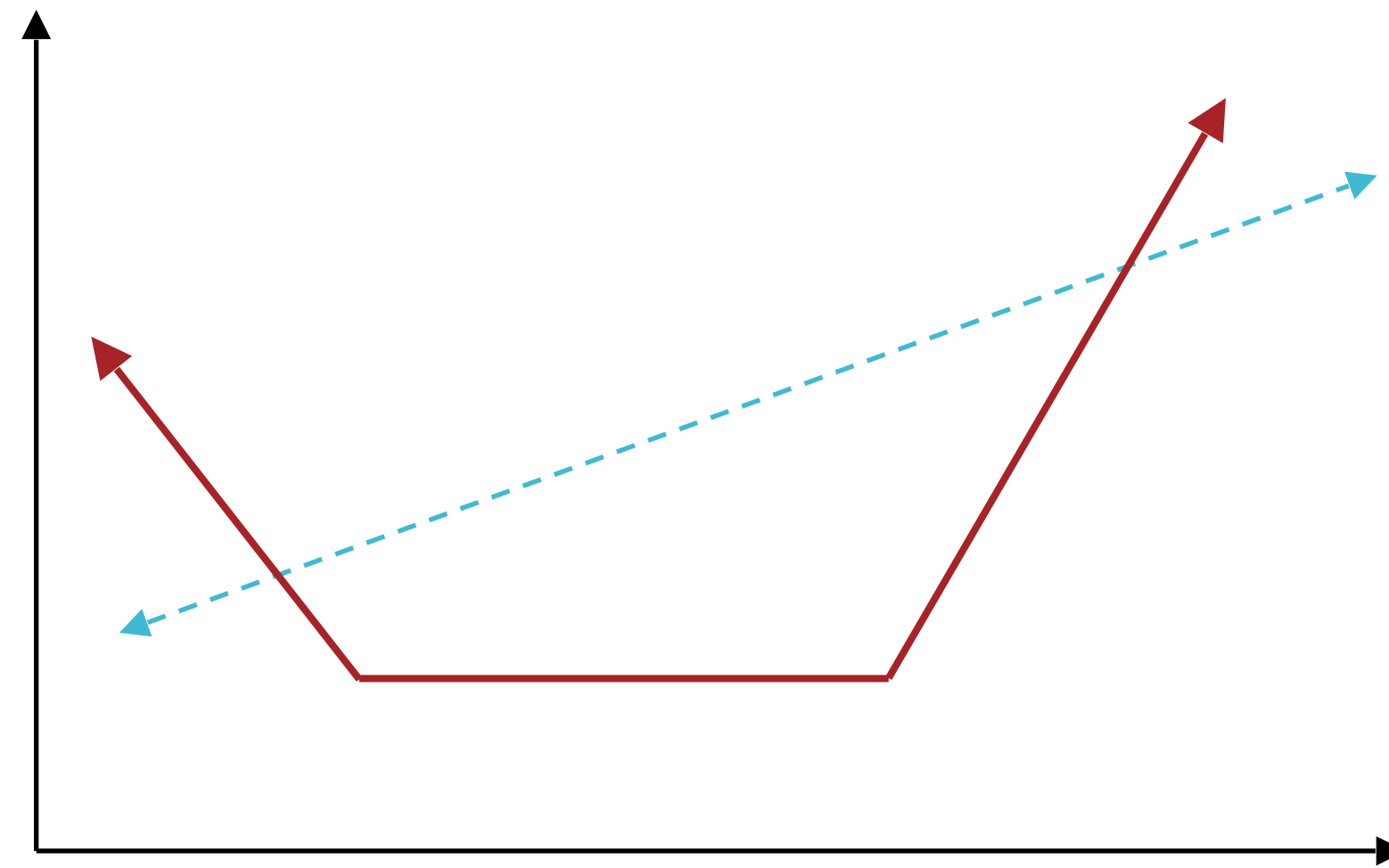
- Similar idea for sets: convex if any line segment between two points in the set stays in the set

Convexity and contour plots



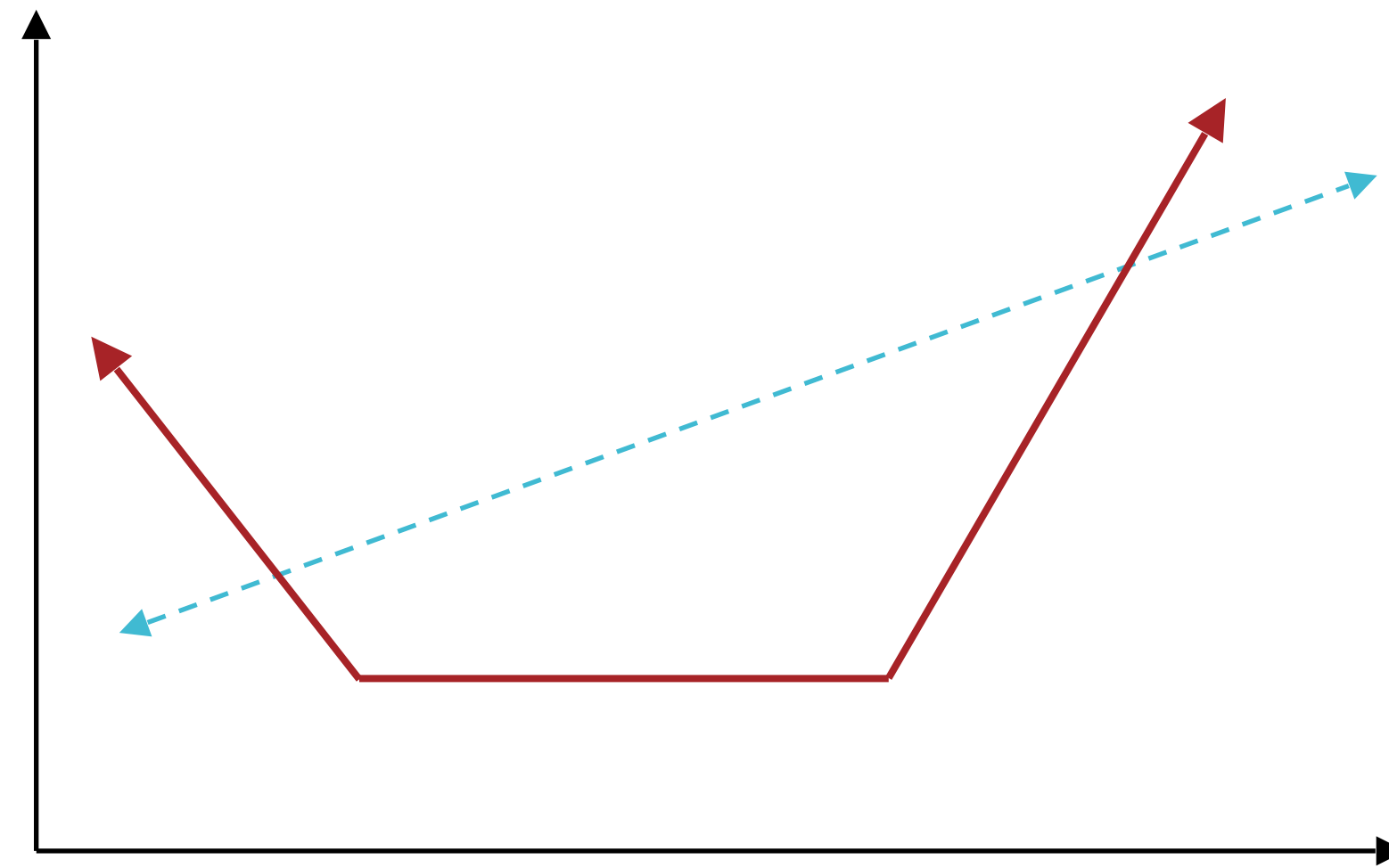
- If f convex, its sublevel sets $(f(x) \leq k)$ are convex
 - ▶ draw line segment between any two points on a contour
 - ▶ by function convexity, function stays below endpoints (all points on segment are inside sublevel set)

Convexity: local and global minima

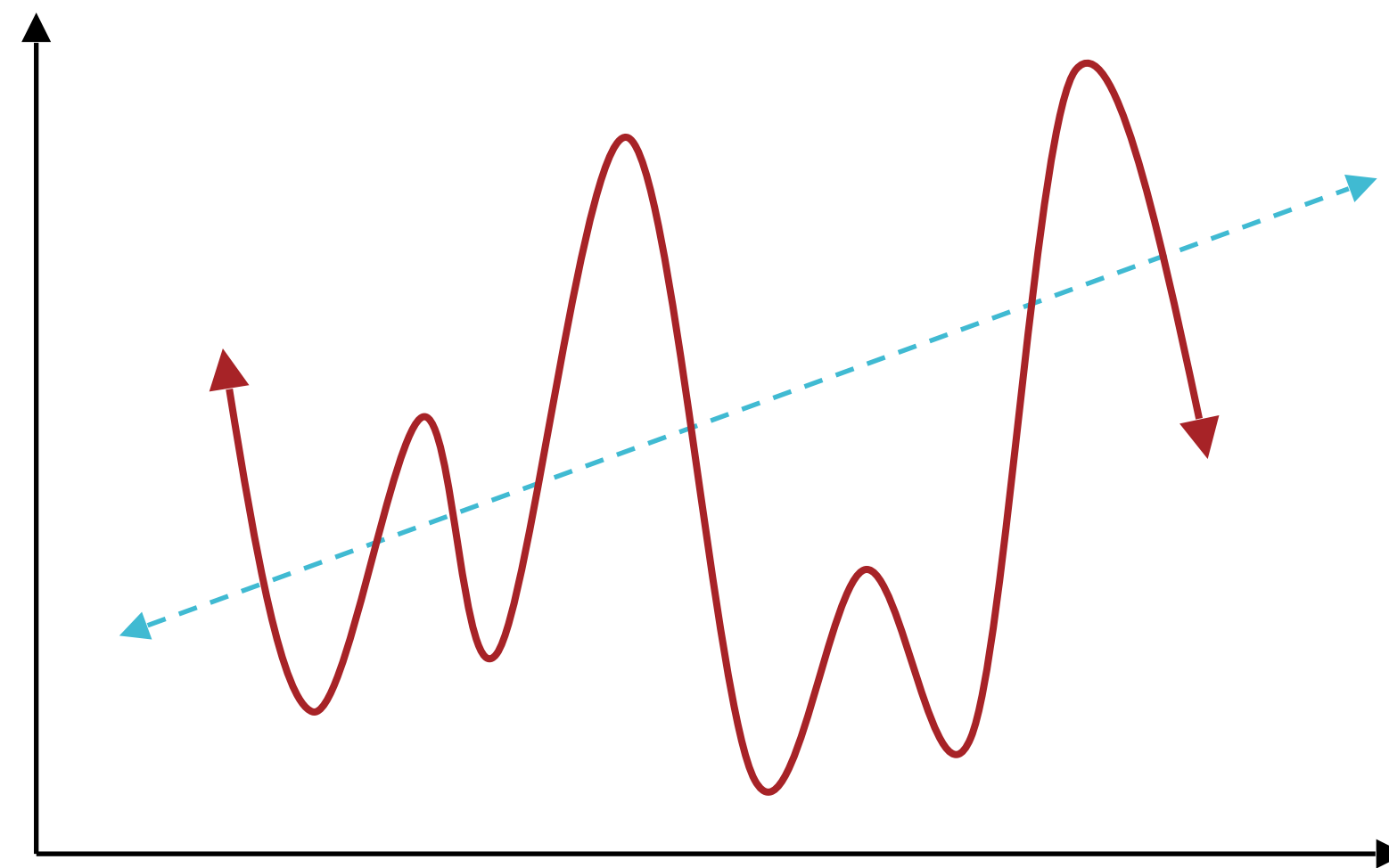


Convex functions

Convexity: local and global minima

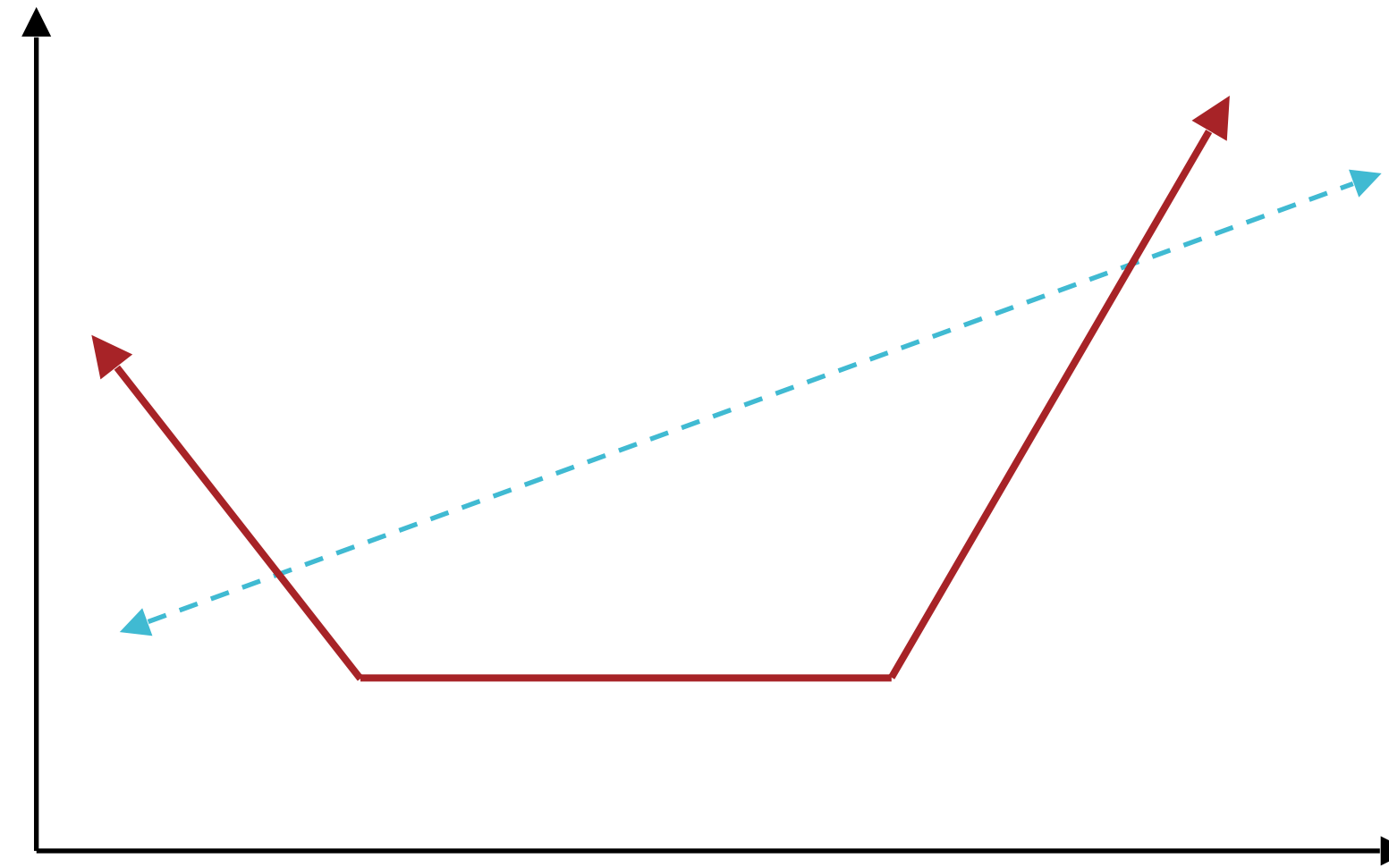


Convex functions

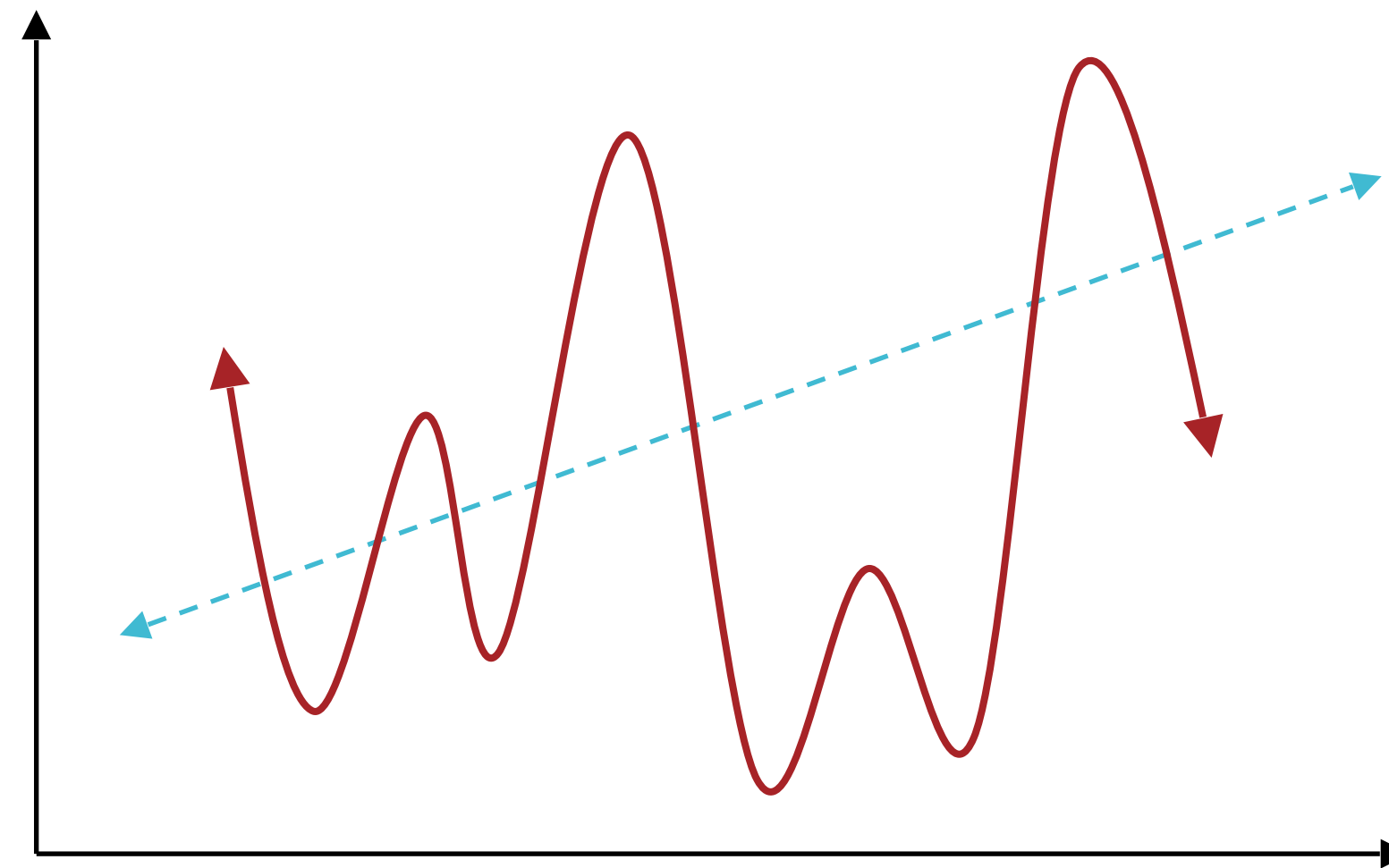


Non-convex functions

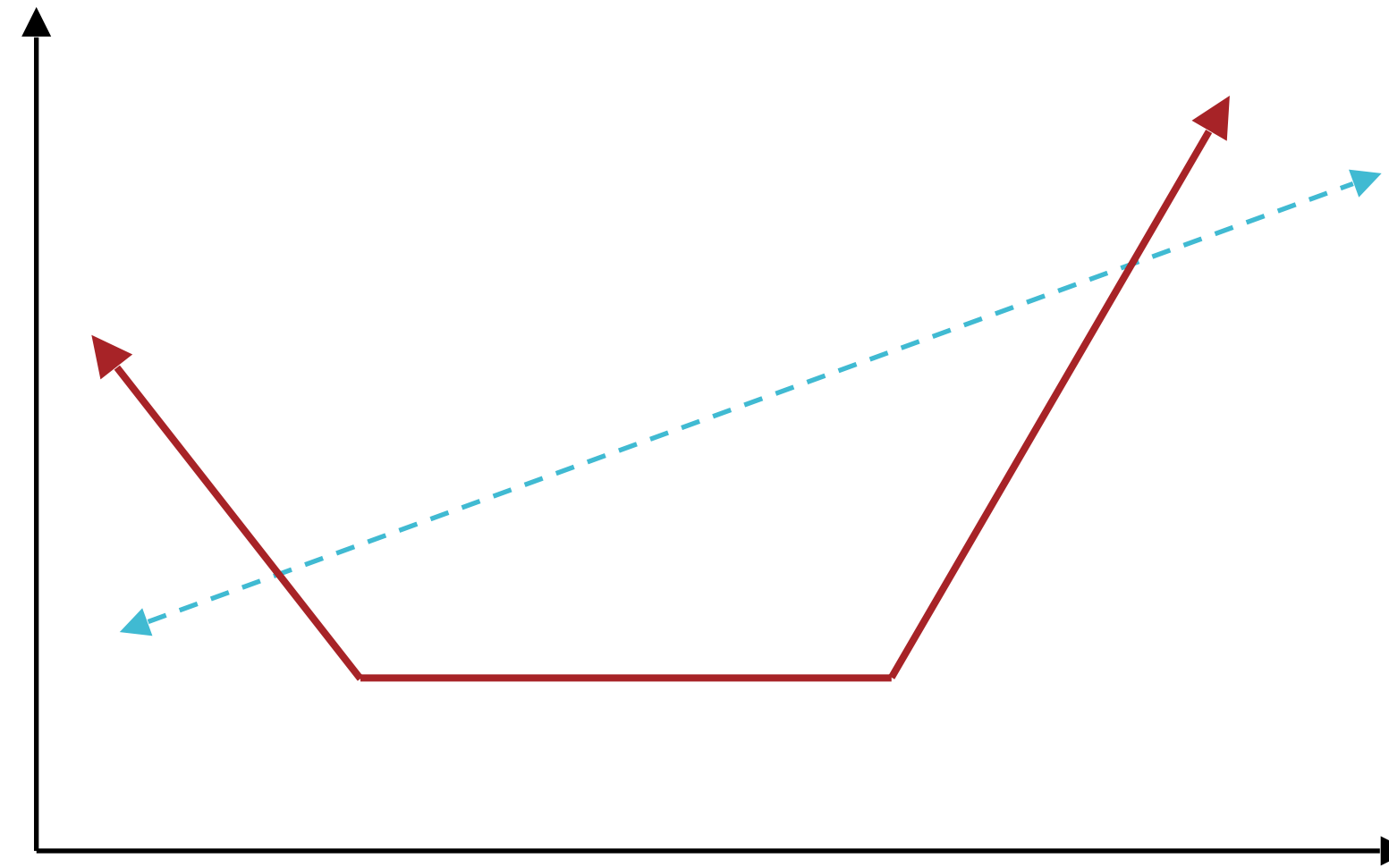
Convexity: local and global minima



Given a function $f: \mathbb{R}^D \rightarrow \mathbb{R}$

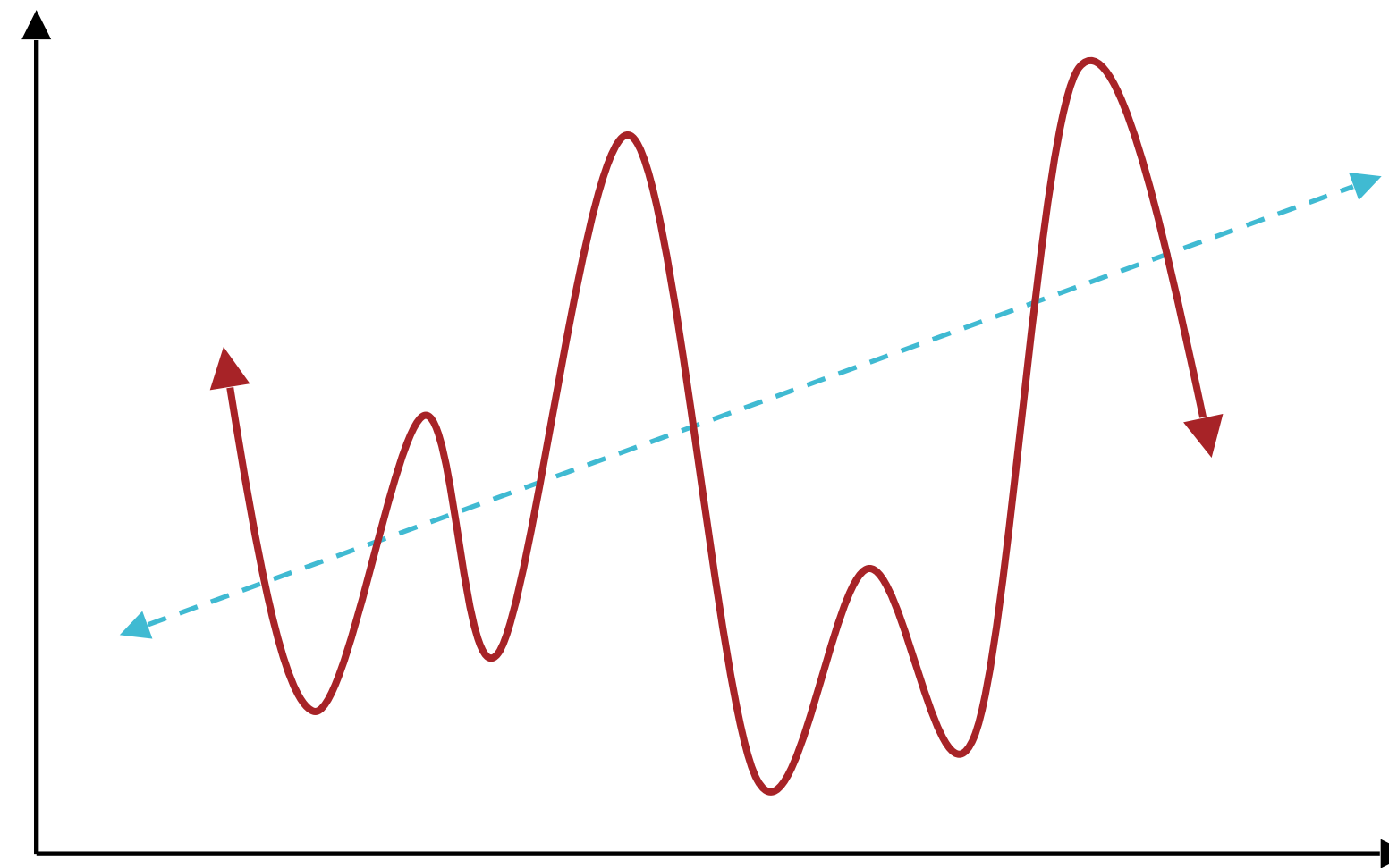


Convexity: local and global minima

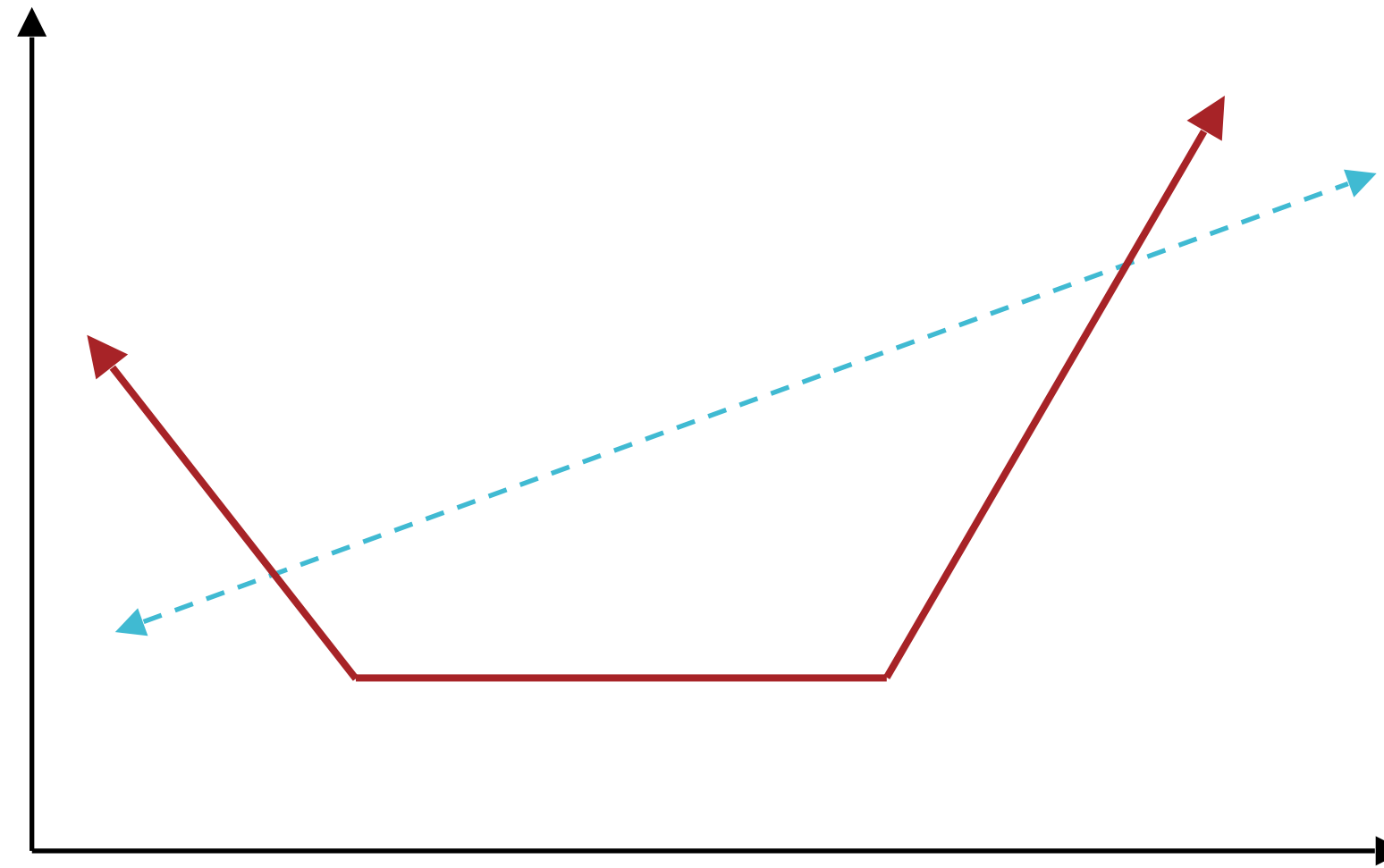


Given a function $f: \mathbb{R}^D \rightarrow \mathbb{R}$

- $\mathbf{x}^*$ is a global minimum iff
$$f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \forall \mathbf{x} \in \mathbb{R}^D$$

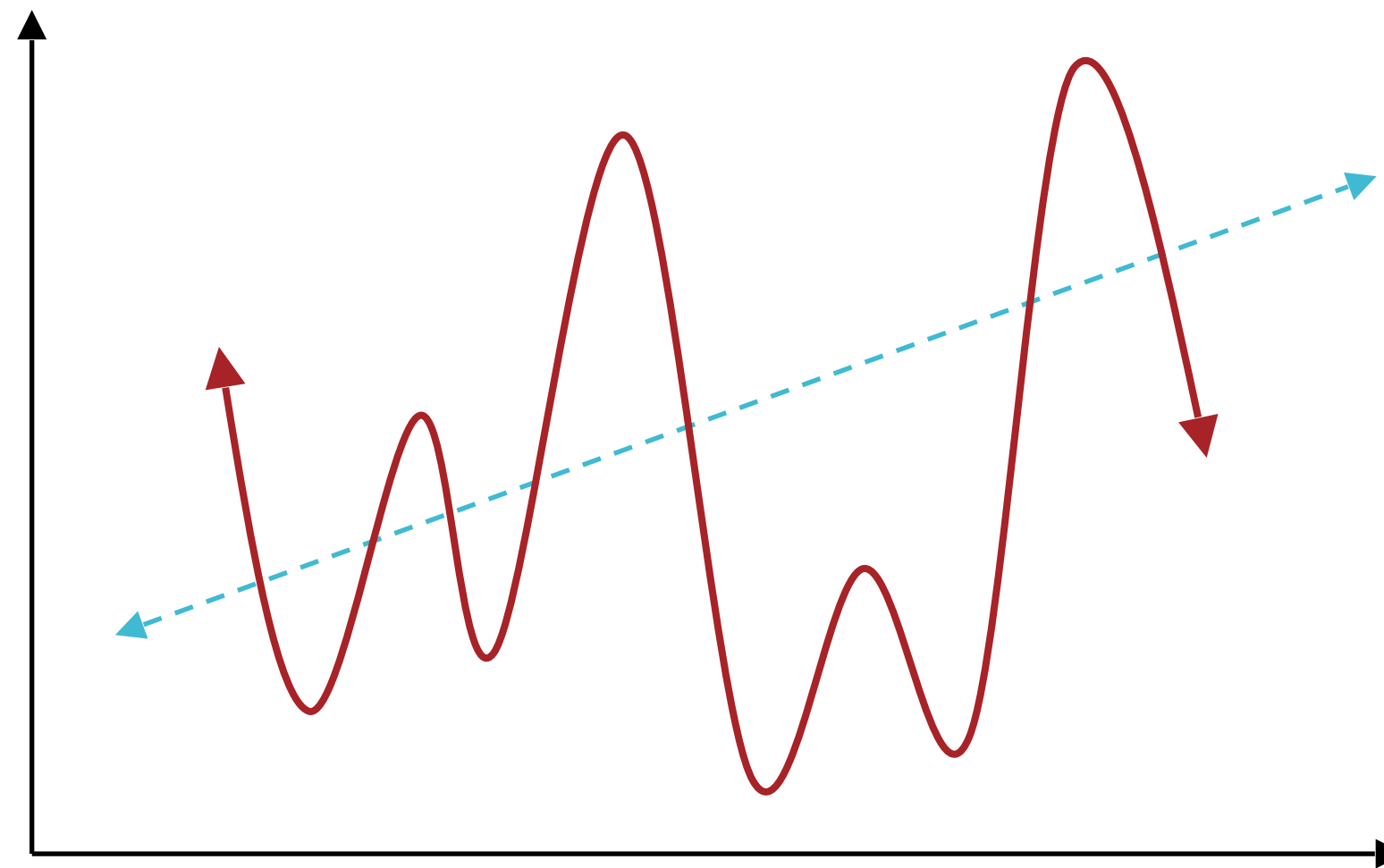


Convexity: local and global minima



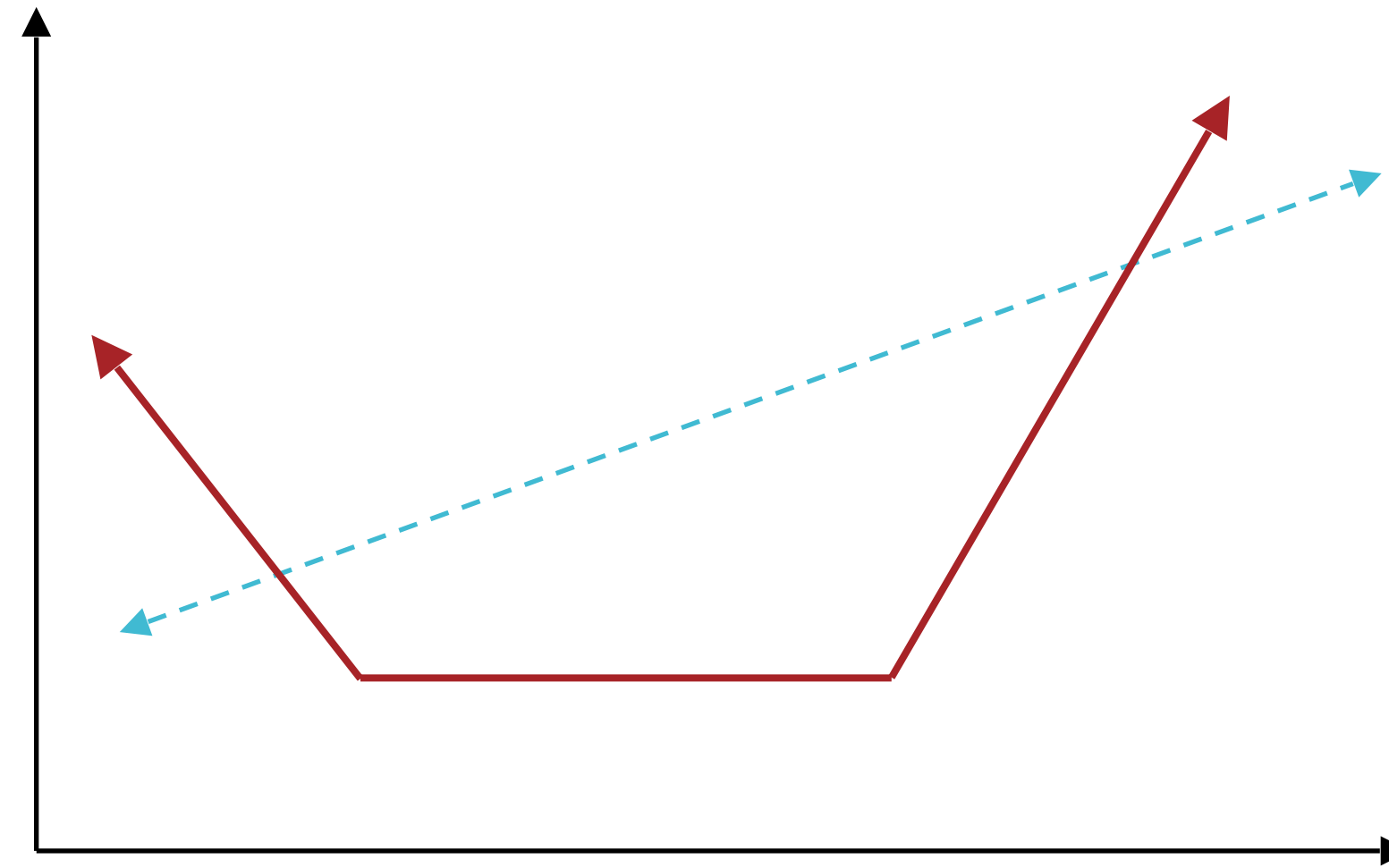
Given a function $f: \mathbb{R}^D \rightarrow \mathbb{R}$

- $\mathbf{x}^*$ is a global minimum iff
$$f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \forall \mathbf{x} \in \mathbb{R}^D$$

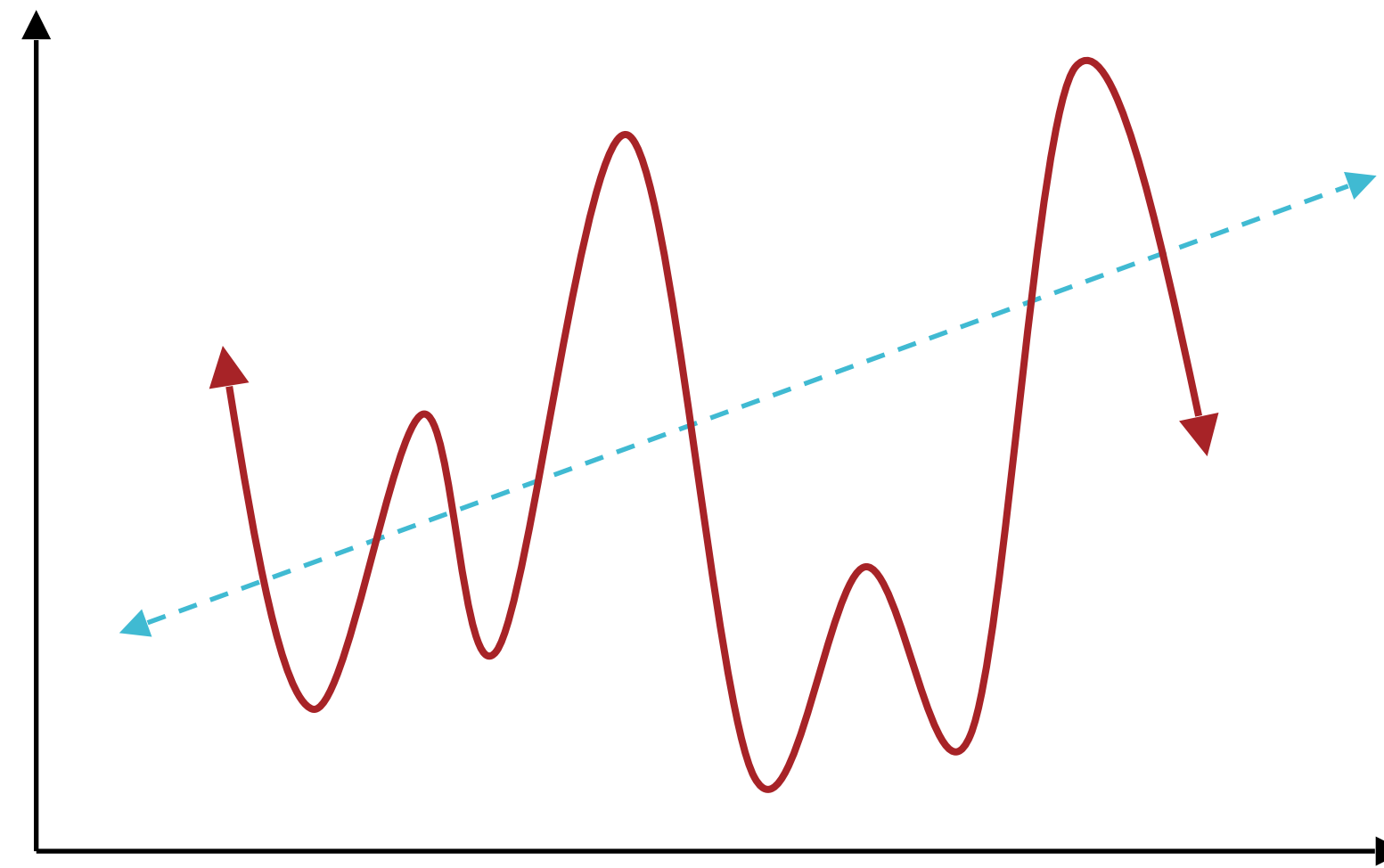


- $\mathbf{x}^*$ is a local minimum iff
$$\exists \epsilon \text{ s.t. } f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \forall \mathbf{x} \text{ s.t. } \|\mathbf{x} - \mathbf{x}^*\|_2 < \epsilon$$

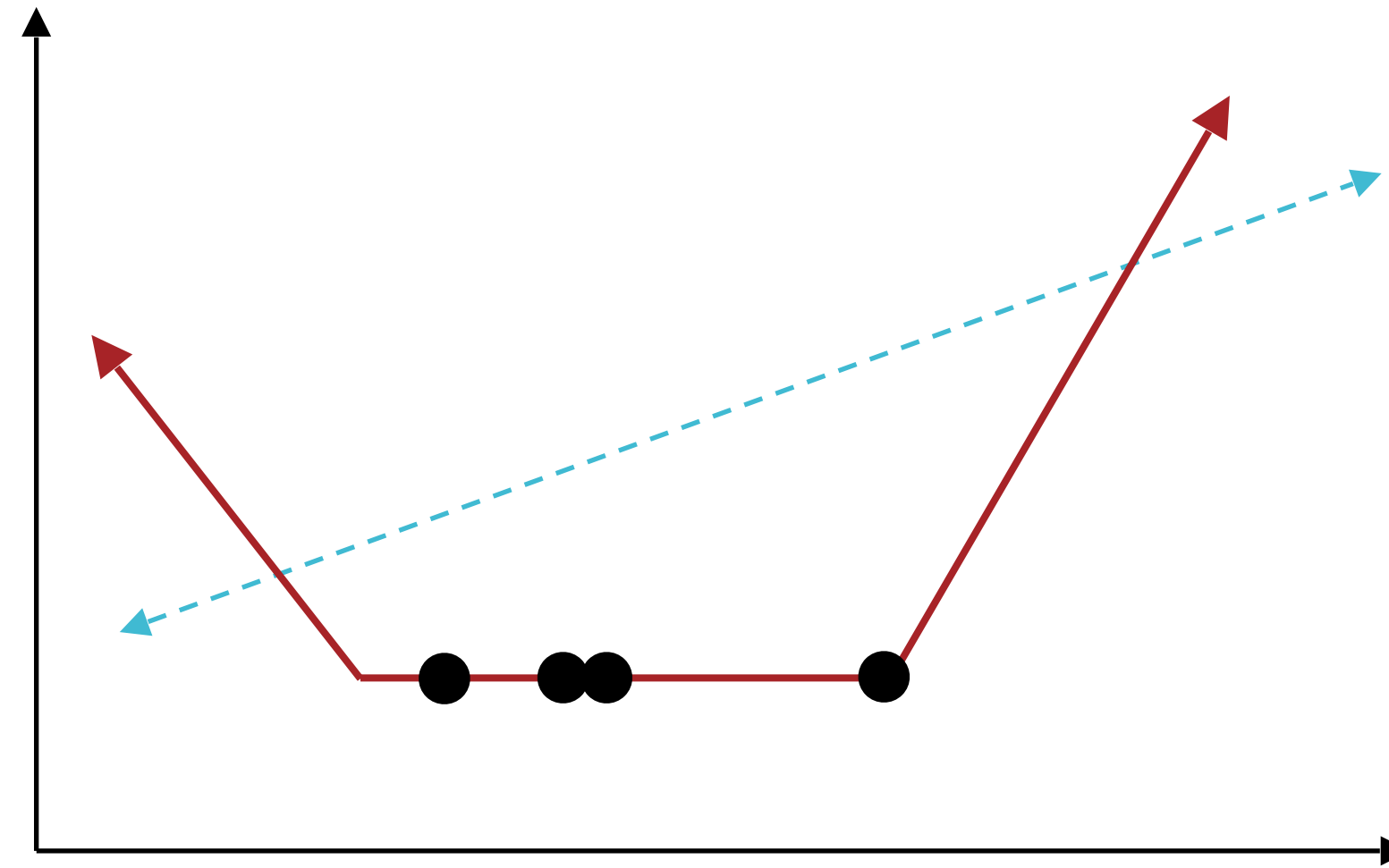
Convexity: local and global minima



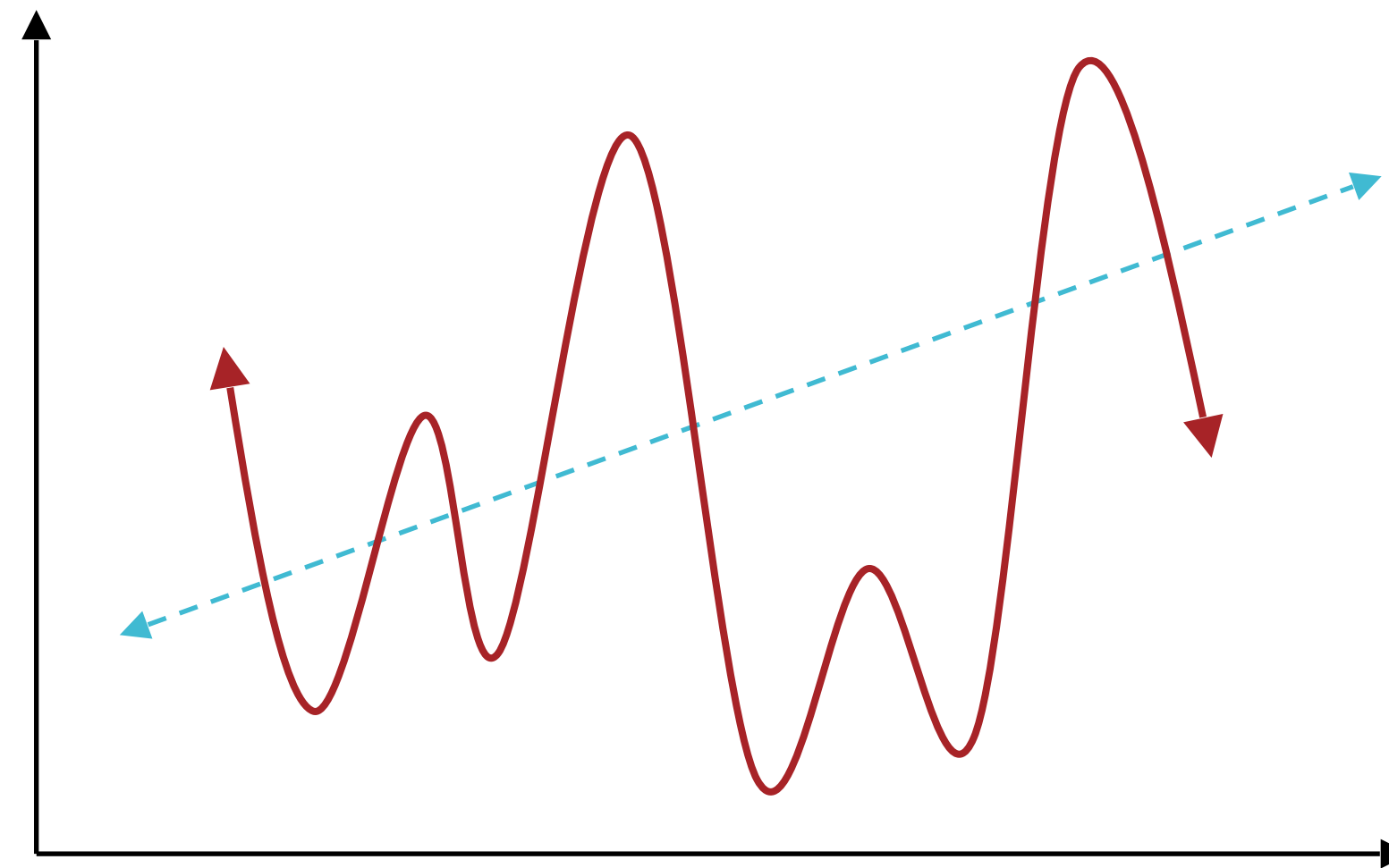
Convex functions:
Each local minimum is a
global minimum!



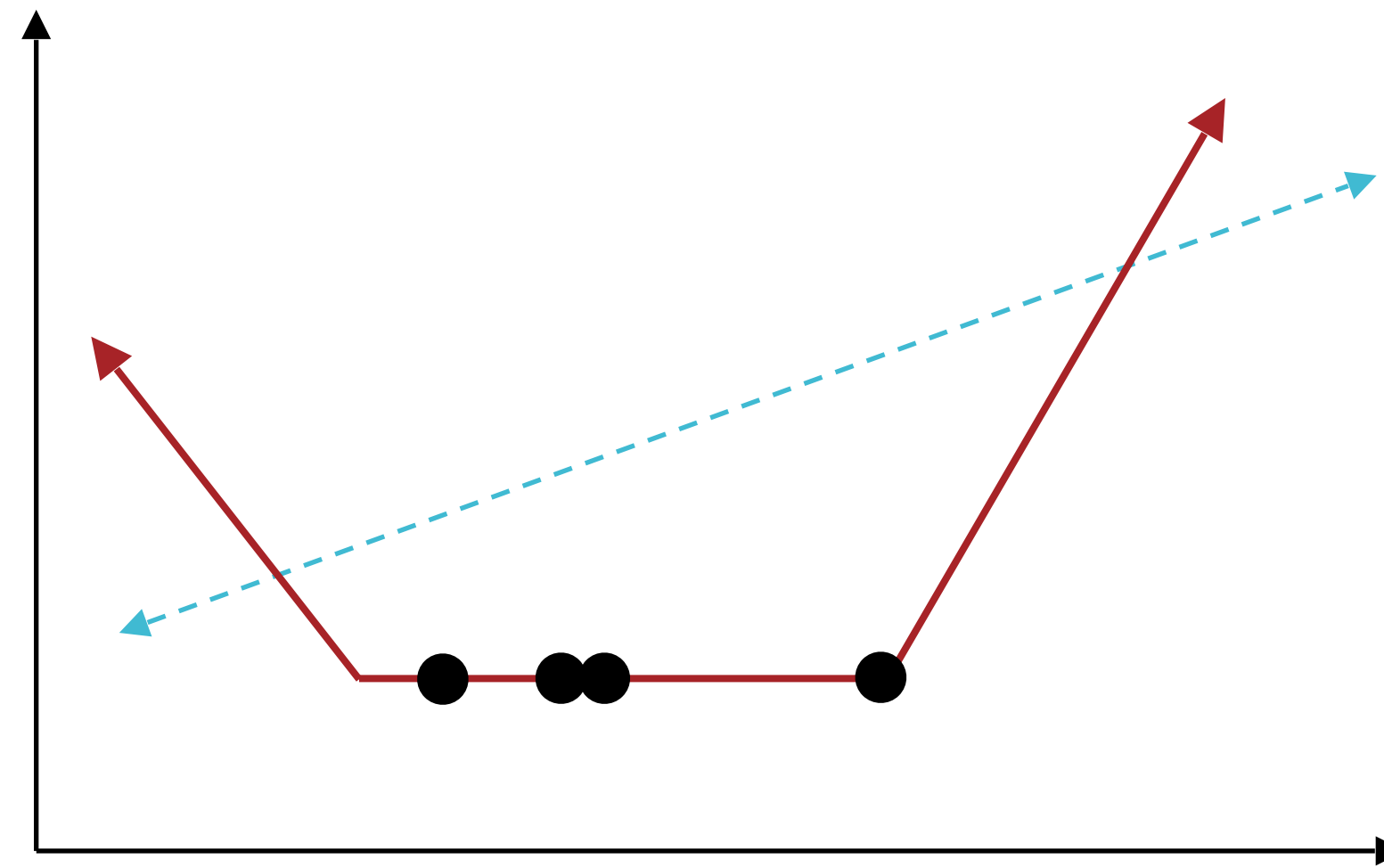
Convexity: local and global minima



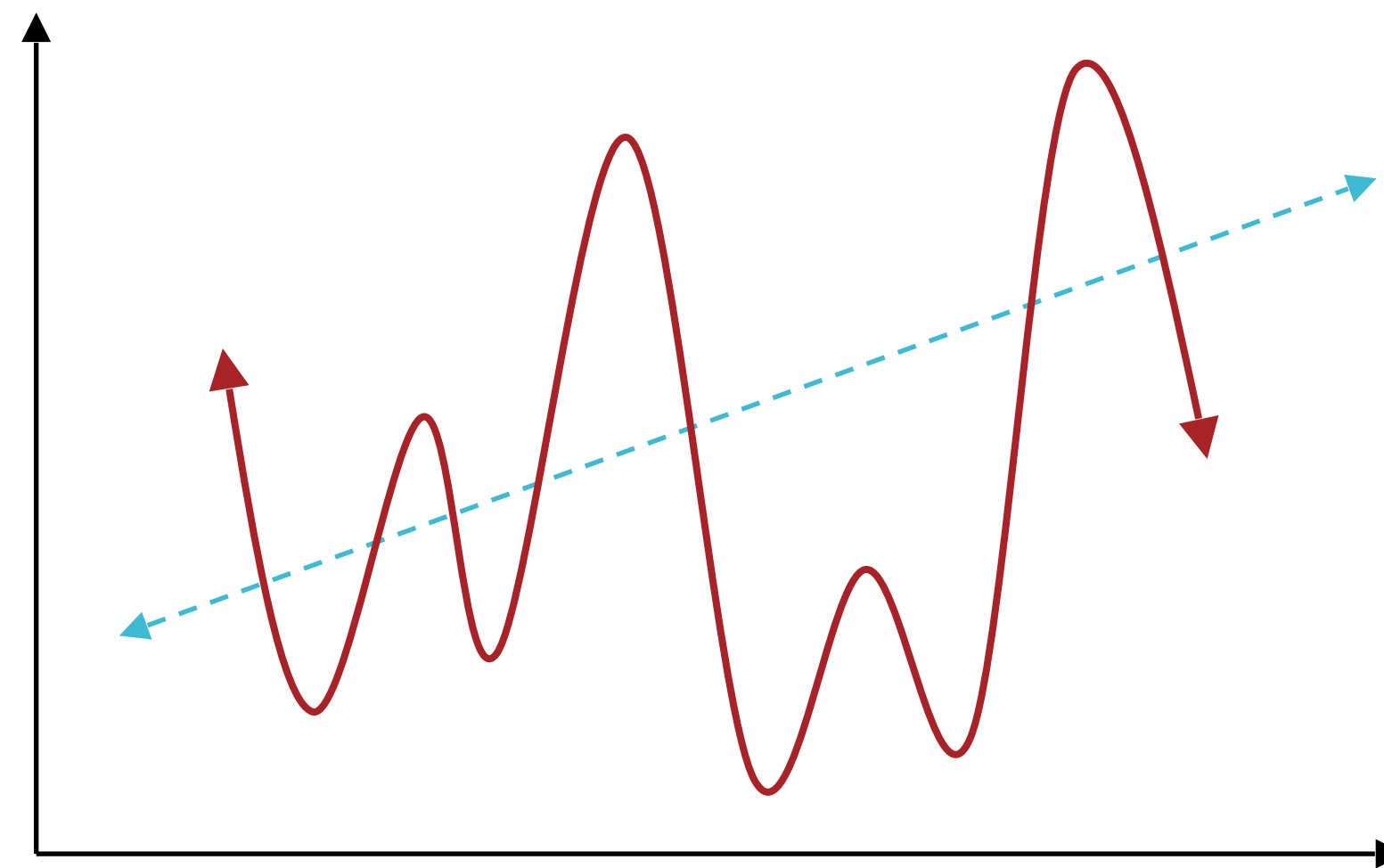
Convex functions:
Each local minimum is a
global minimum!



Convexity: local and global minima

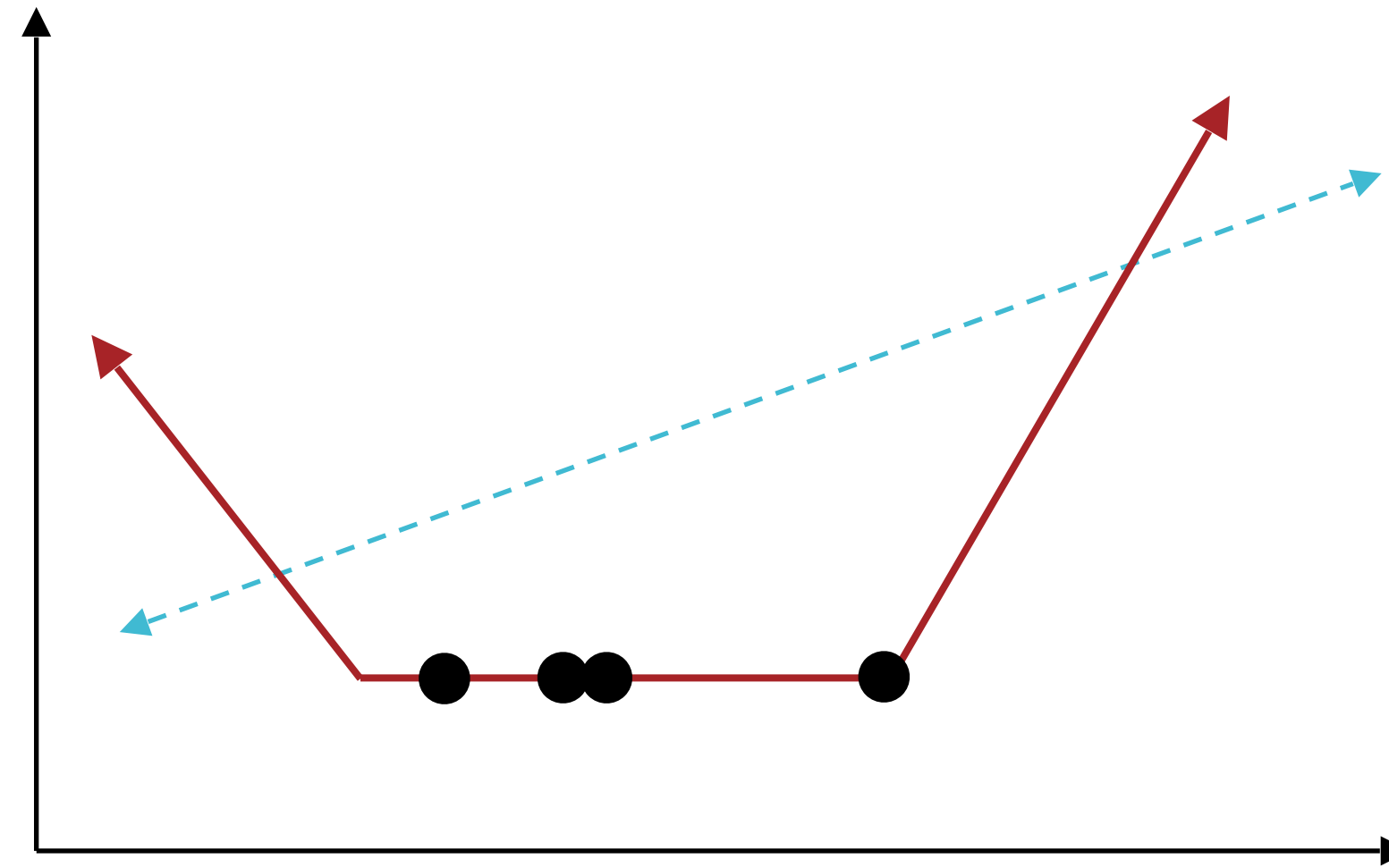


Convex functions:
Each local minimum is a
global minimum!

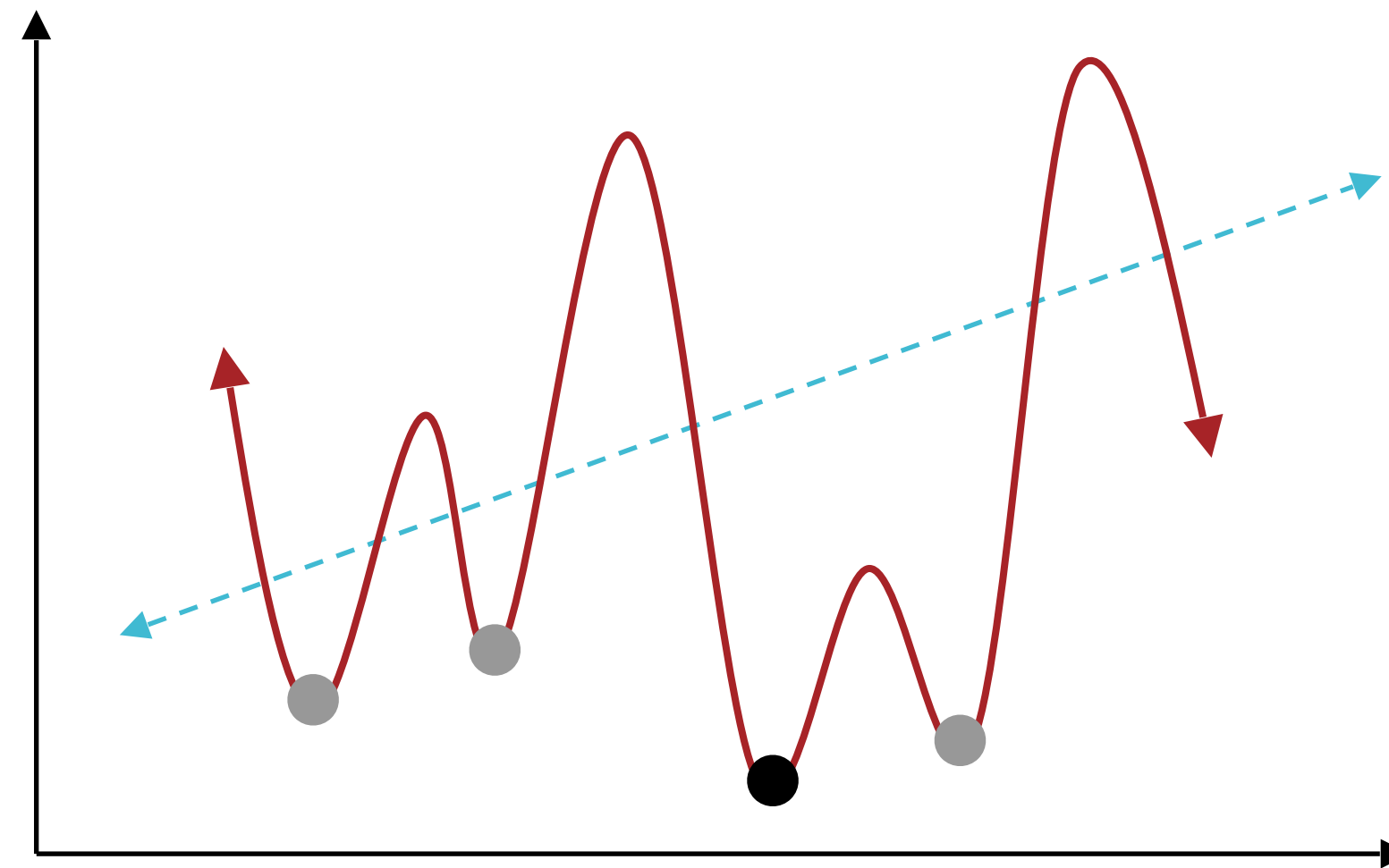


Non-convex functions:
A local minimum may or may
not be a global minimum...

Convexity: local and global minima

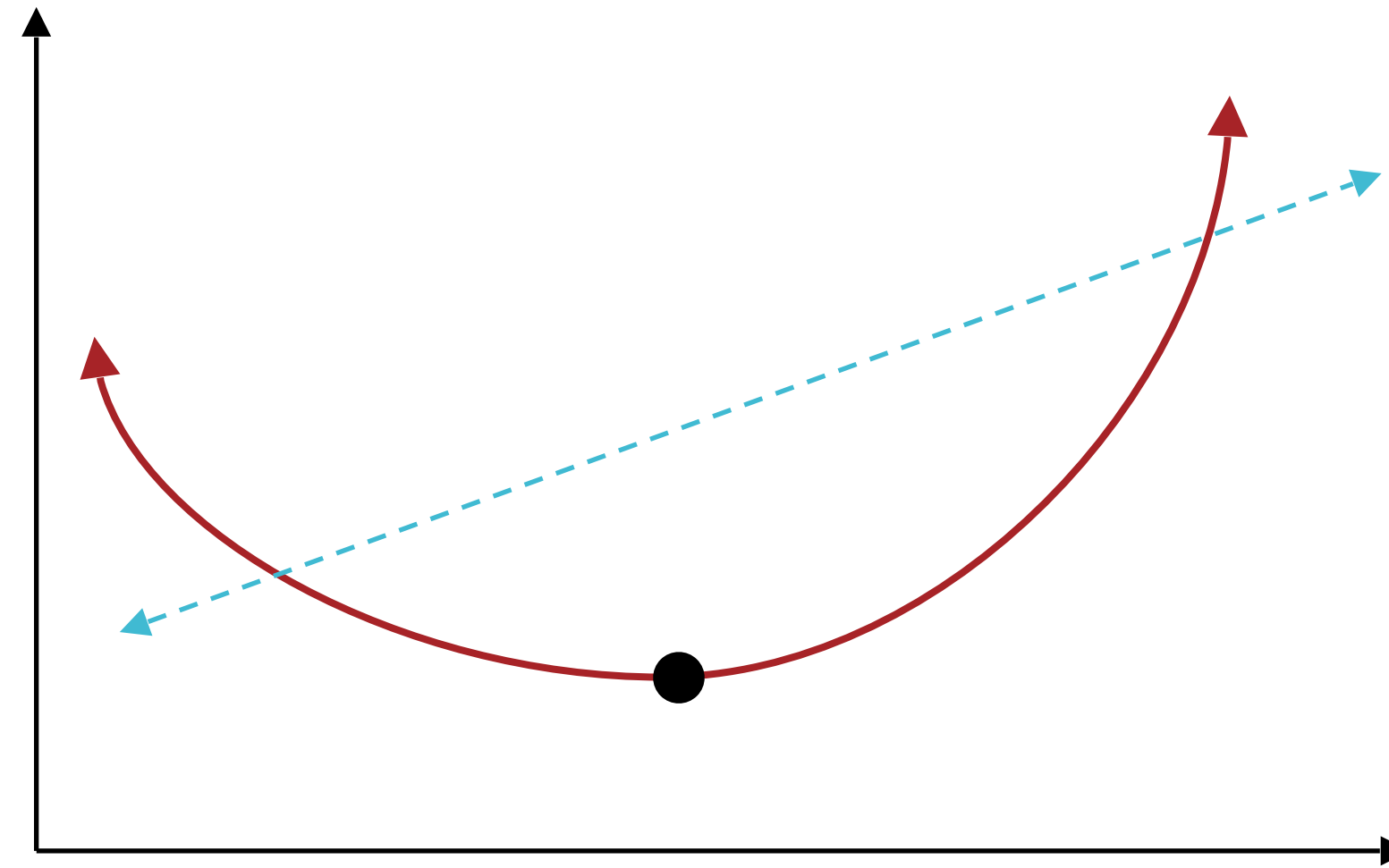


Convex functions:
Each local minimum is a
global minimum!

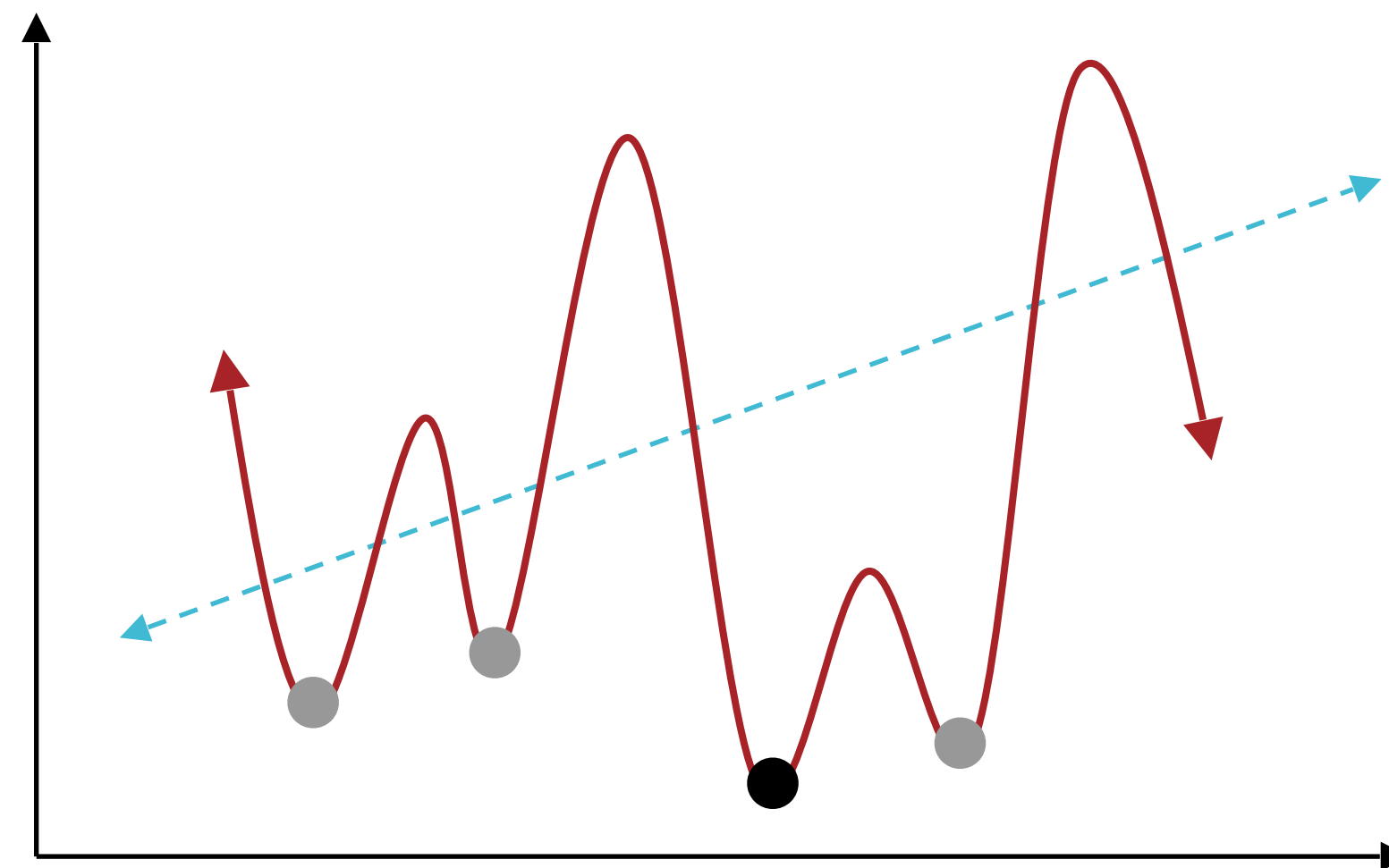


Non-convex functions:
A local minimum may or may
not be a global minimum...

Convexity: local and global minima



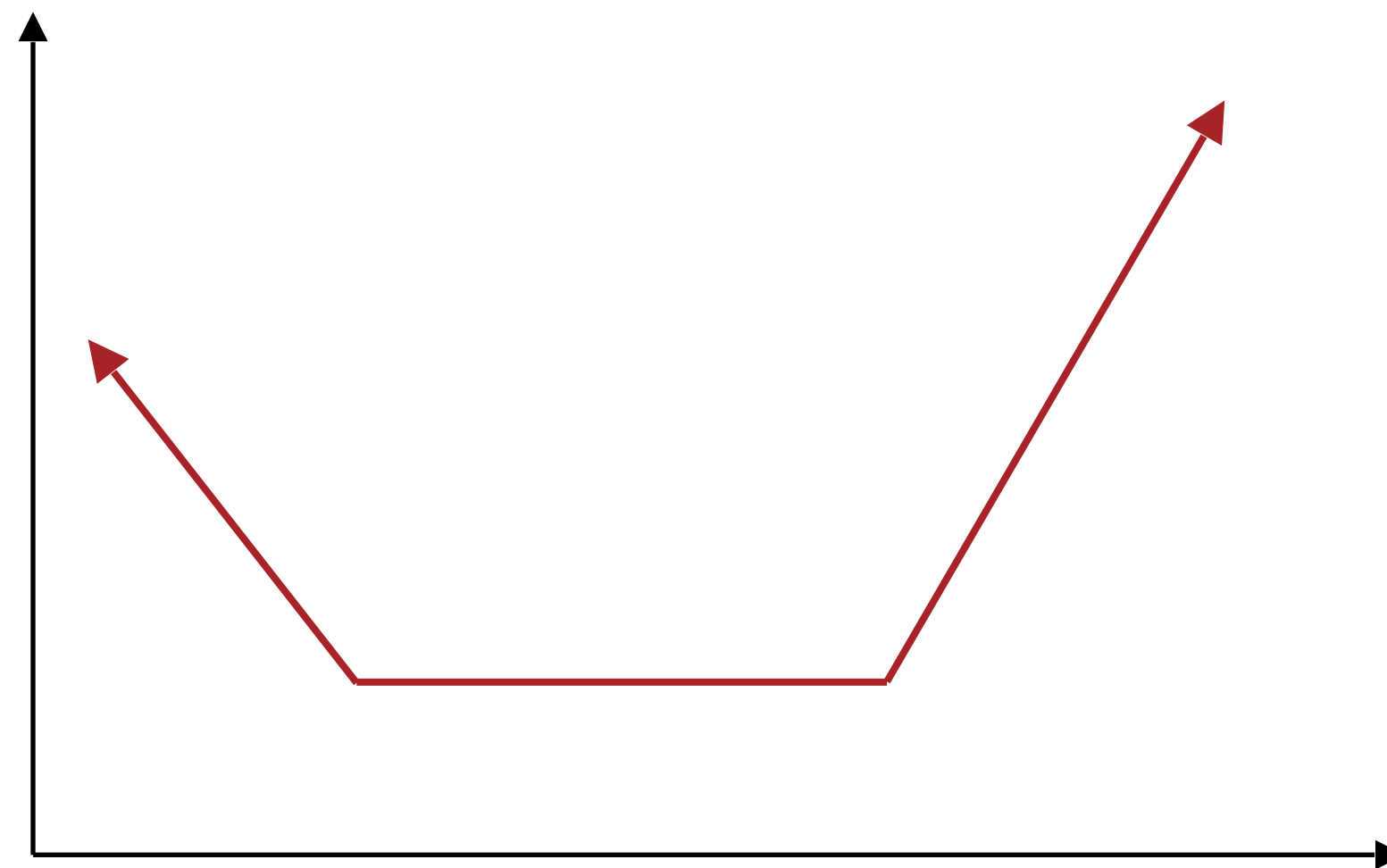
Strictly convex functions:
There exists a unique global
minimum!



Non-convex functions:
A local minimum may or may
not be a global minimum...

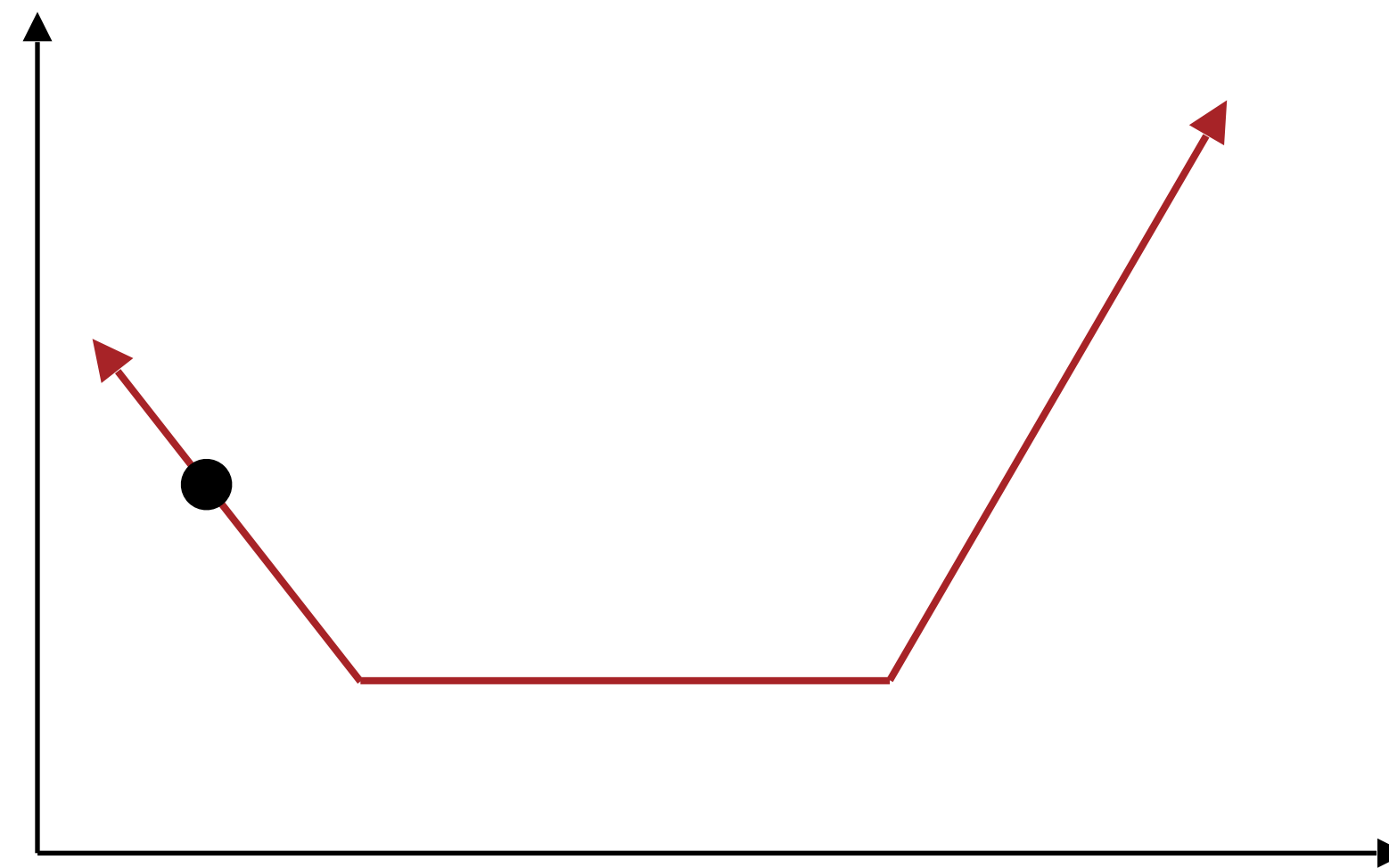
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Works great if the objective function is convex!



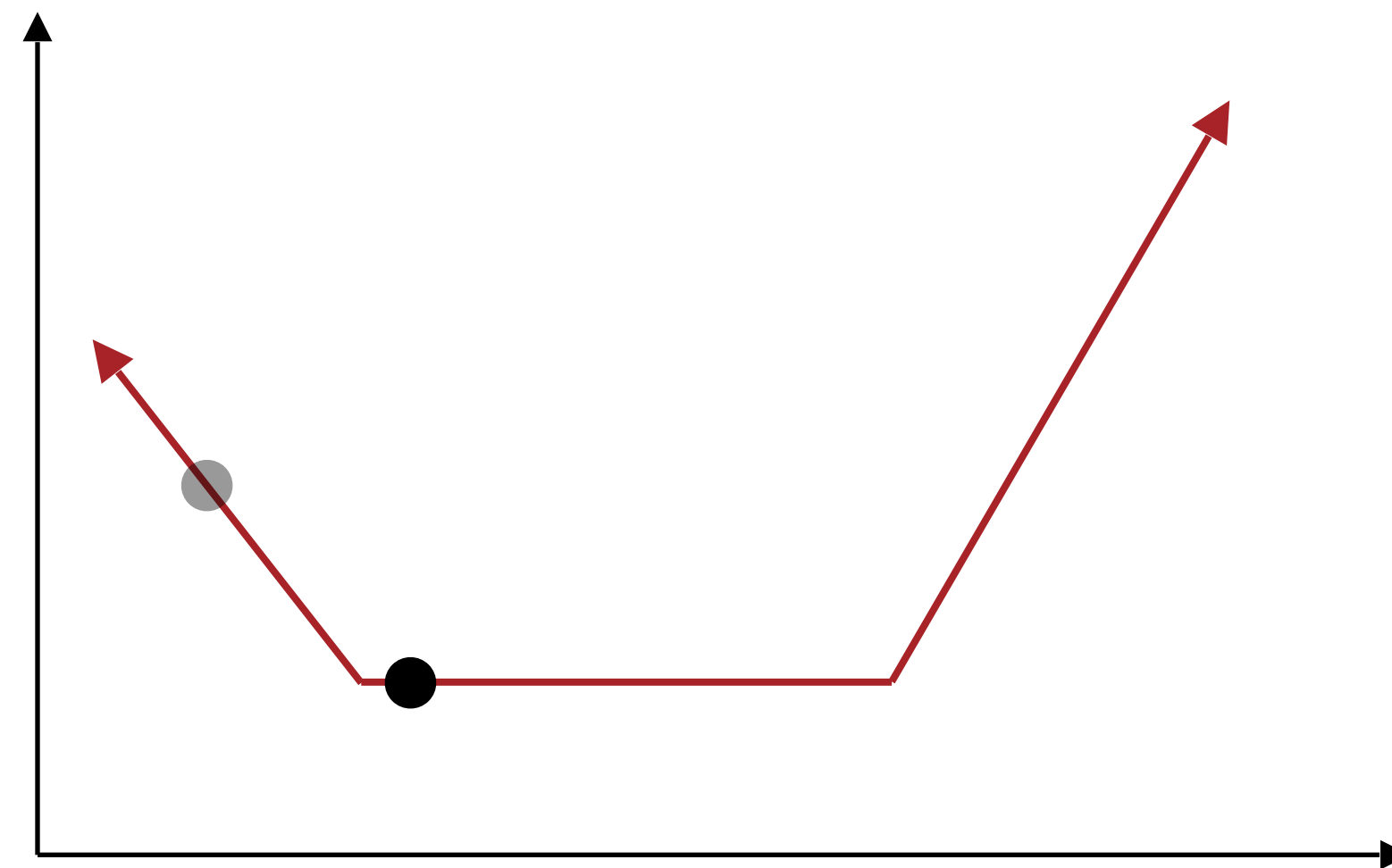
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Works great if the objective function is convex!



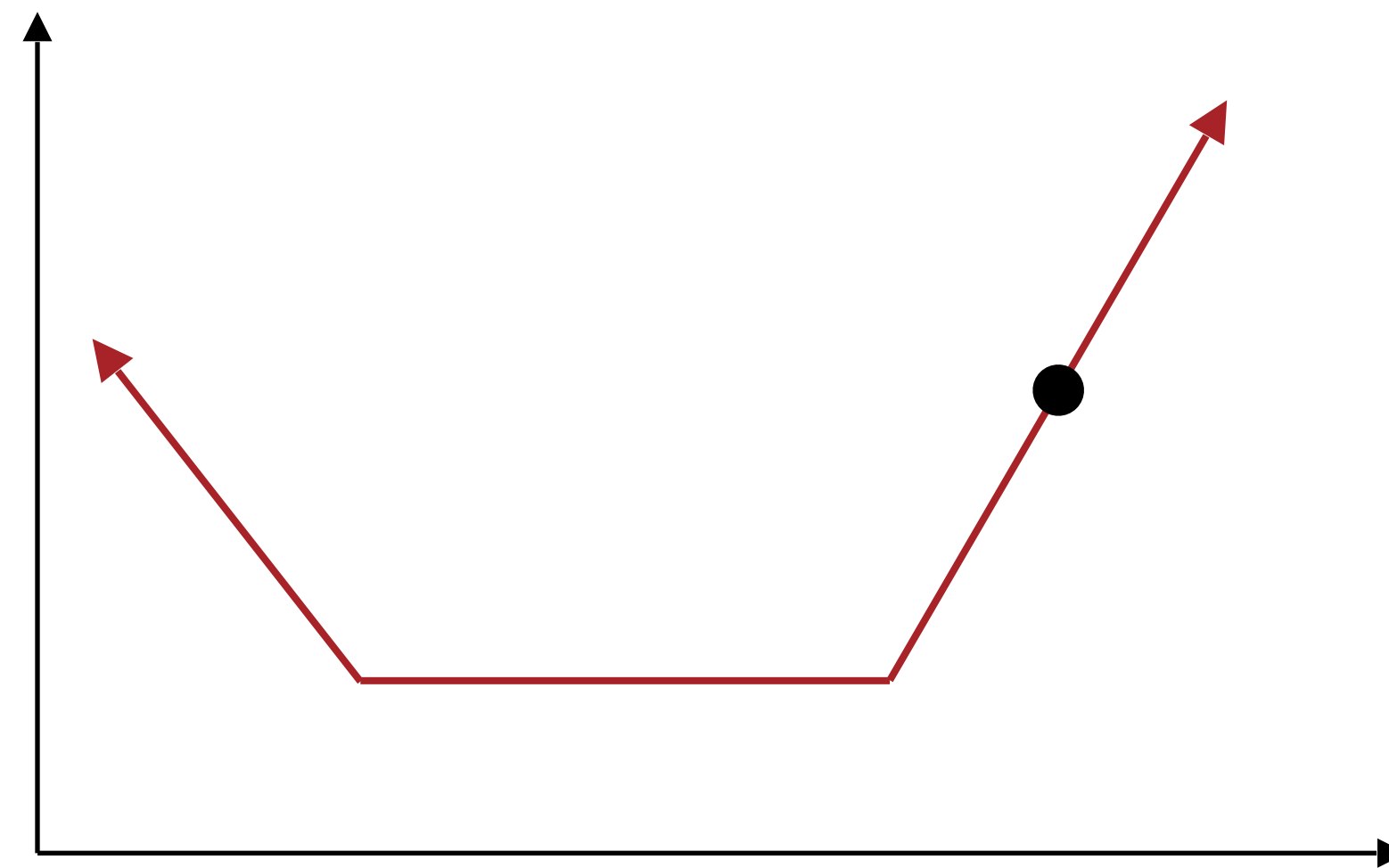
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Works great if the objective function is convex!



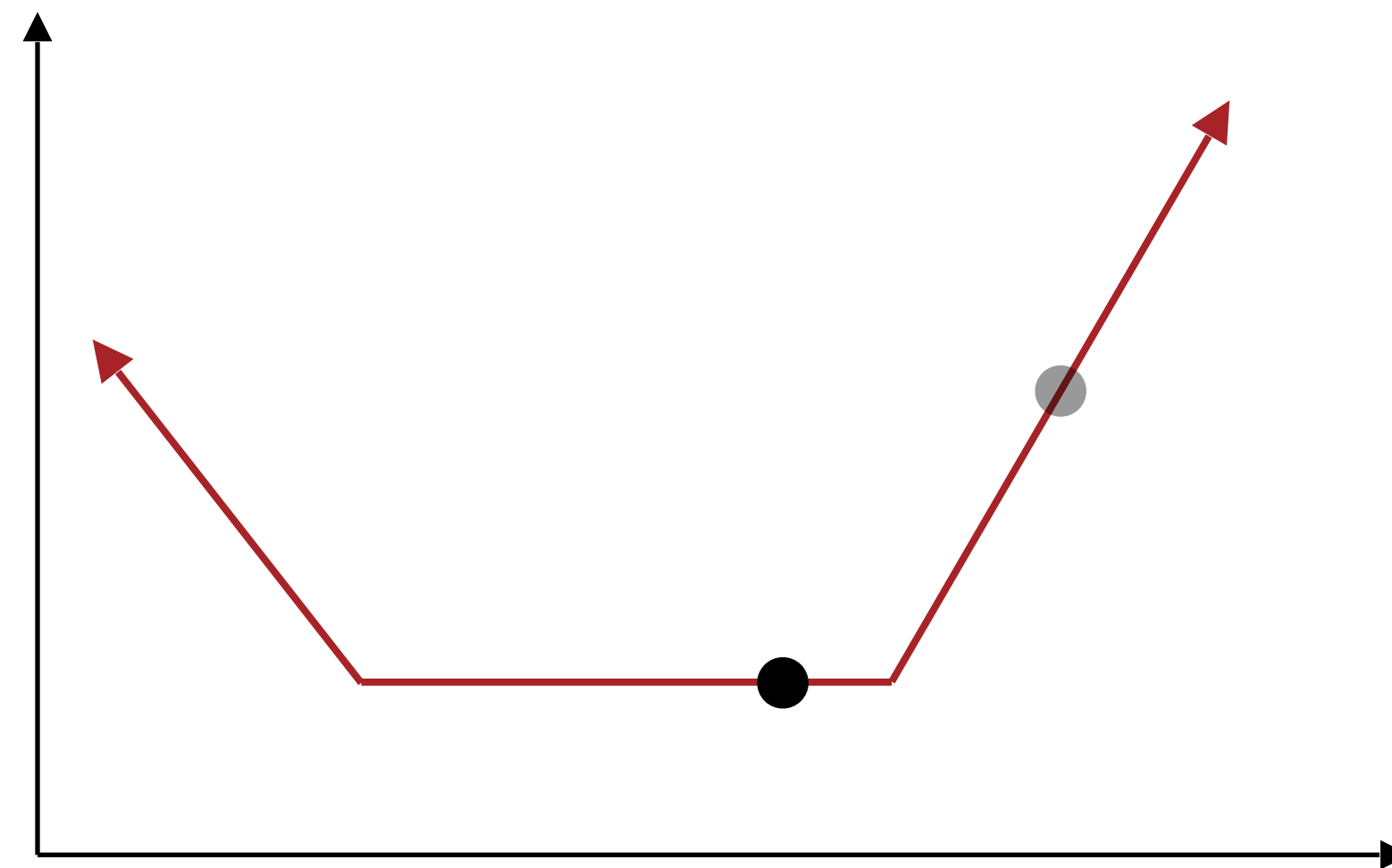
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Works great if the objective function is convex!



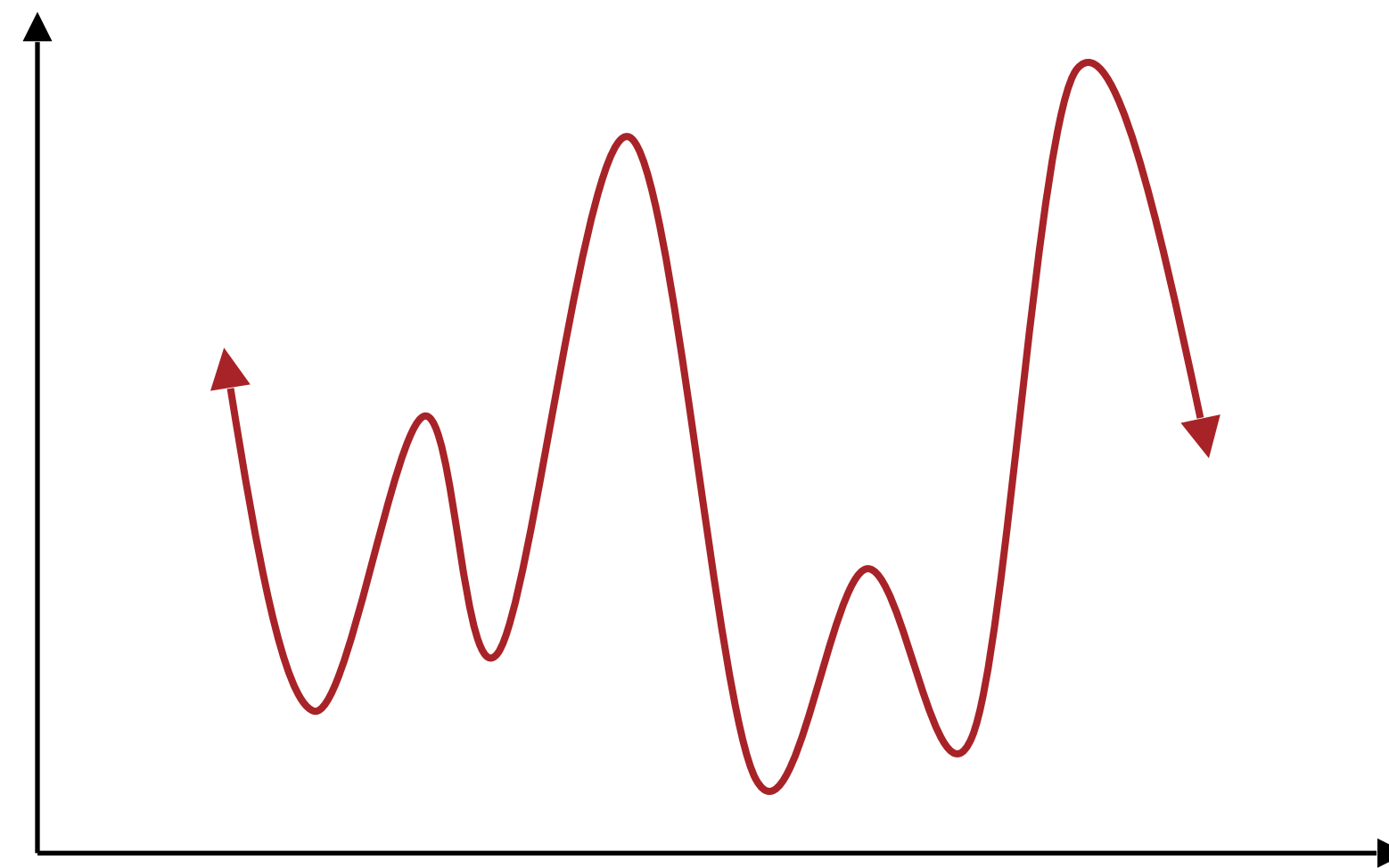
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Works great if the objective function is convex!



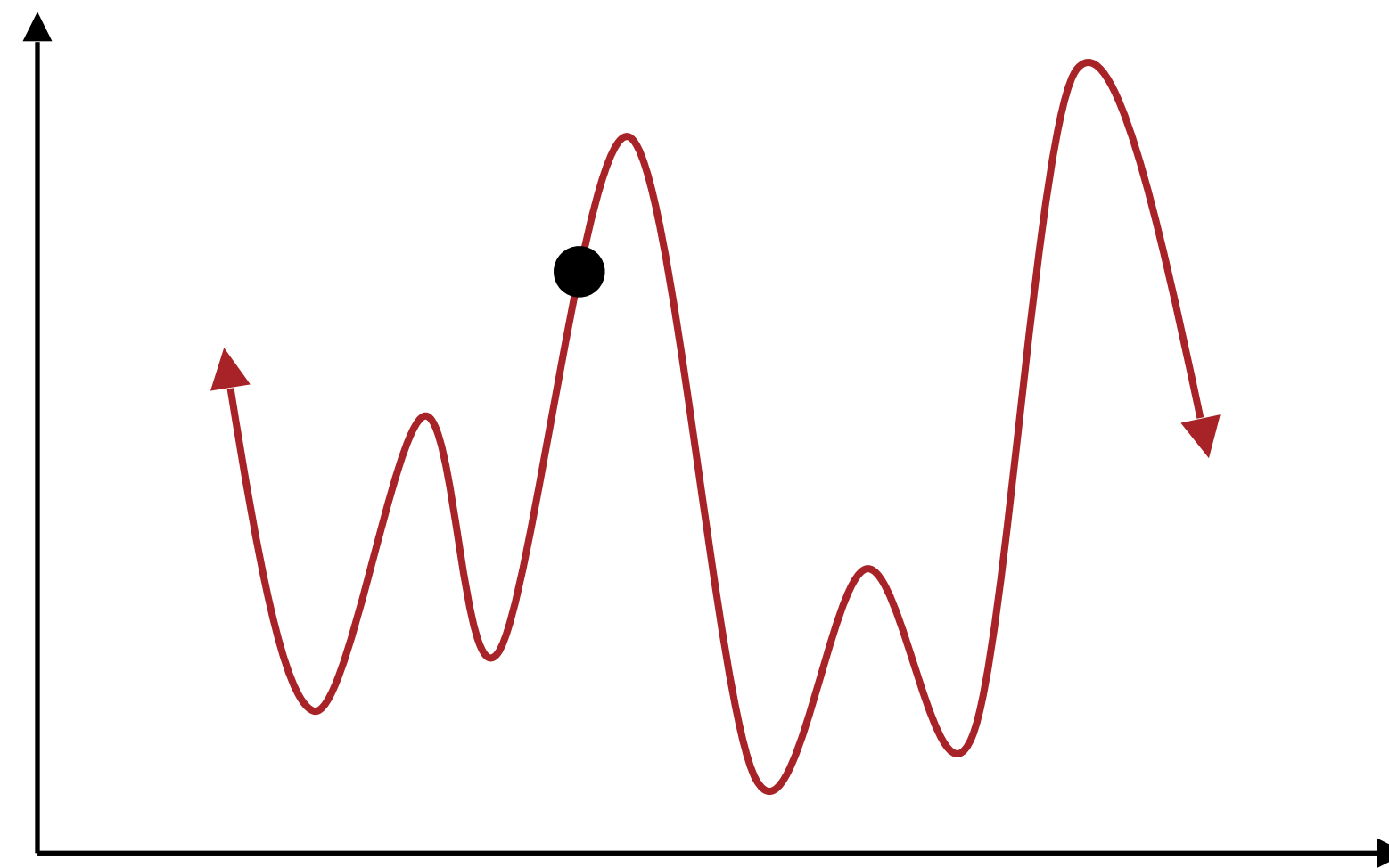
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Not ideal if the objective function is non-convex...



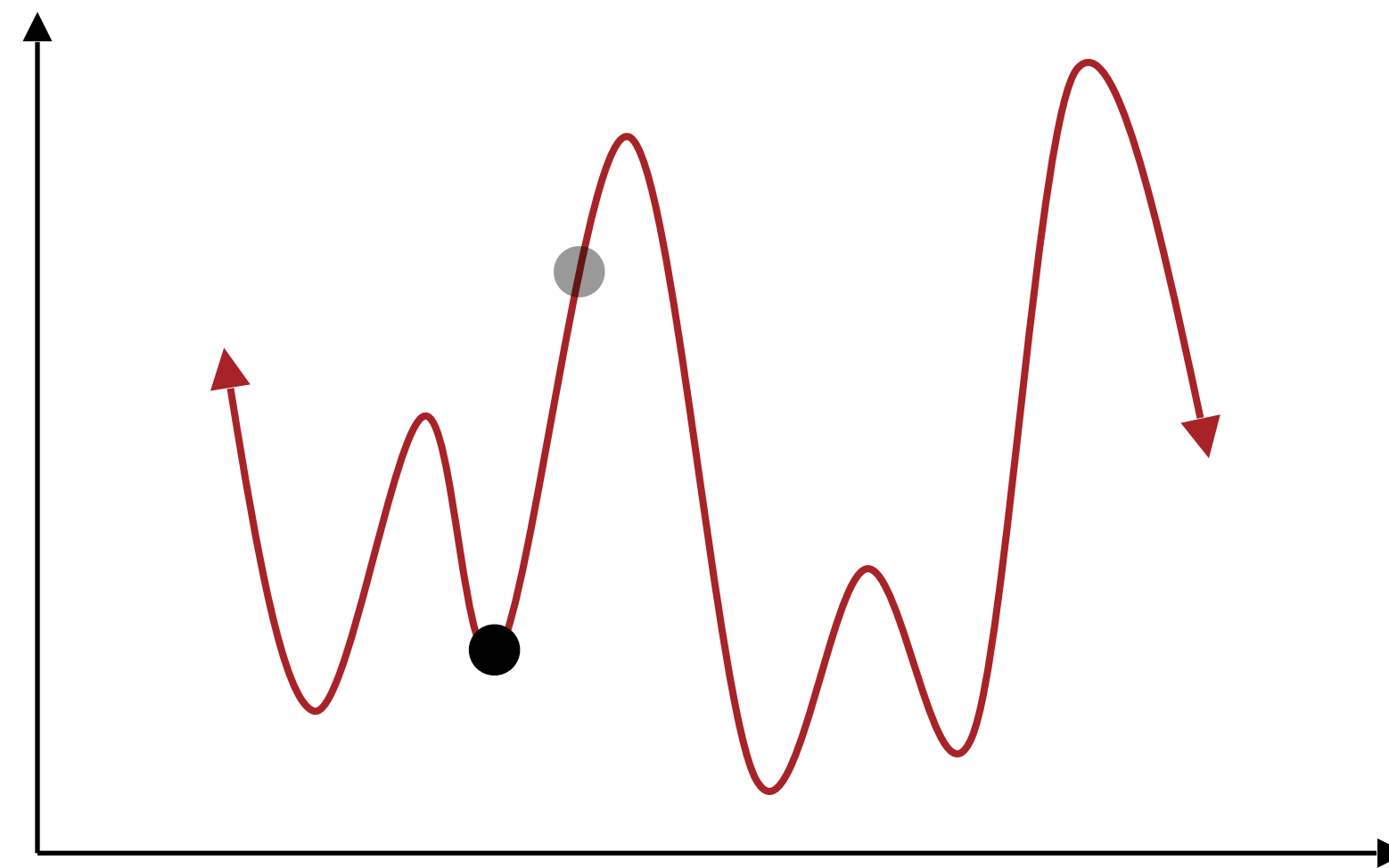
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Not ideal if the objective function is non-convex...



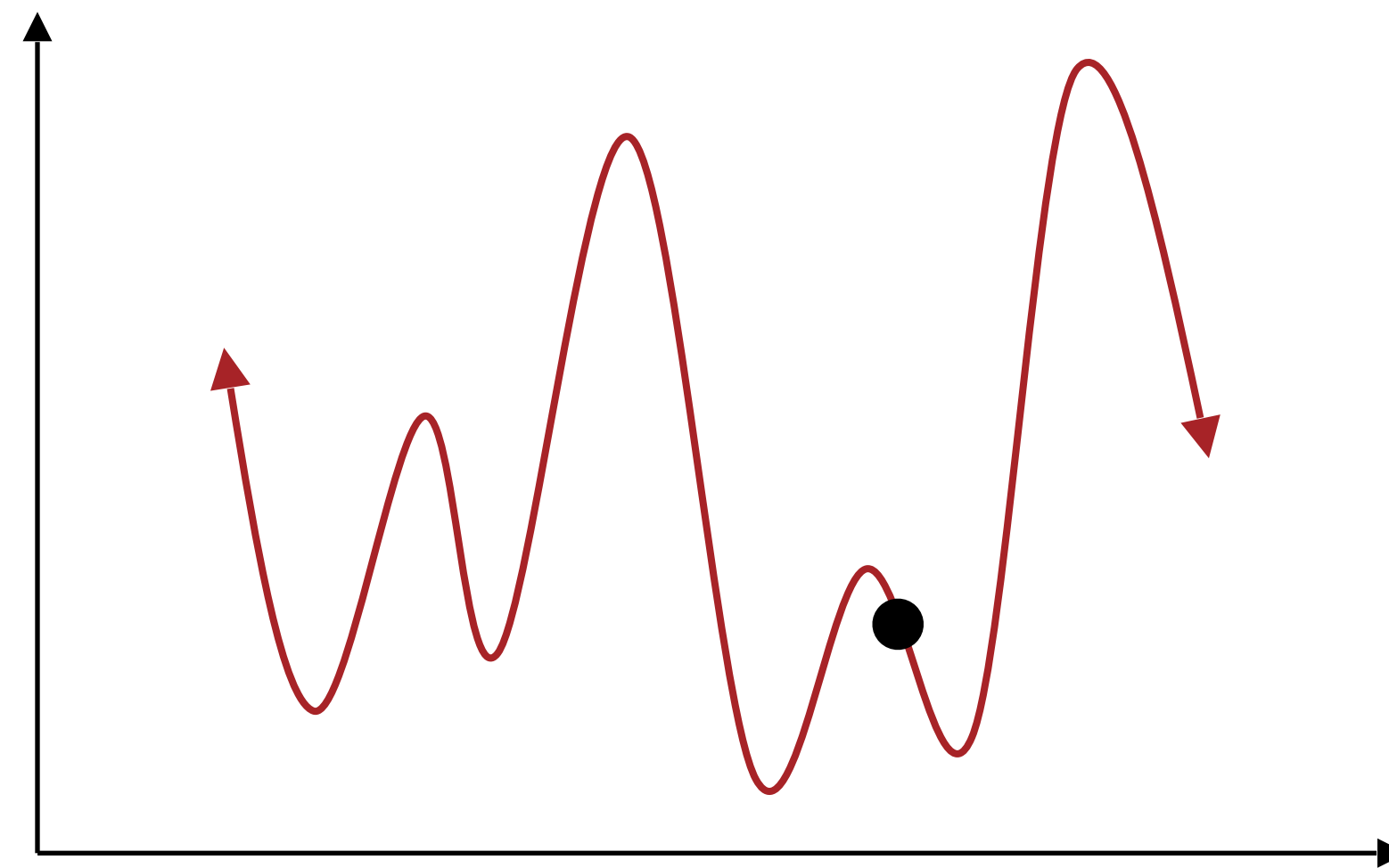
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Not ideal if the objective function is non-convex...



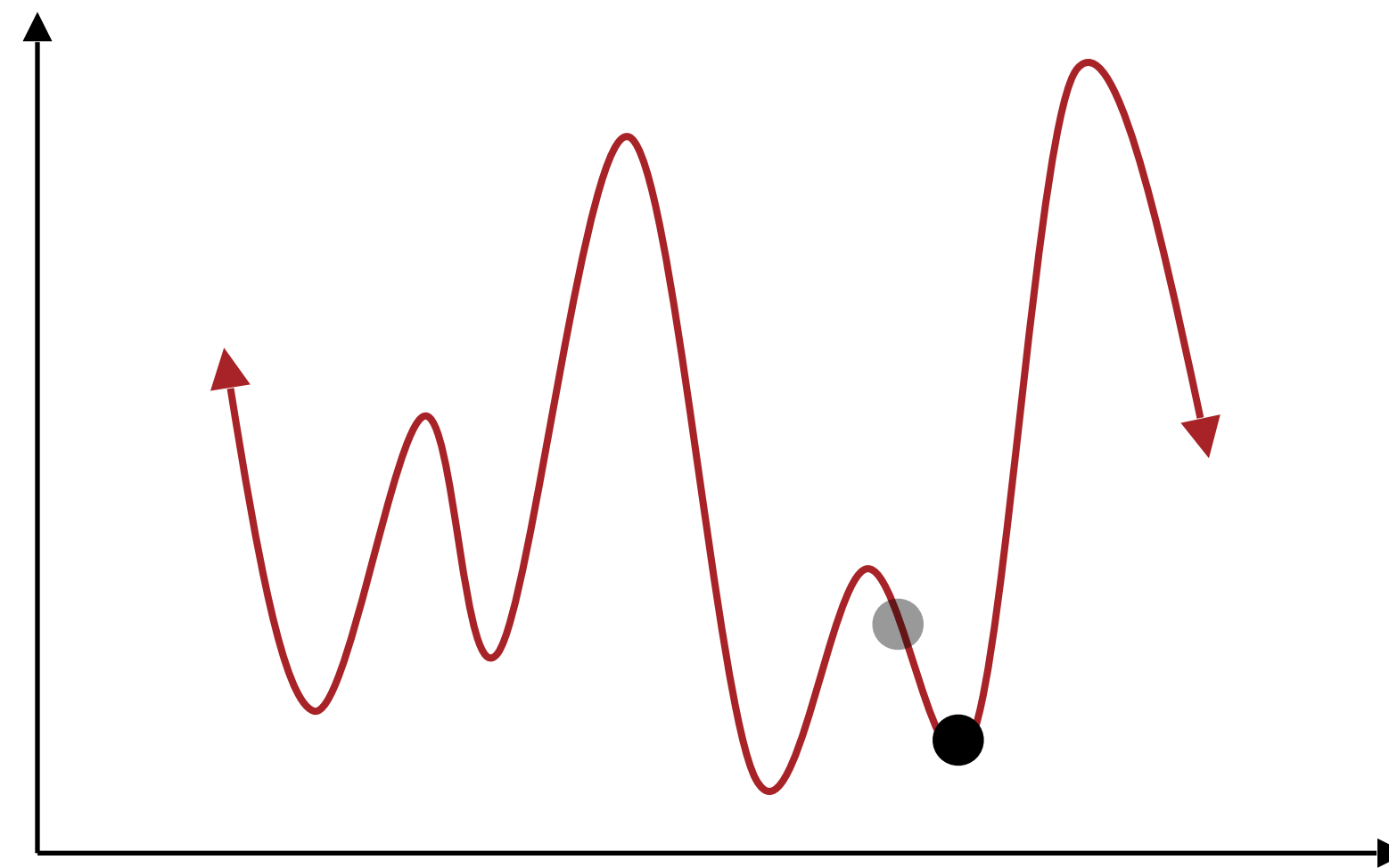
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Not ideal if the objective function is non-convex...



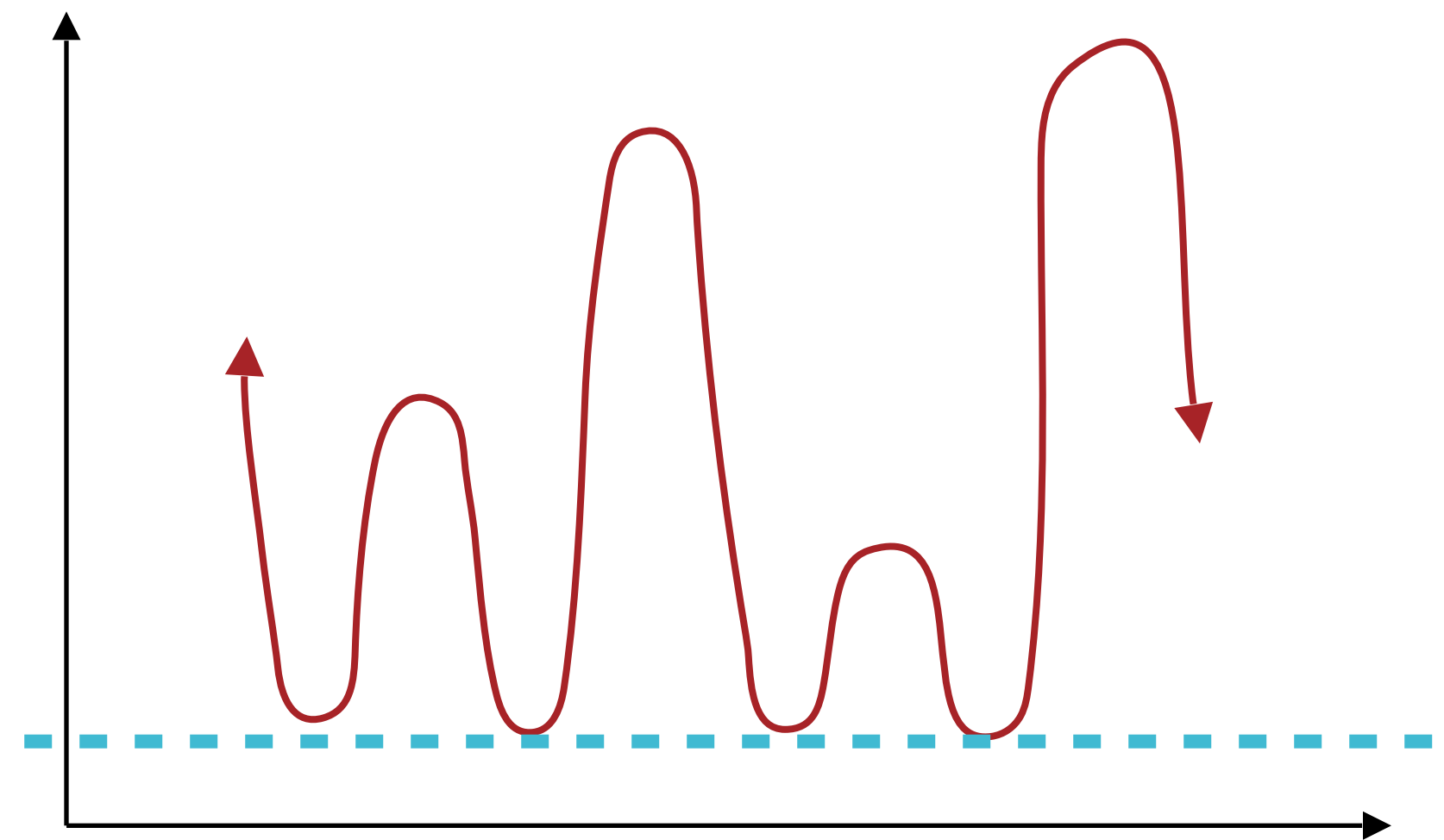
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ Not ideal if the objective function is non-convex...



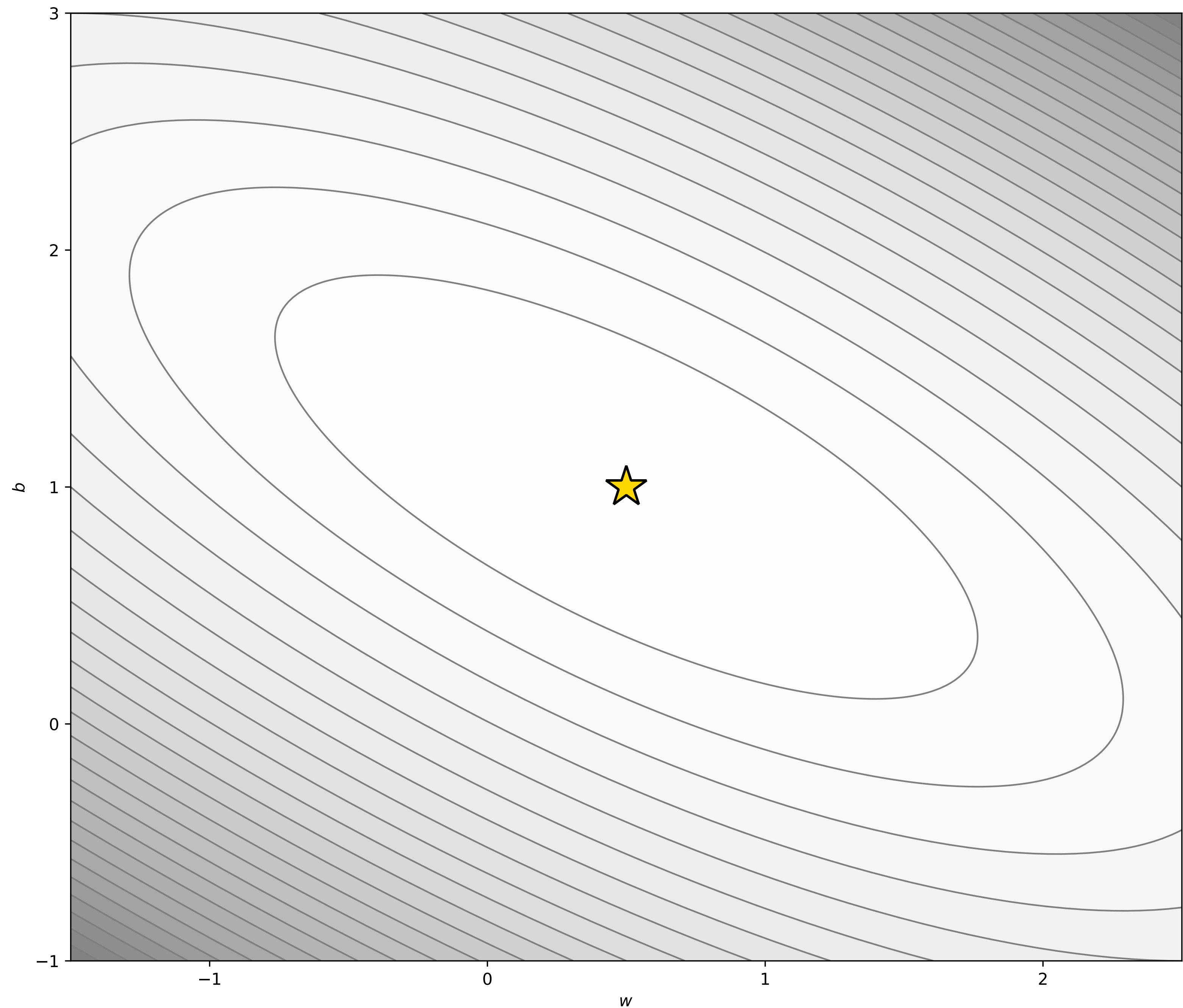
Gradient Descent & Convexity

- Gradient descent is a local optimization algorithm – it will converge to a local minimum (if it converges)
 - ▶ But can be OK if $\exists$ lots of pretty good local optima



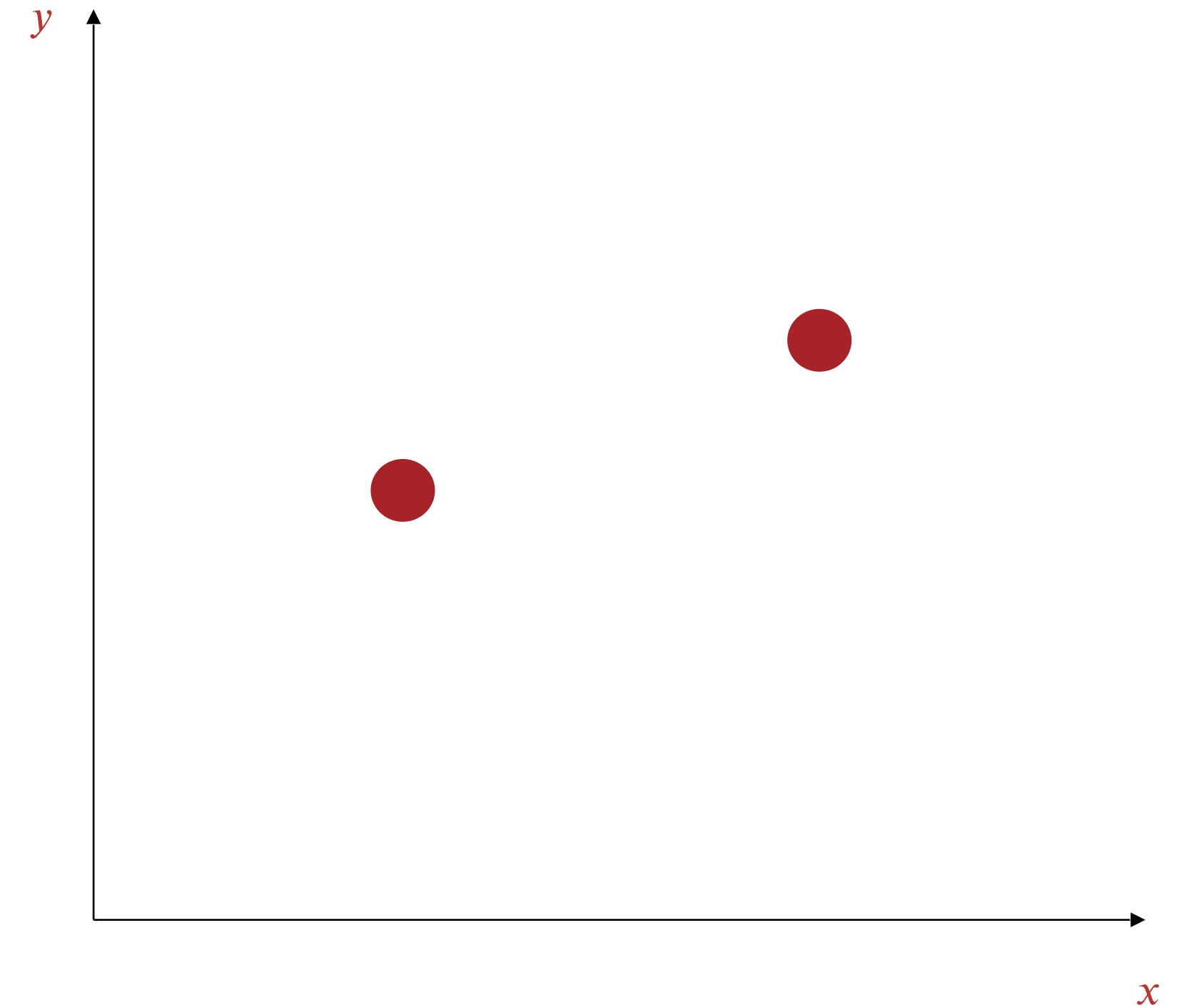
Gradient descent for linear regression: SSE is convex!

*In our simple problem,
strictly convex*



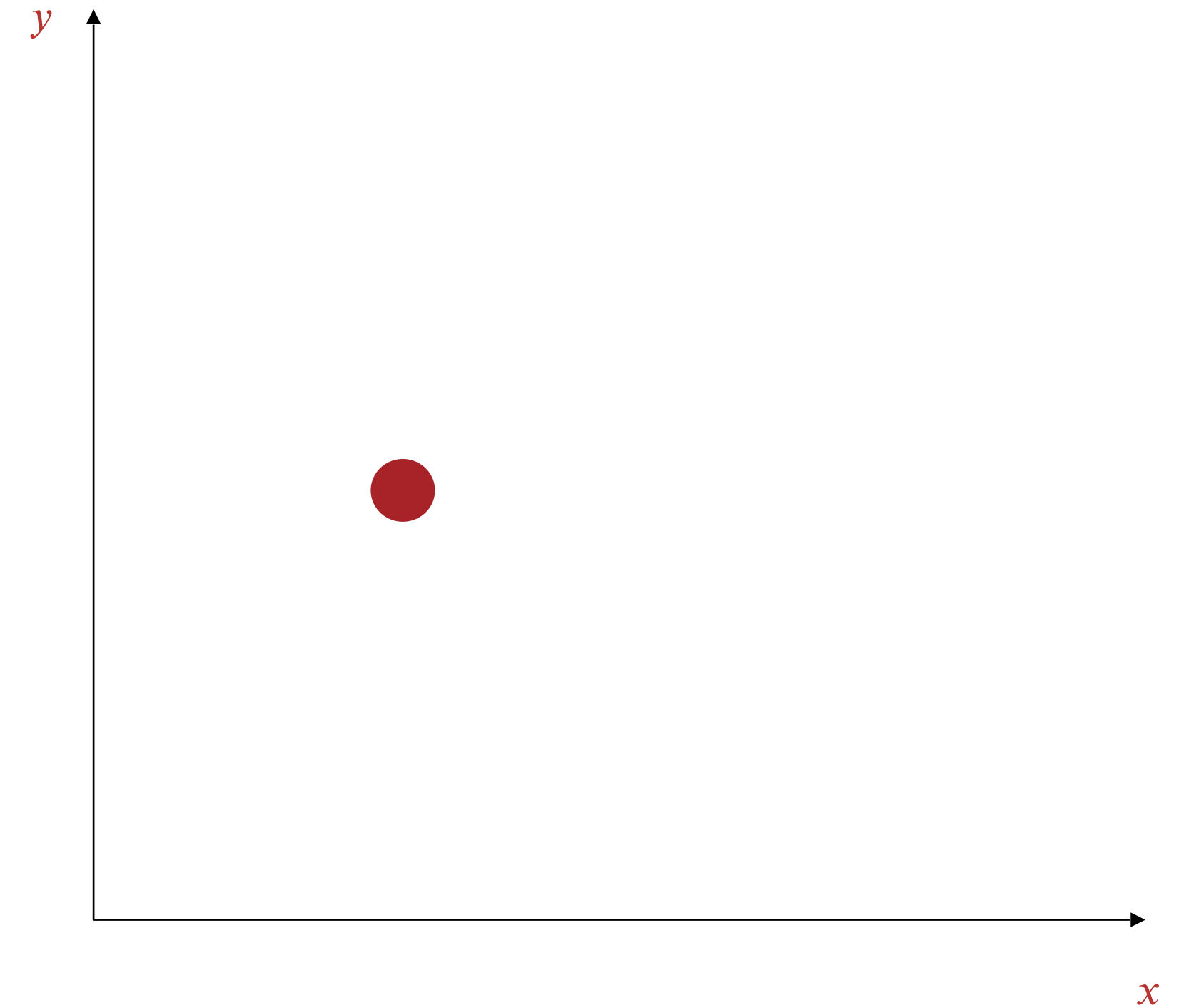
Linear Regression: Uniqueness

- Consider a 1D linear regression model trained to minimize the sum of squared errors: how many optimal solutions (i.e., sets of parameters w, b) are there for the given dataset?



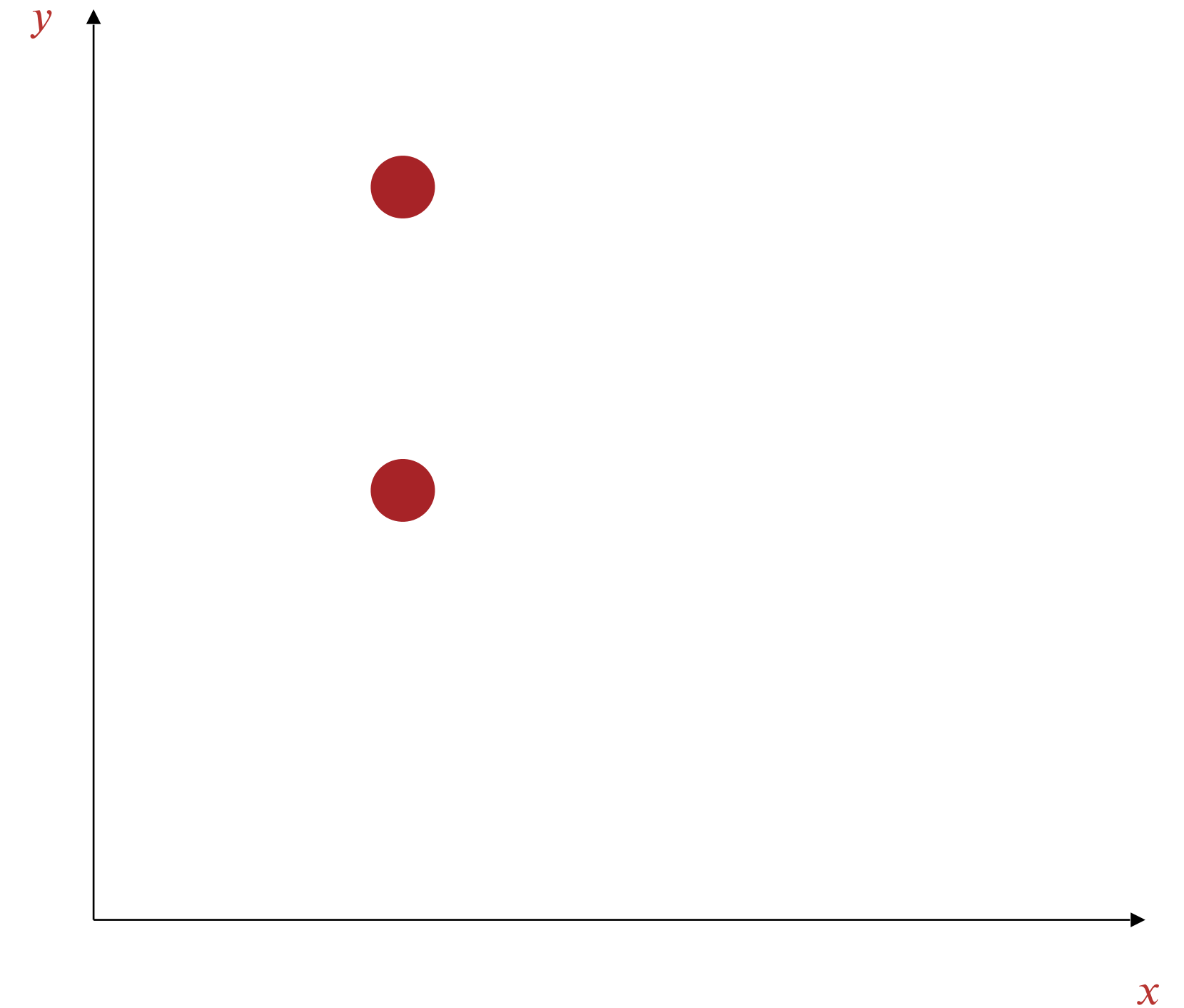
Linear Regression: Uniqueness

- Consider a 1D linear regression model trained to minimize the sum of squared errors: how many optimal solutions (i.e., sets of parameters w, b) are there for the given dataset?



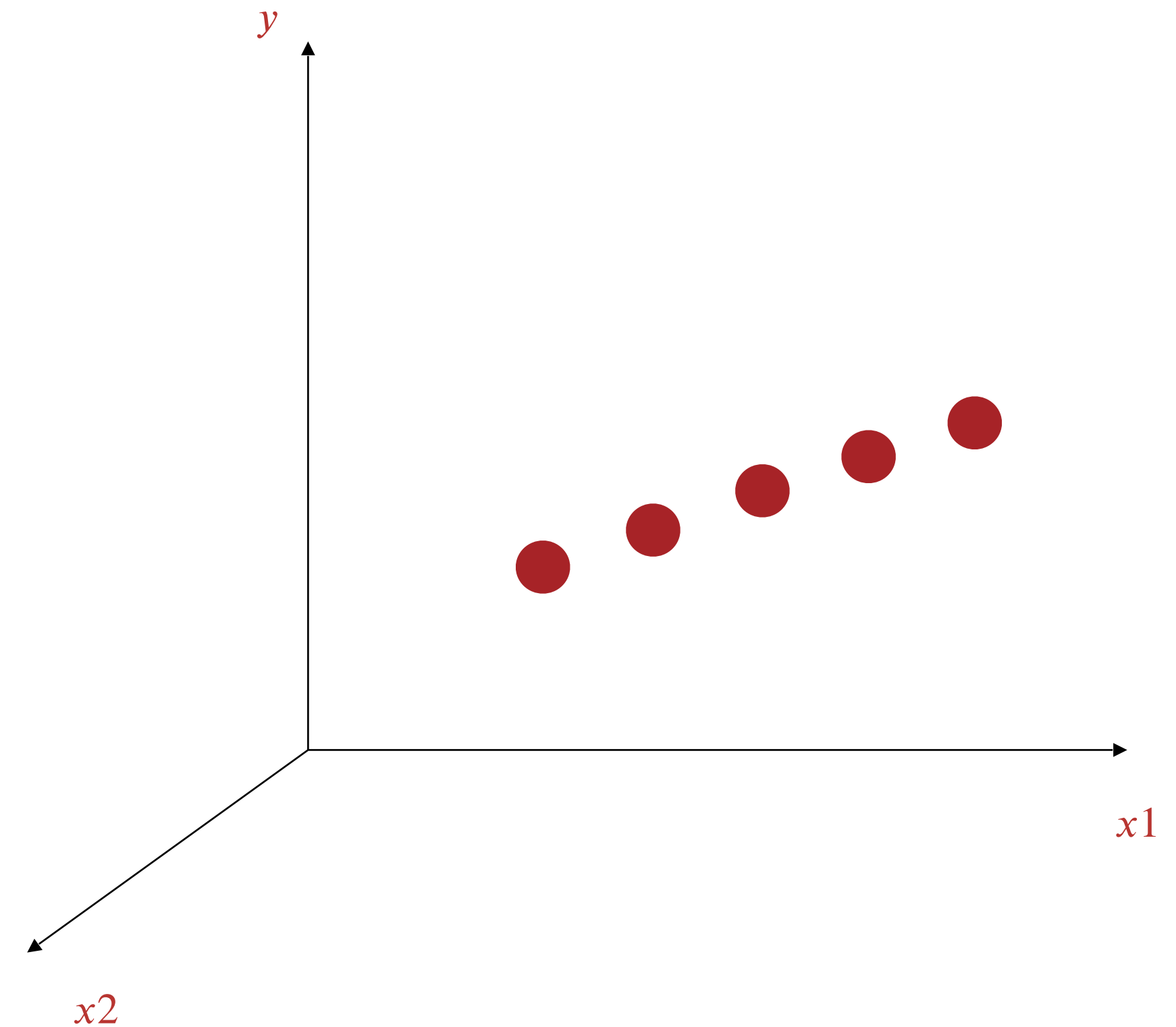
Linear Regression: Uniqueness

- Consider a 1D linear regression model trained to minimize the sum of squared errors: how many optimal solutions (i.e., sets of parameters w, b) are there for the given dataset?



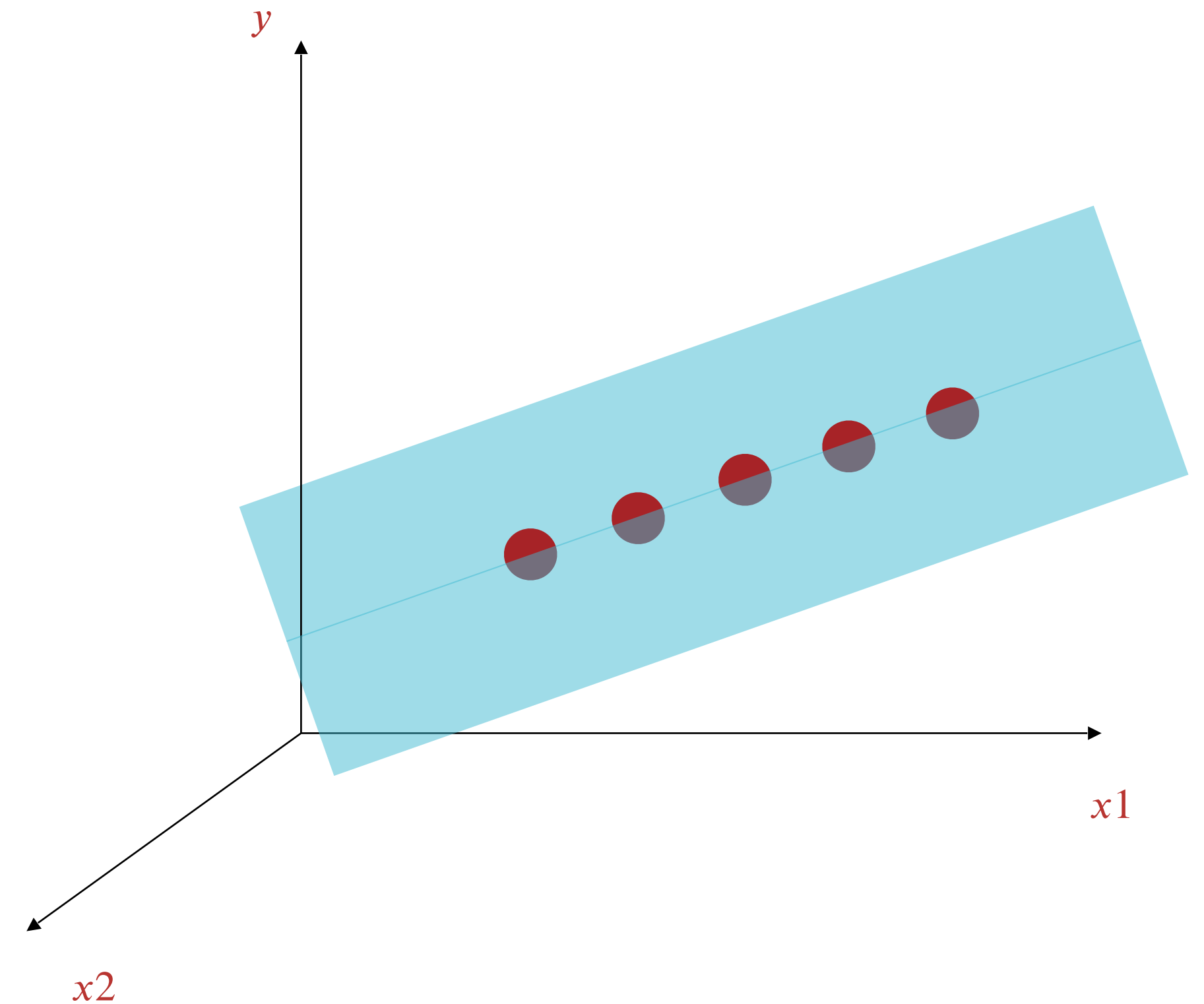
Linear Regression: Uniqueness

- Consider a 2D linear regression model trained to minimize the sum of squared errors: how many optimal solutions (i.e., sets of parameters w, b) are there for the given dataset?



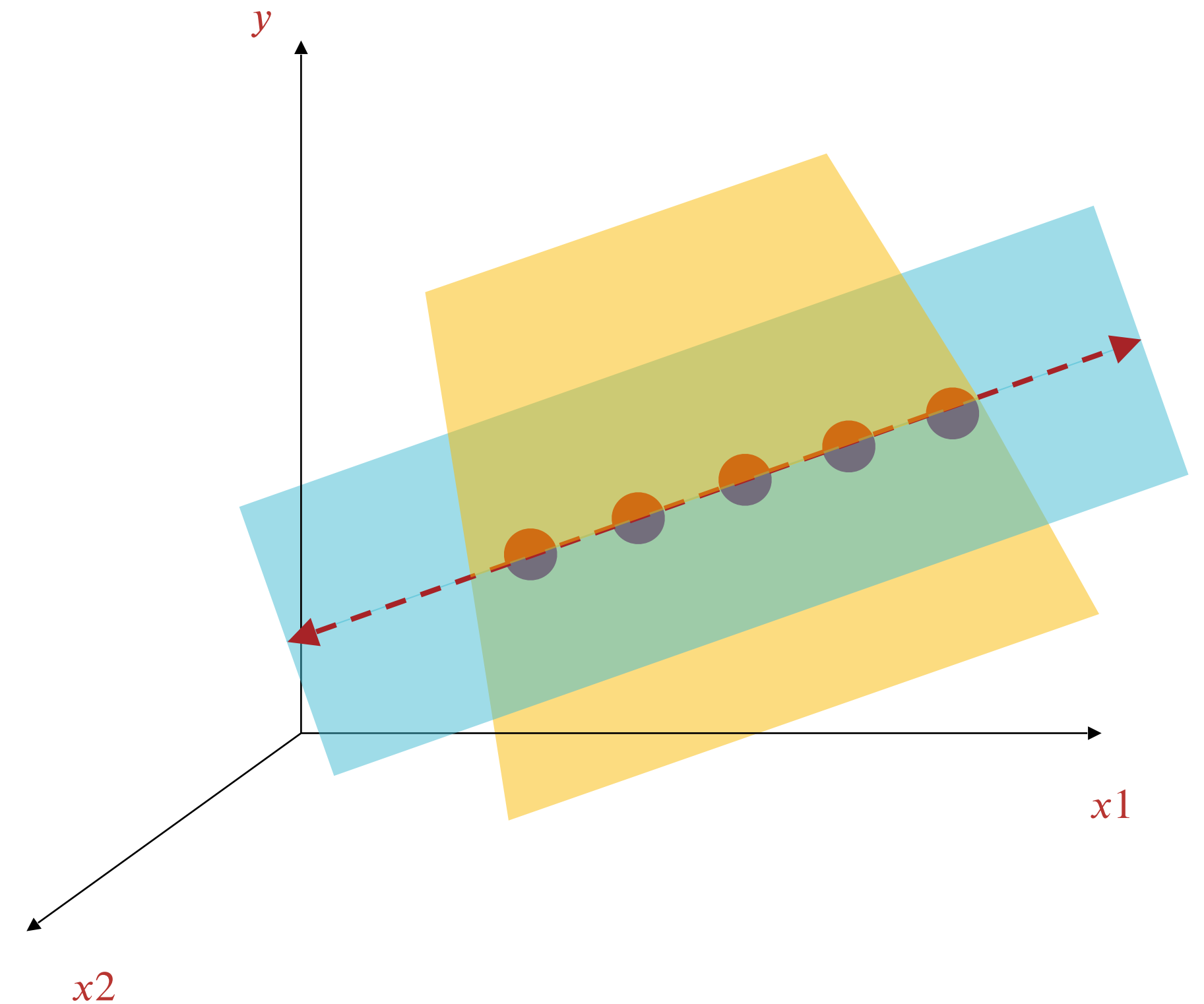
Linear Regression: Uniqueness

- Consider a 2D linear regression model trained to minimize the sum of squared errors: how many optimal solutions (i.e., sets of parameters w, b) are there for the given dataset?



Linear Regression: Uniqueness

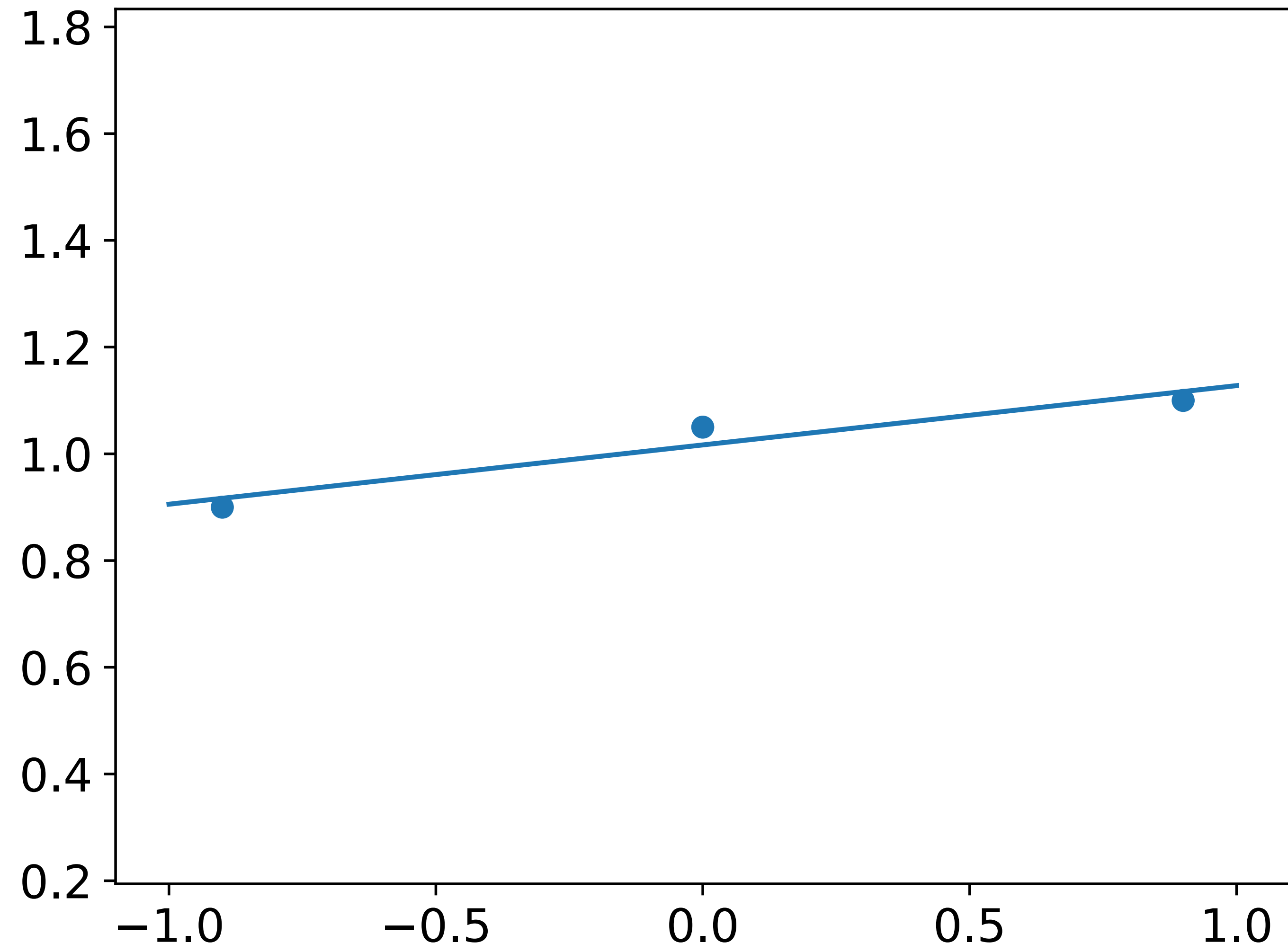
- Consider a 2D linear regression model trained to minimize the sum of squared errors: how many optimal solutions (i.e., sets of parameters w, b) are there for the given dataset?



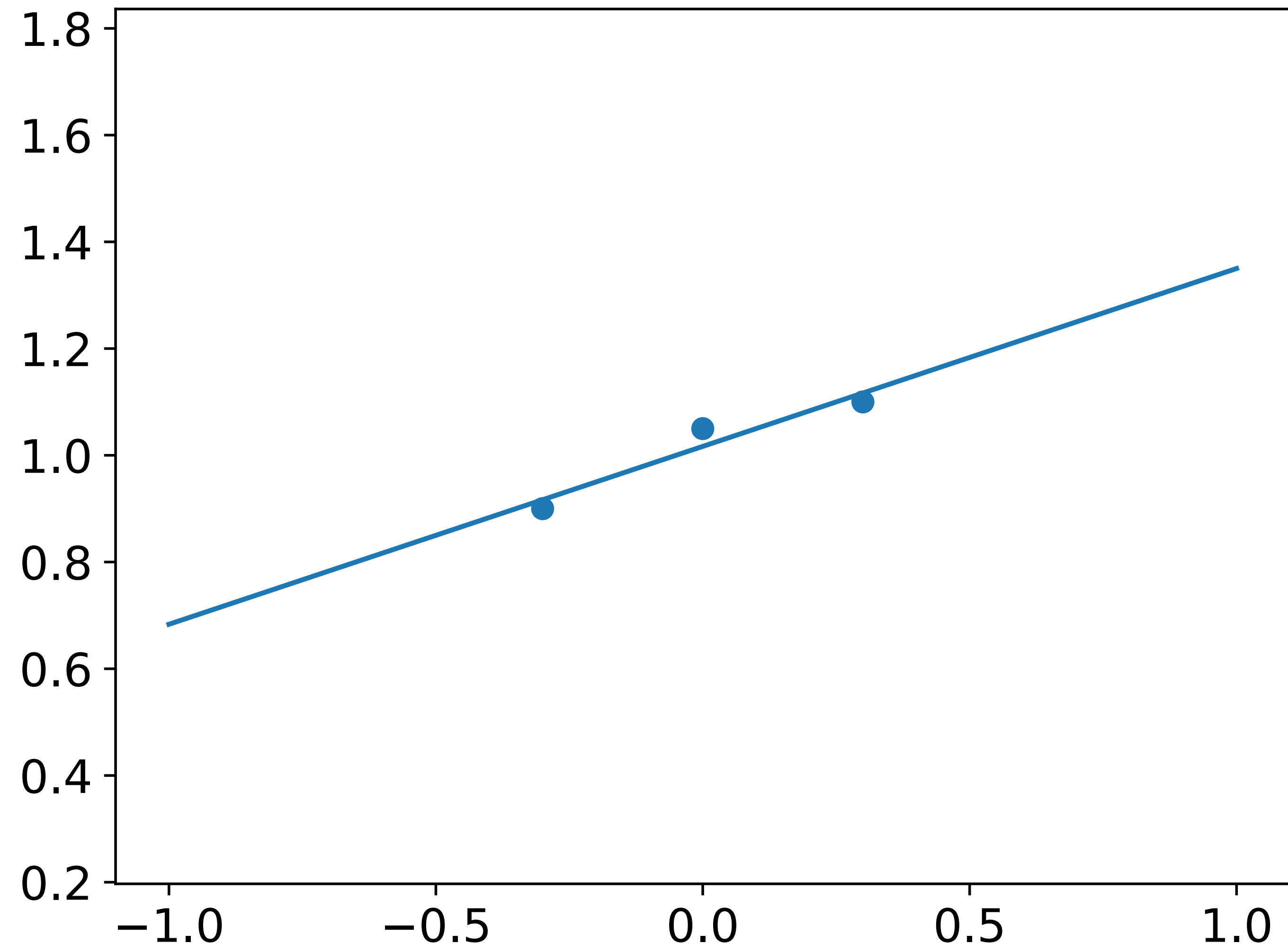
Uniqueness conditions

- Solution is unique if # examples N is *at least as large as* number of features k and all features are *linearly independent* (rank of X is k)
 - ▶ or if ridge parameter λ is strictly positive
- Can drop features to get independence, or add ridge, or resolve ambiguity by (e.g.) minimum norm solution

A tale of two datasets

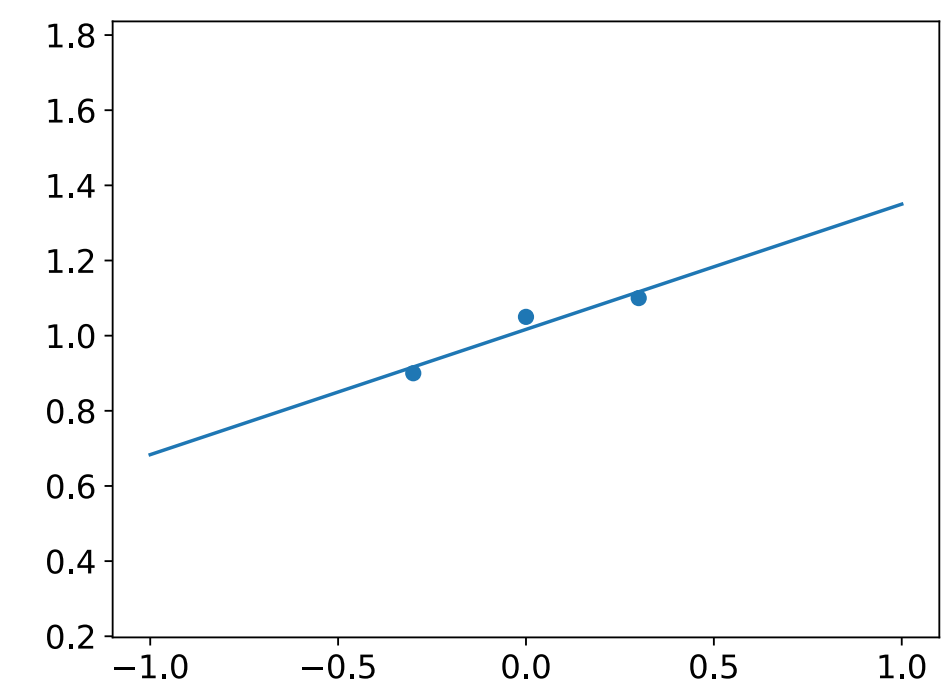
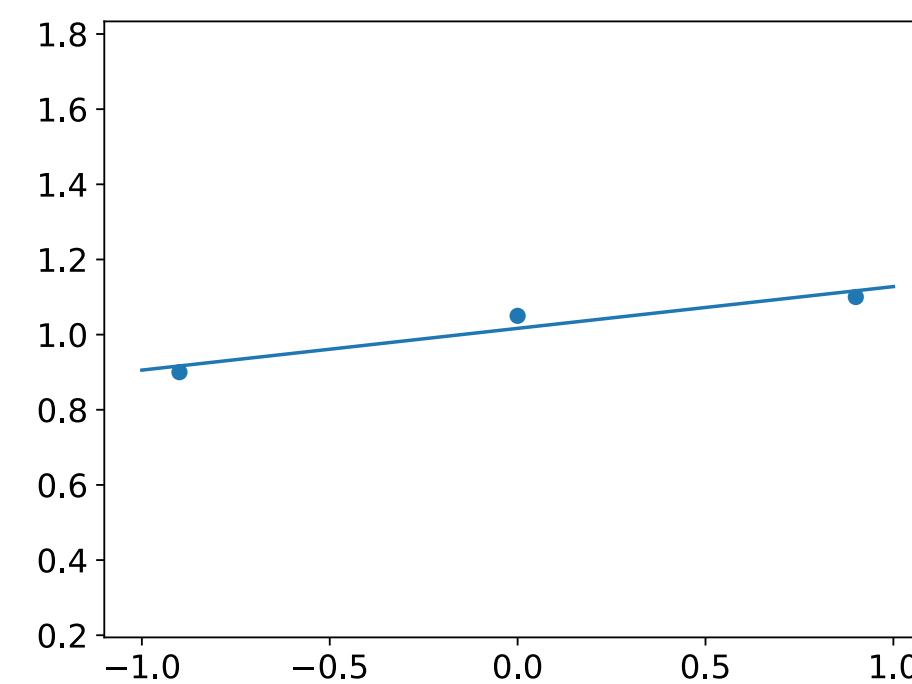
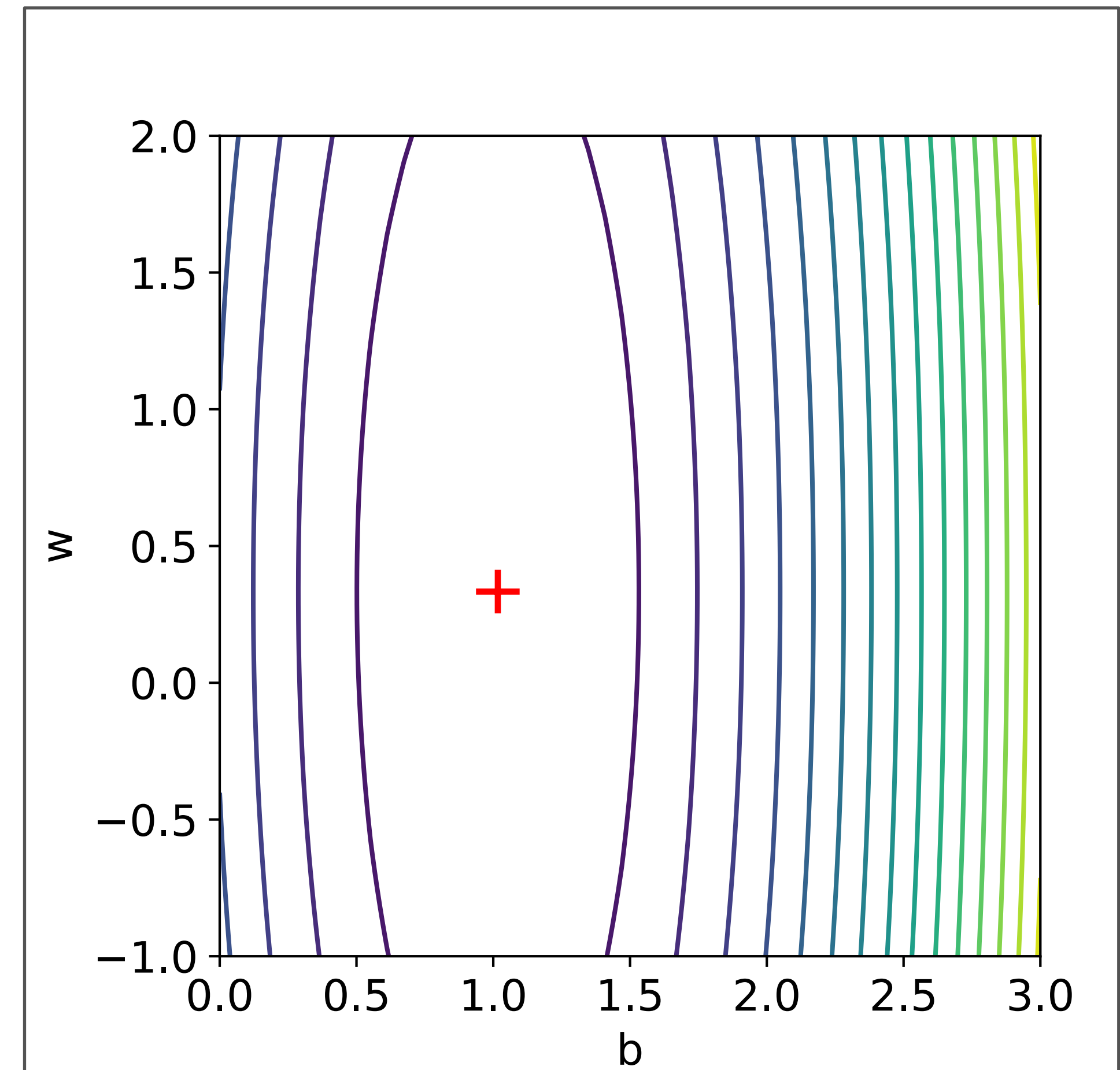
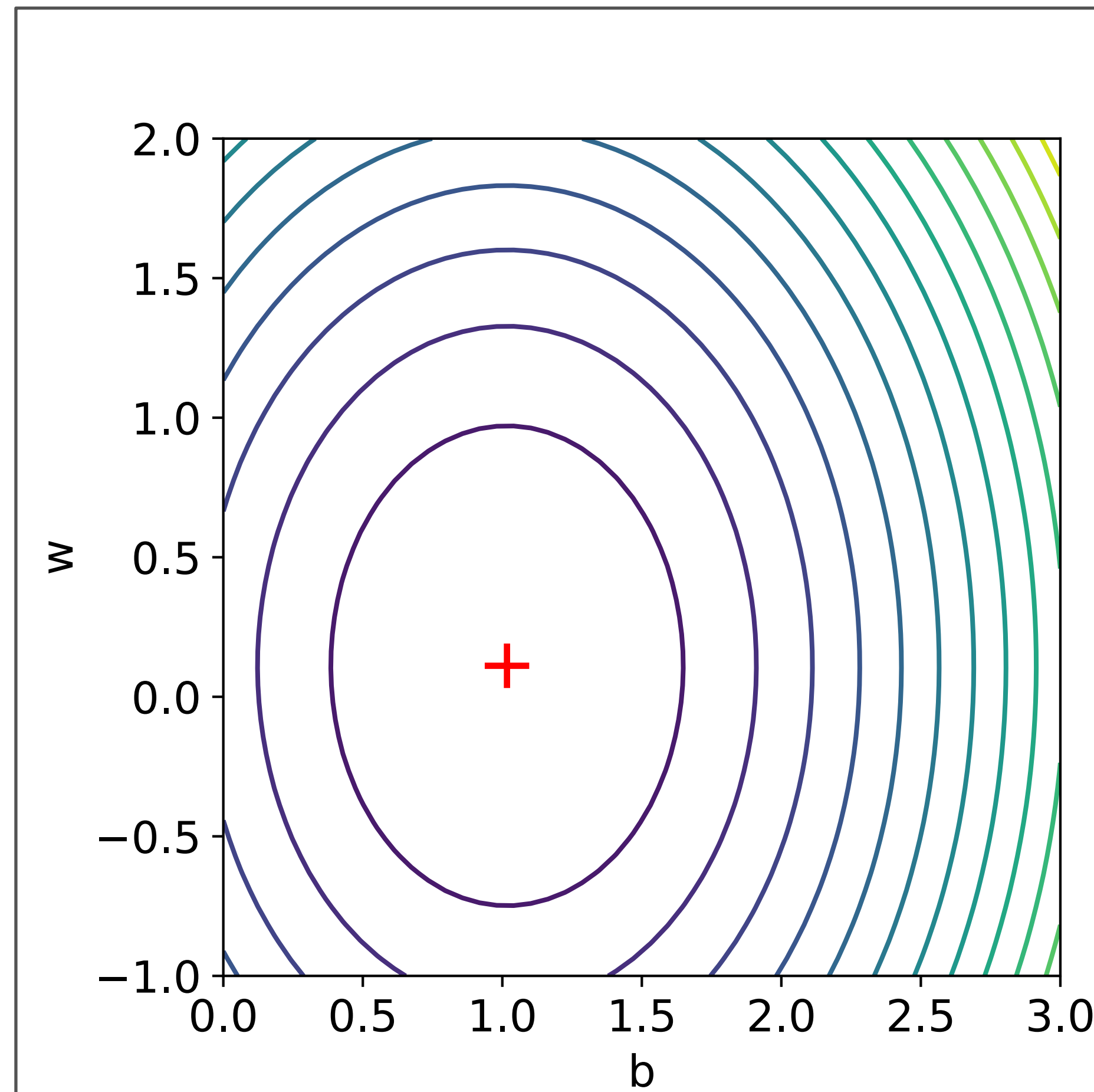


A tale of two datasets

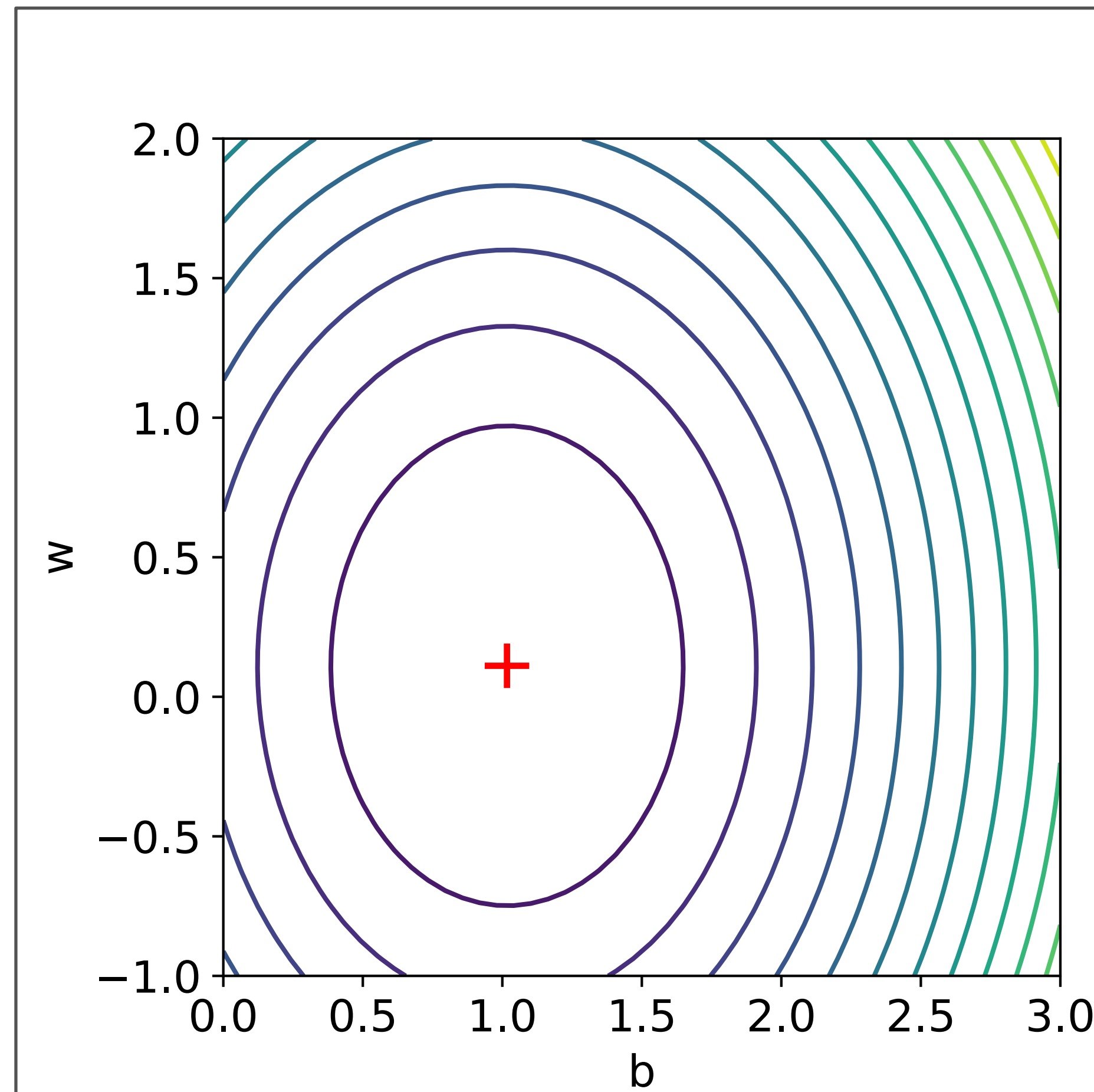


MSE loss

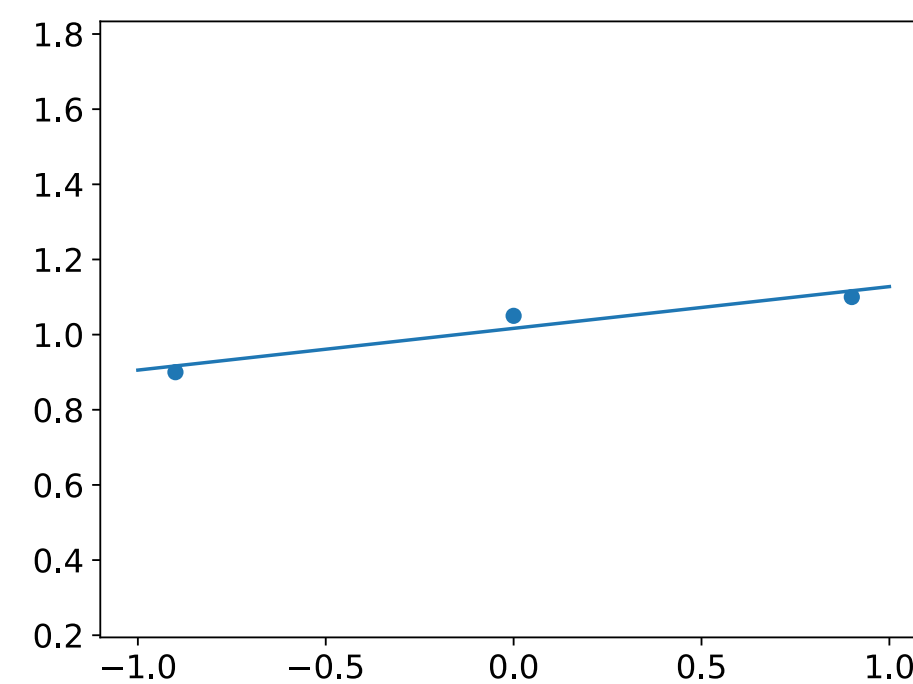
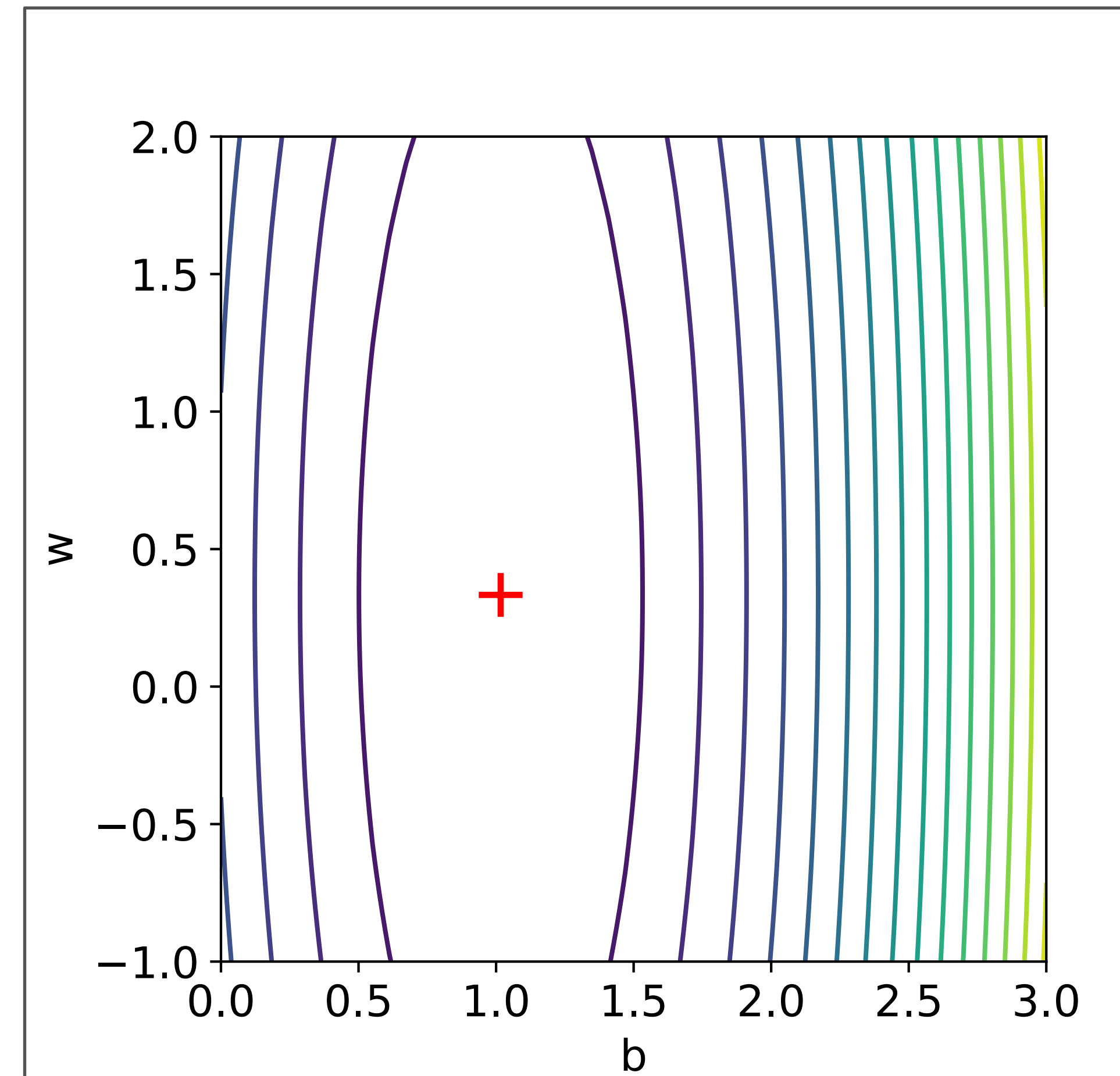
*non-unique solution is
the extreme example
of poor conditioning*



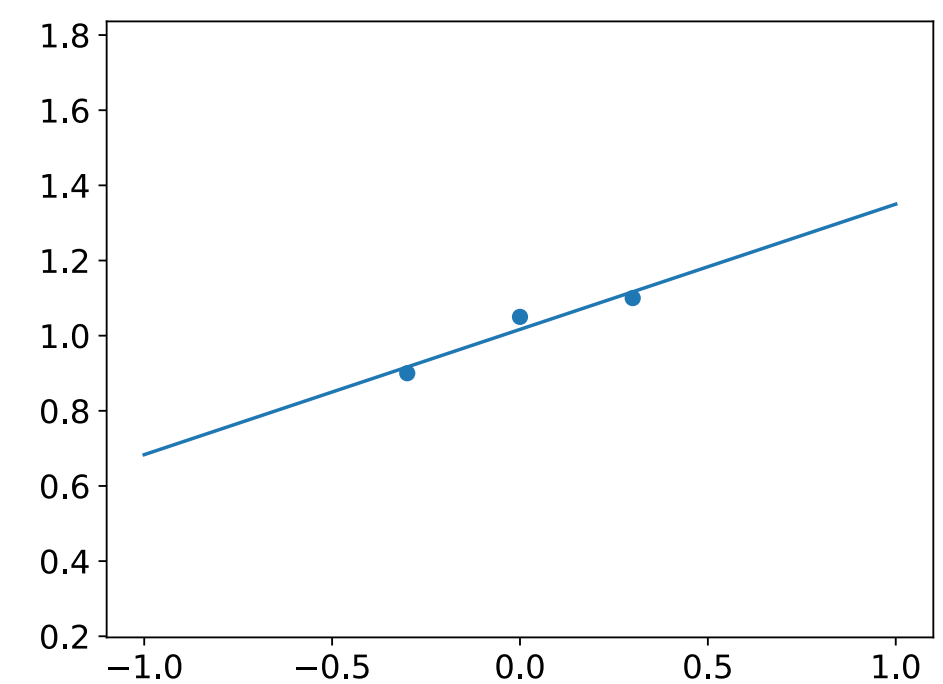
MSE loss



*non-unique solution is
the extreme example
of poor conditioning*

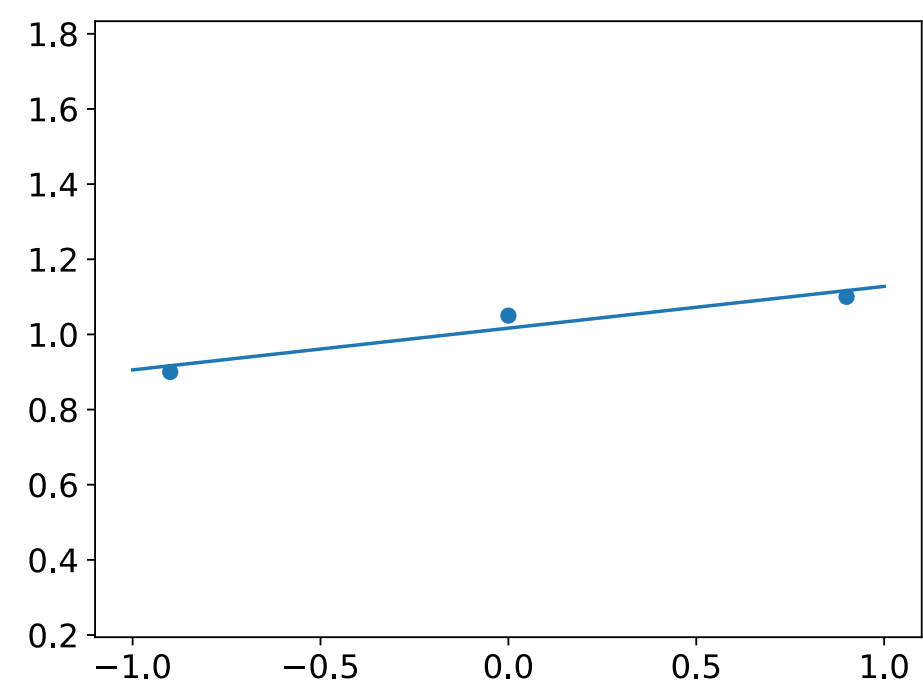


well conditioned

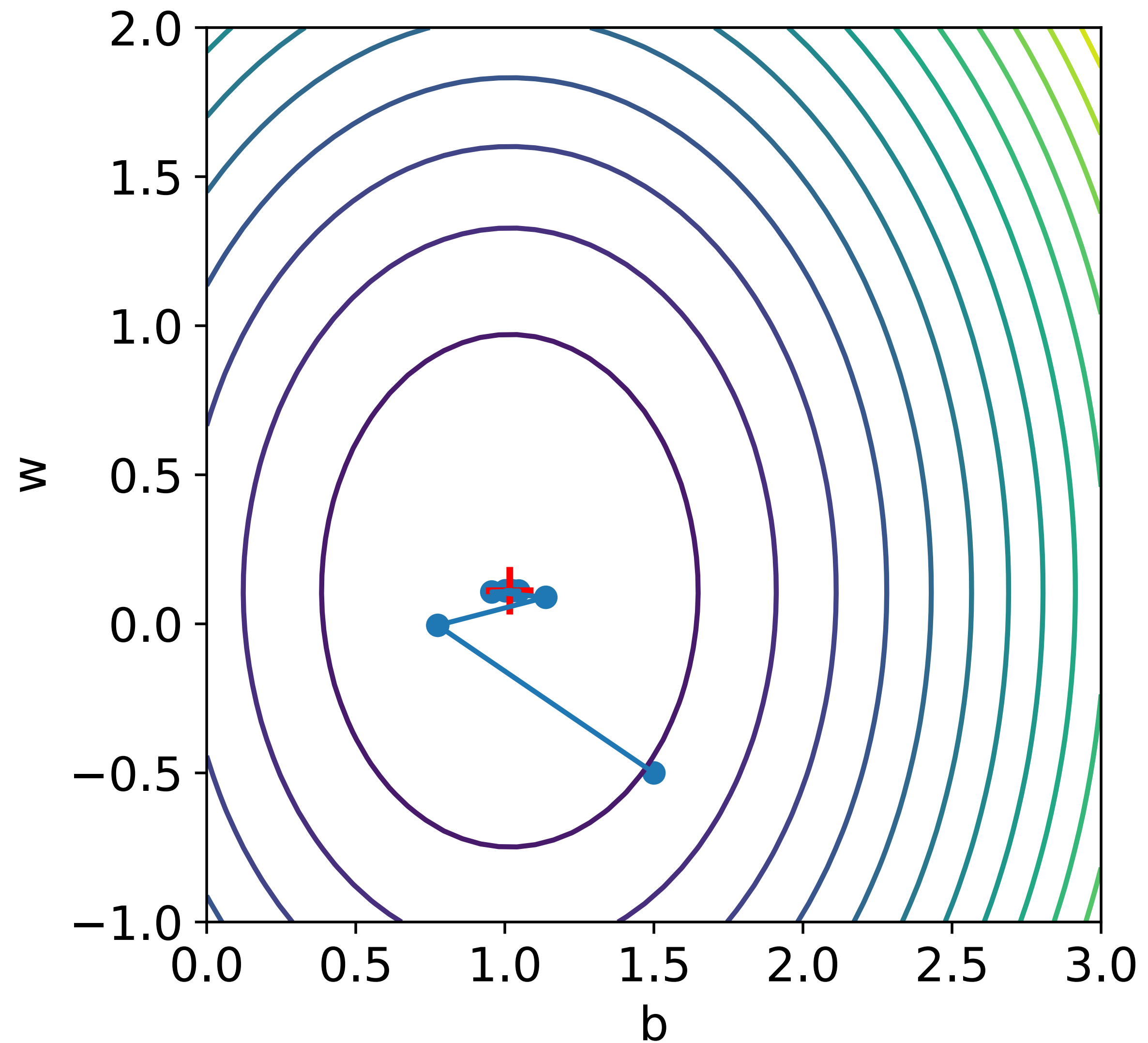


poorly conditioned

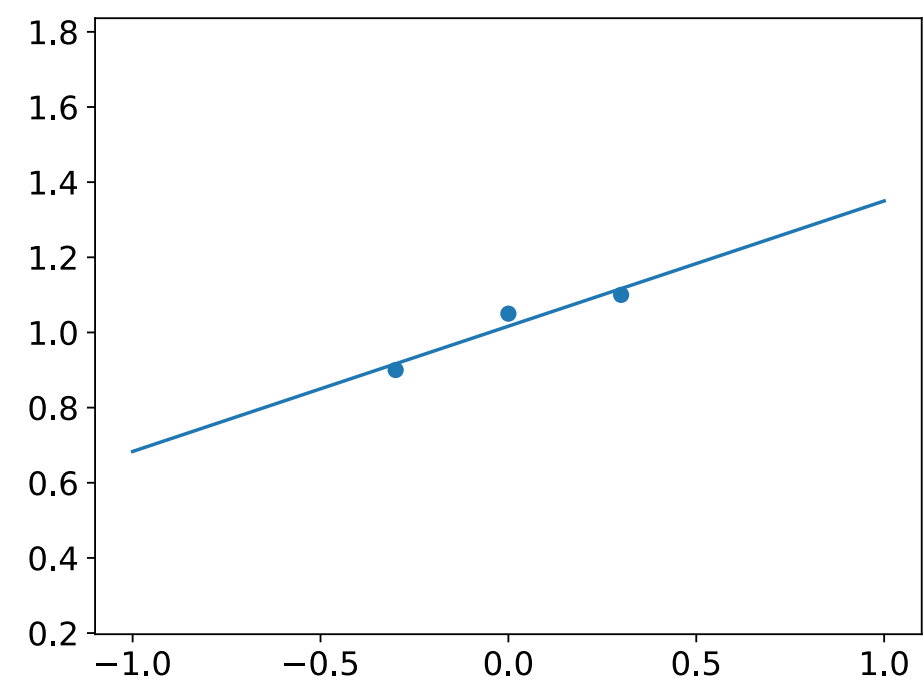
Gradient descent (good conditioning)



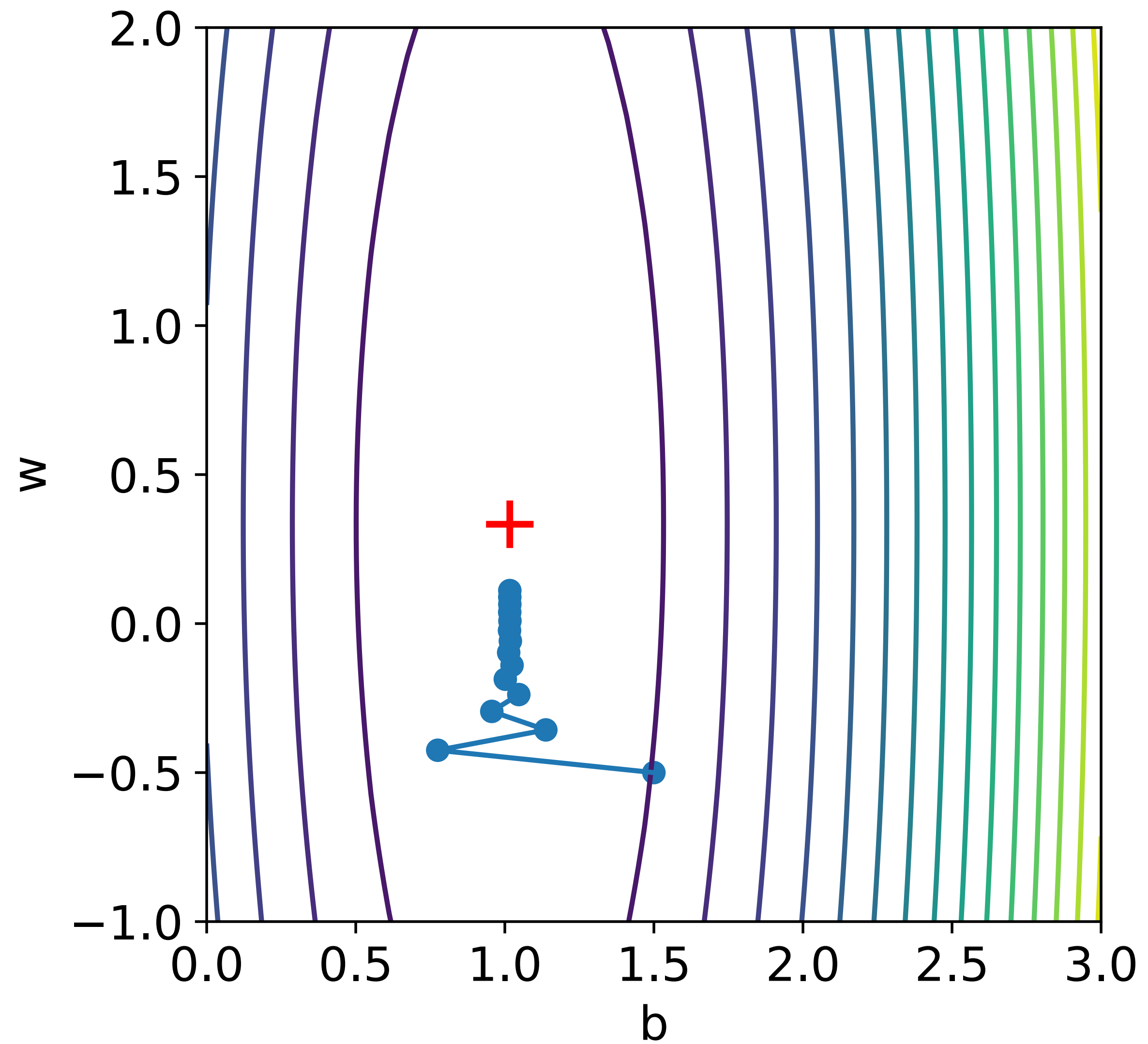
learning rate 0.5



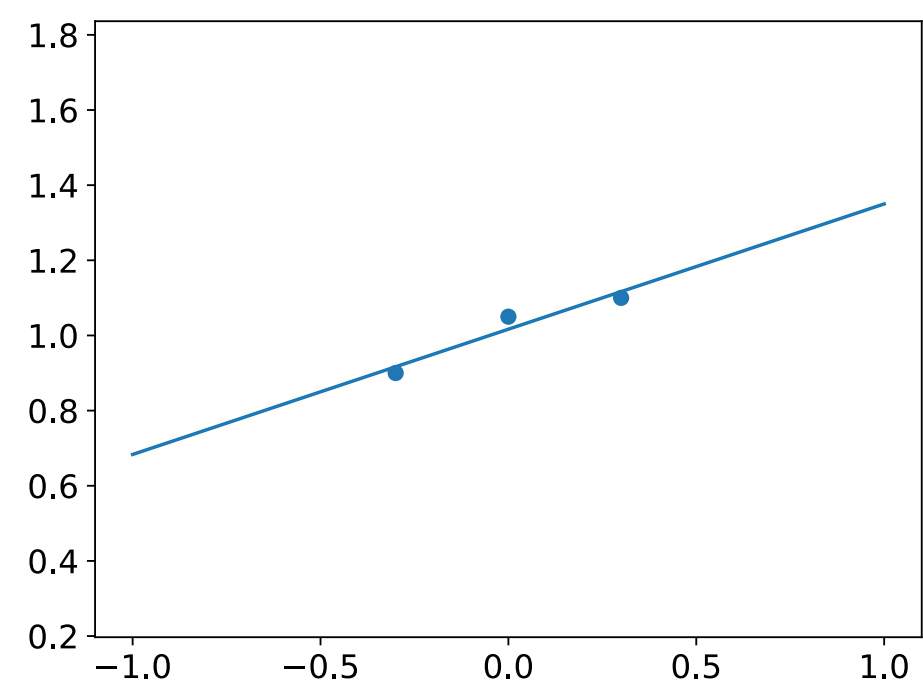
Gradient descent (poor conditioning)



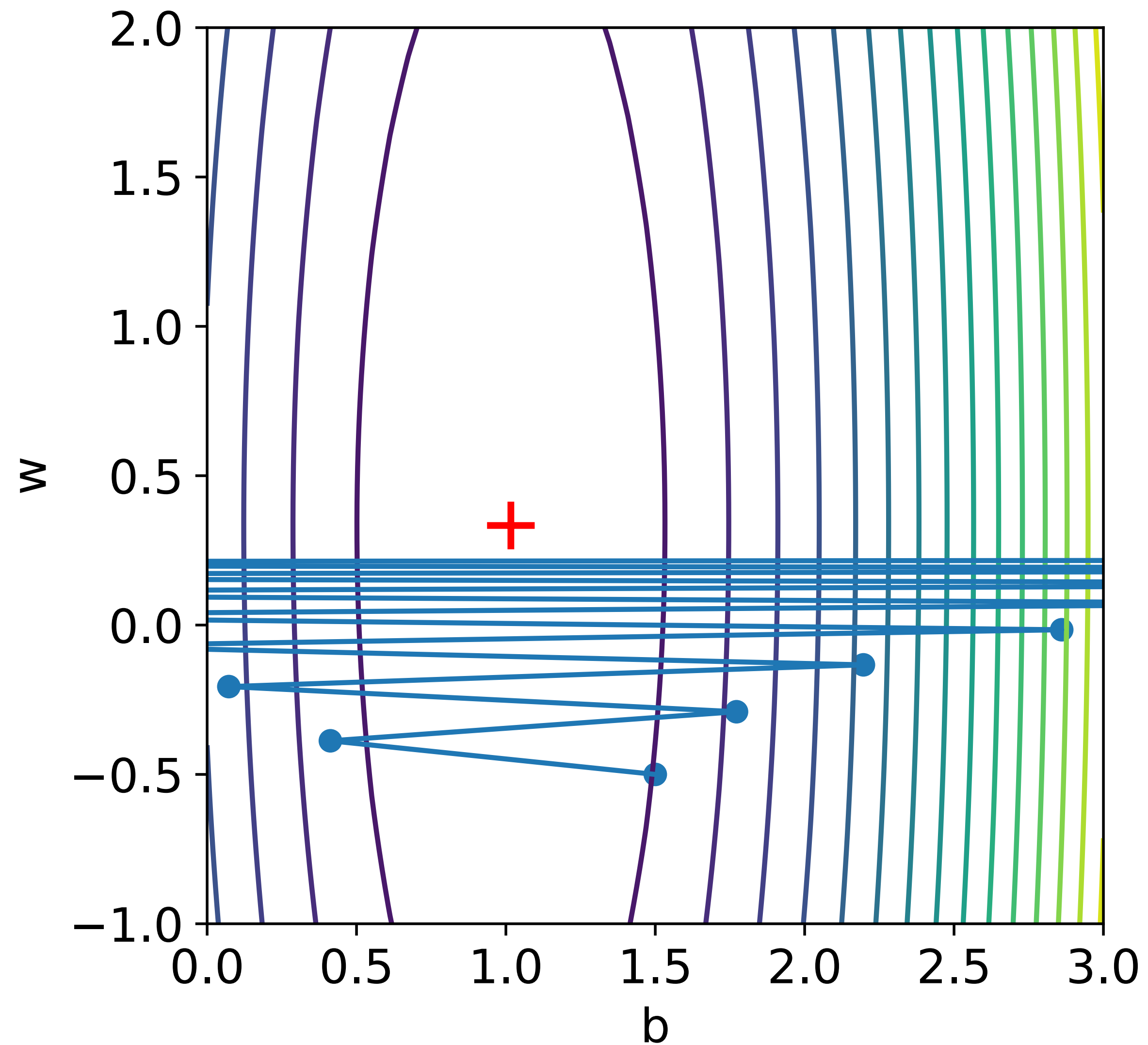
learning rate 0.5



Gradient descent (poor conditioning)



learning rate 0.75



What could we do?

- For horizontal direction (b), want a small learning rate to prevent oscillation
- For vertical direction (w), want a large learning rate to make fast enough progress
- Wouldn't it be great if we could automatically scale down the learning rate in directions where we start to oscillate?
 - ▶ then we'd be free to set learning rate high enough to make progress in other dimensions

EMA

Time scale: $\frac{1}{1 - \beta}$
e.g., $\beta = 0.95 \rightarrow$ about 20 steps

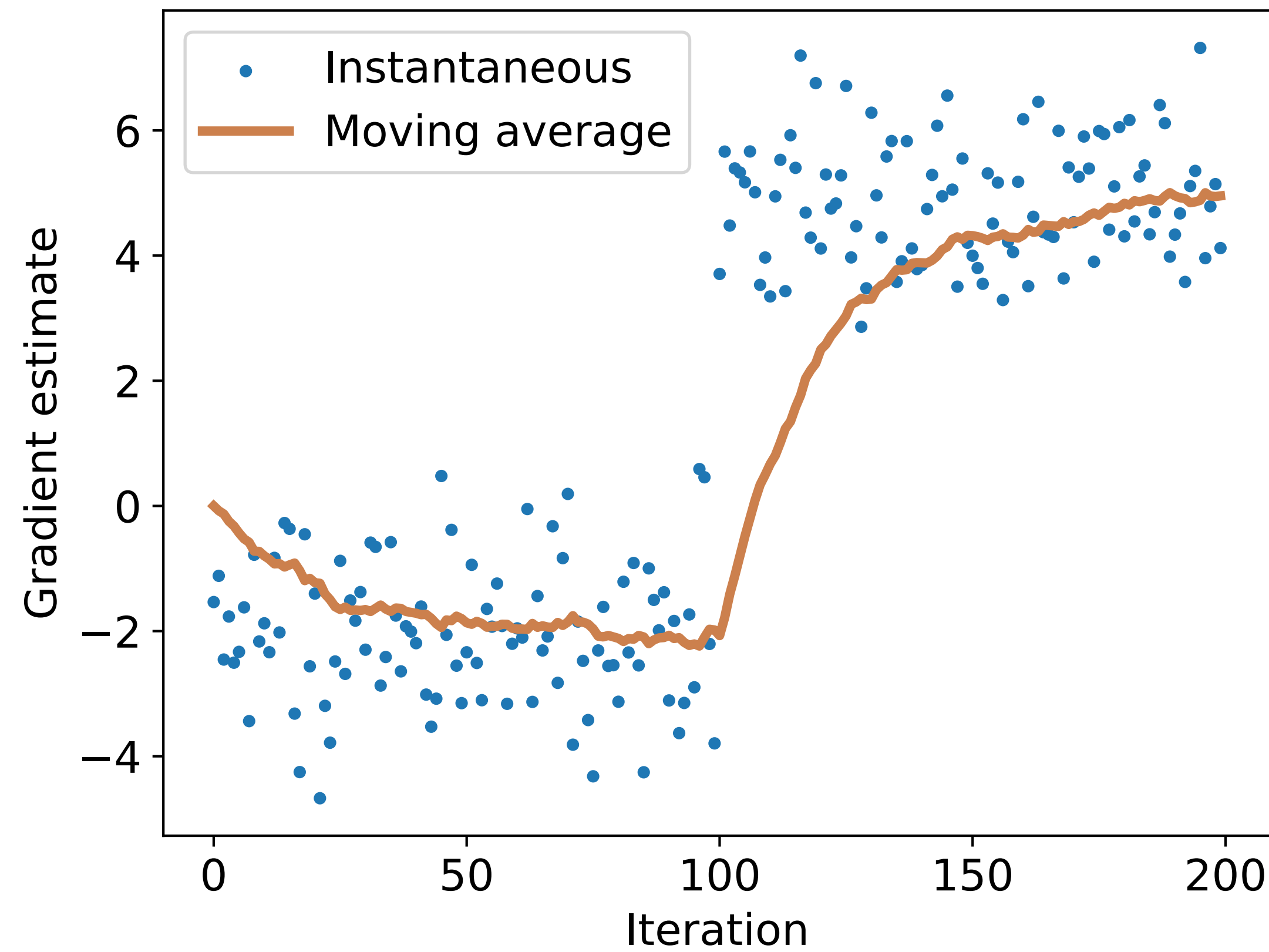
- Exponential moving average

if gradients always point $\uparrow$

if gradients alternate $\uparrow \downarrow$

- Use EMA instead of gradient in GD: called ***momentum***

EMA transient



$$\beta = 0.95$$

- EMA parameter β trades off *noise* vs. *transient*

Gradient descent with momentum

Called momentum because we tend to keep moving in same direction (even if gradient changes)

This is Polyak or heavy-ball momentum; others exist, notation may vary, documentation not always clear

Algorithm: gradient descent w/ momentum

Inputs: loss function $L(\theta)$, initial value $\theta^{(1)}$, step size η , momentum β

$\bar{g} \leftarrow 0$

For $i \leftarrow 1, 2, \dots$:

if **converged** break

$g \leftarrow \nabla L(\theta^{(i)})$

$\leftarrow$ for linear/ridge regression, $\lambda w + X^T(Xw - Y)$

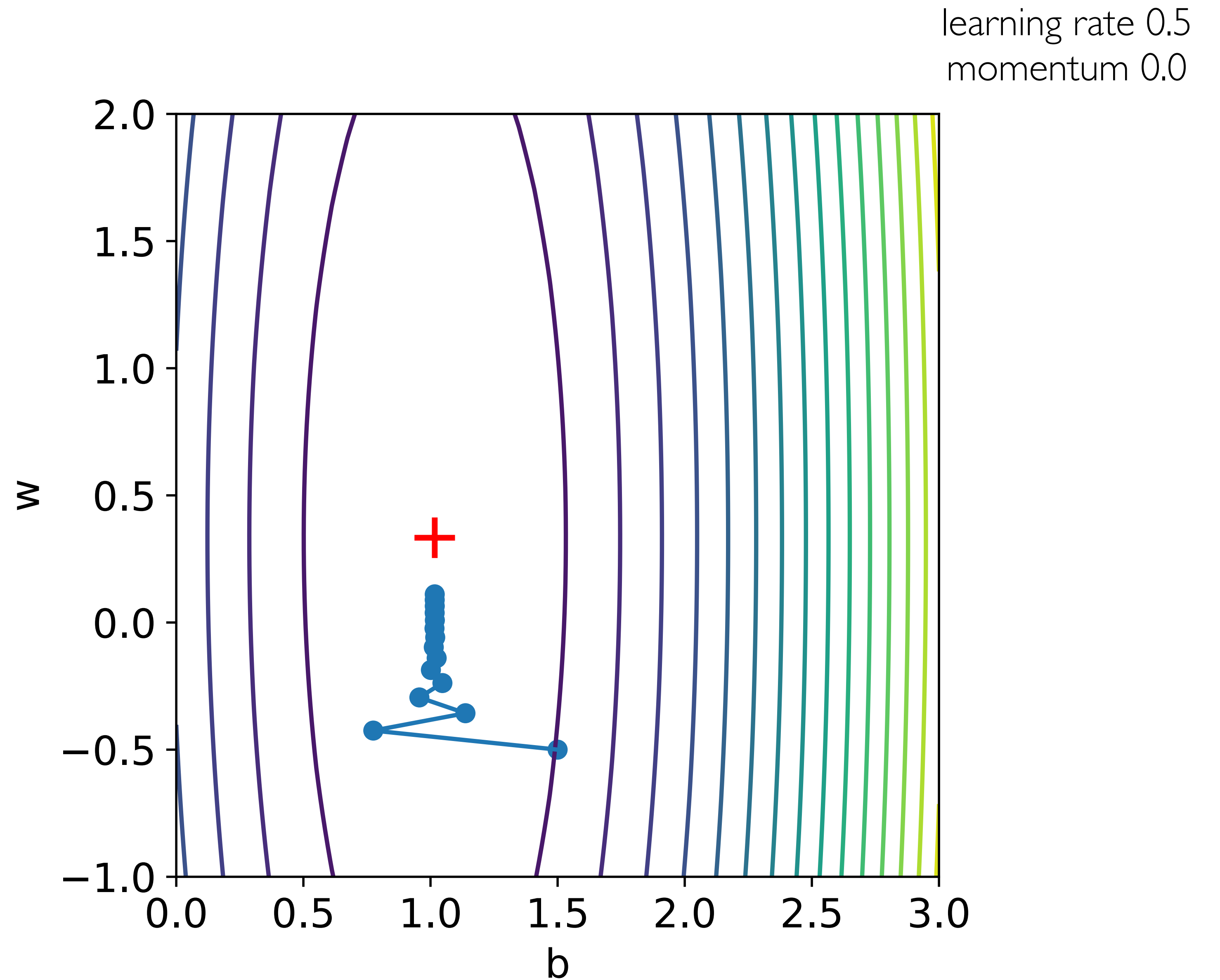
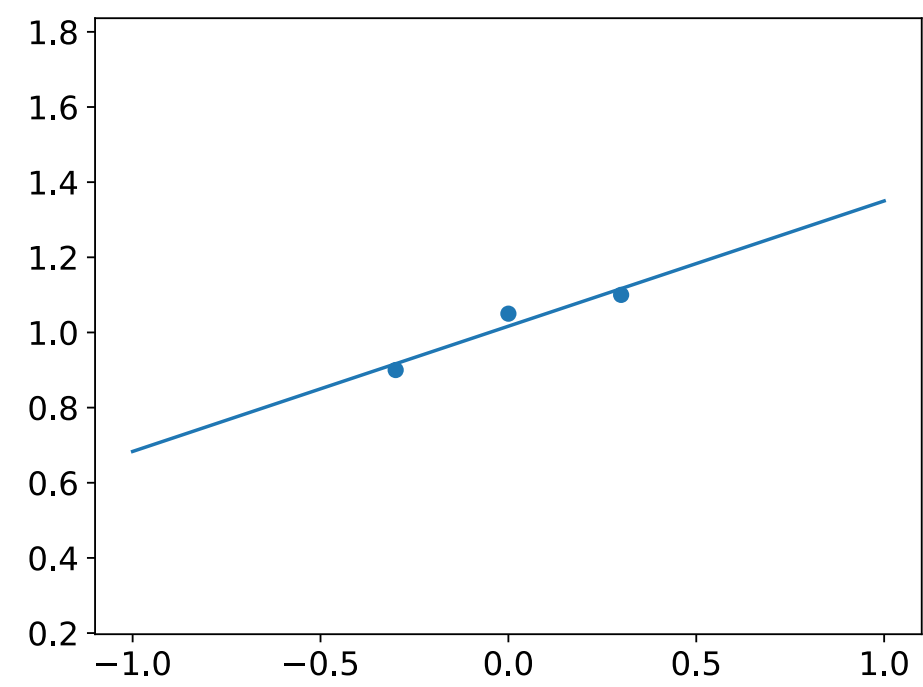
$\bar{g} \leftarrow \beta \bar{g} + (1 - \beta)g$

$\theta^{(i+1)} \leftarrow \theta^{(i)} - \eta \bar{g}$

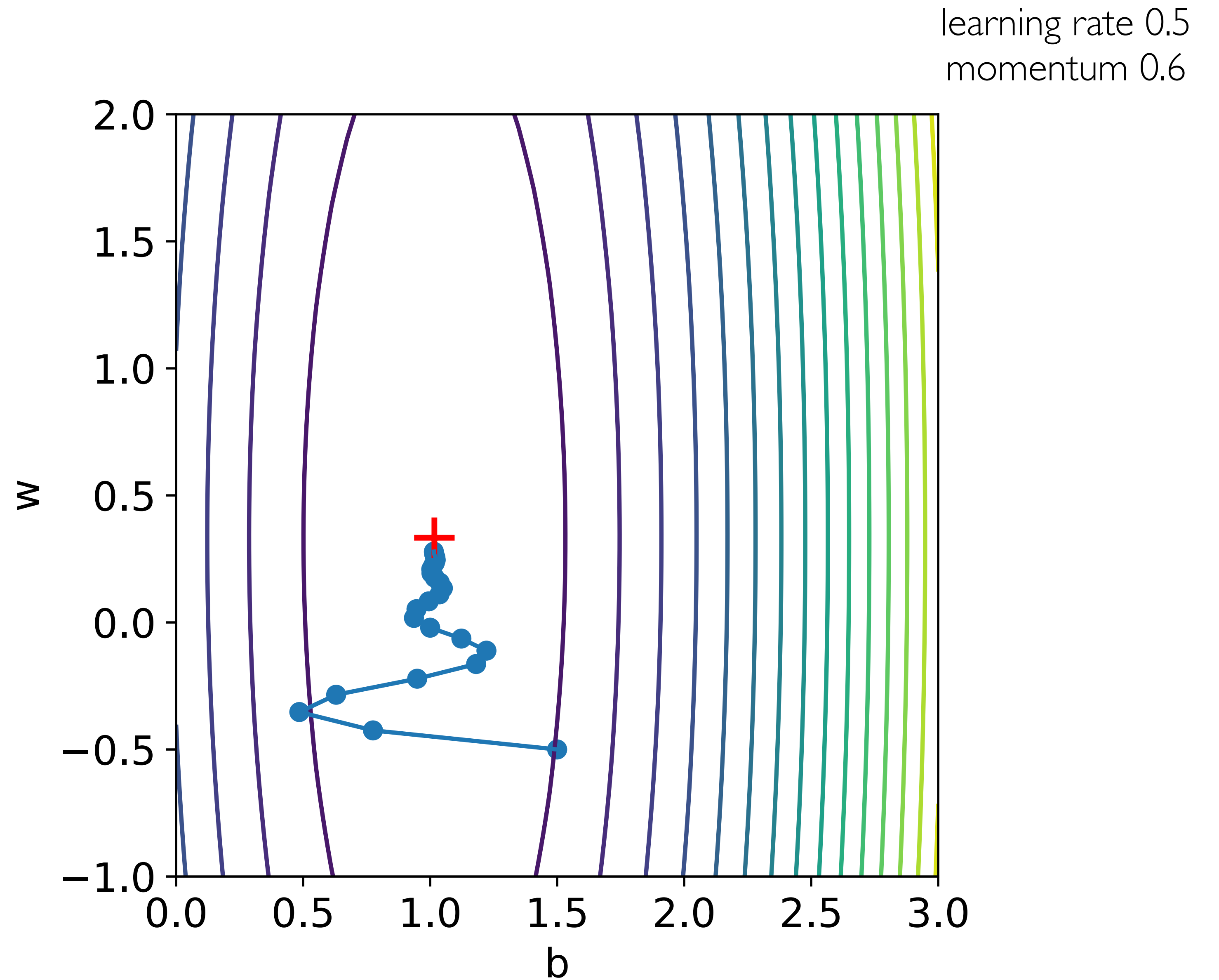
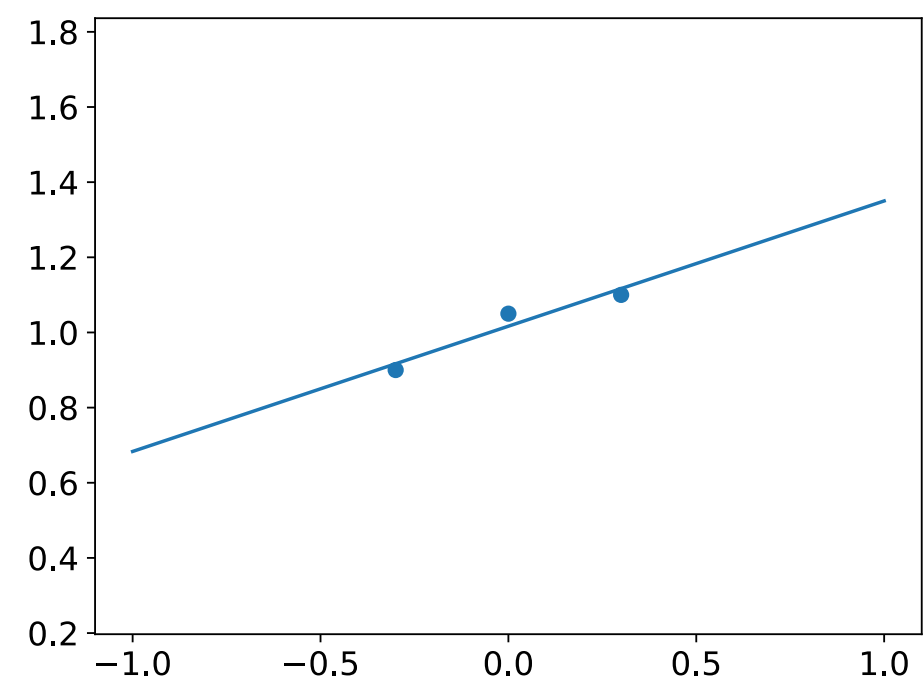
return $\theta^{(i)}$

- Similar convergence checks as above
- $\beta = 0$ is plain GD
- Variant: $\eta^{(i)}$ and $\beta^{(i)}$ (e.g., $\beta^{(i)} = 0, \beta, \beta, \beta, \dots$)
- Variant: scale $\bar{g}$ by $\frac{1}{1 - \beta^i}$ on step i

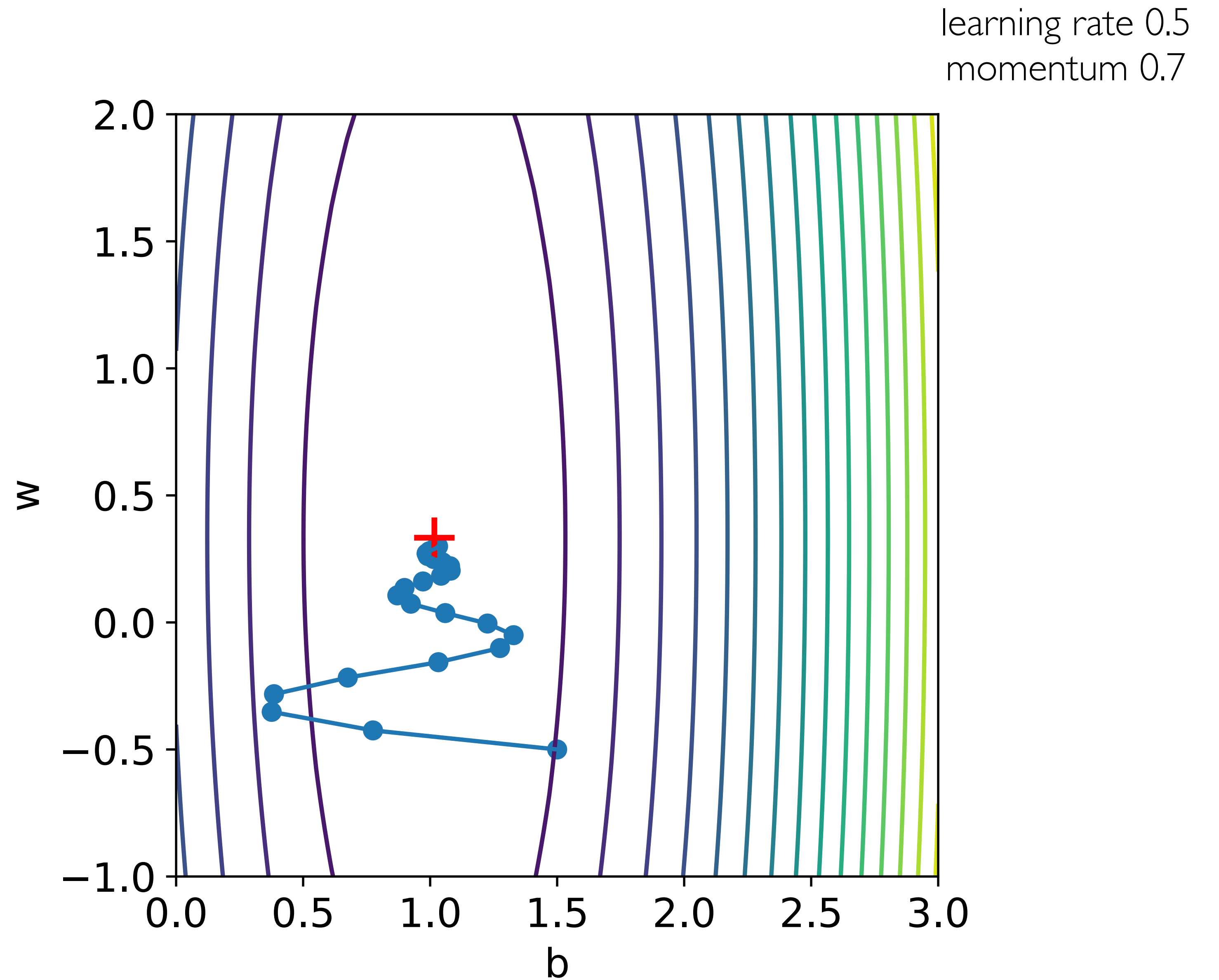
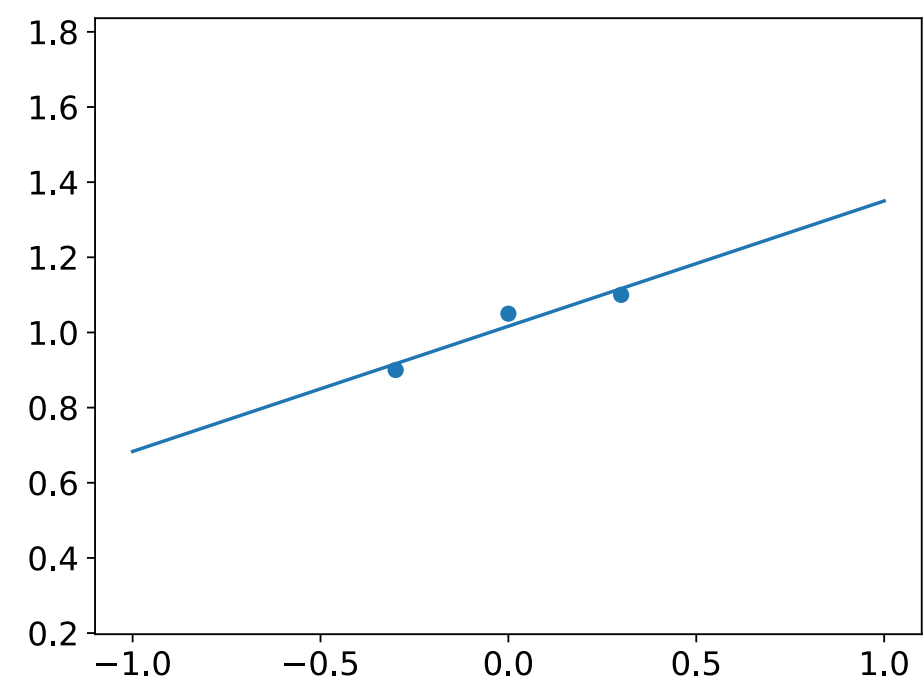
Gradient descent + momentum (poor conditioning)



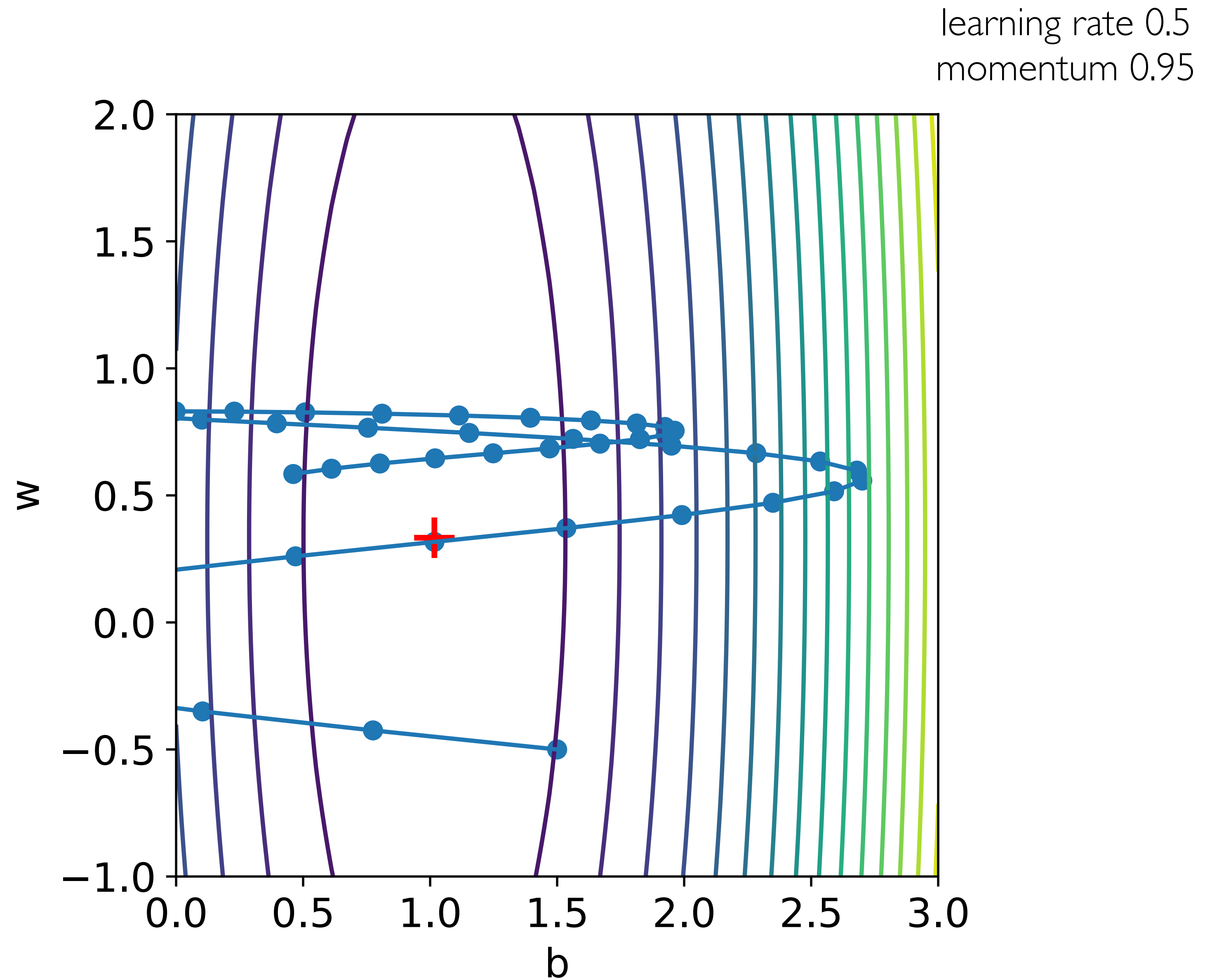
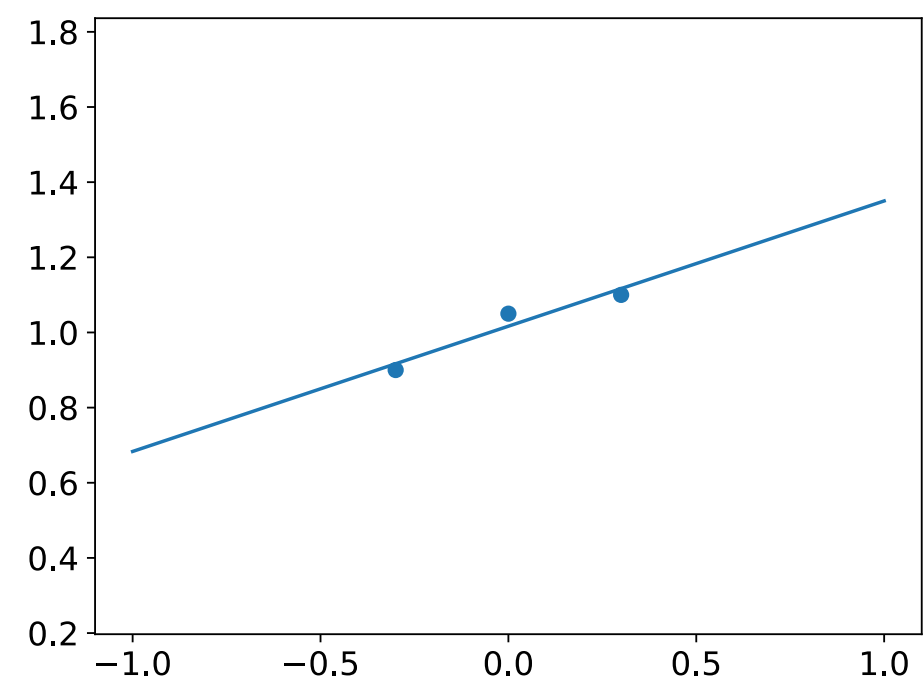
Gradient descent + momentum (poor conditioning)



Gradient descent + momentum (poor conditioning)



Gradient descent + momentum (poor conditioning)



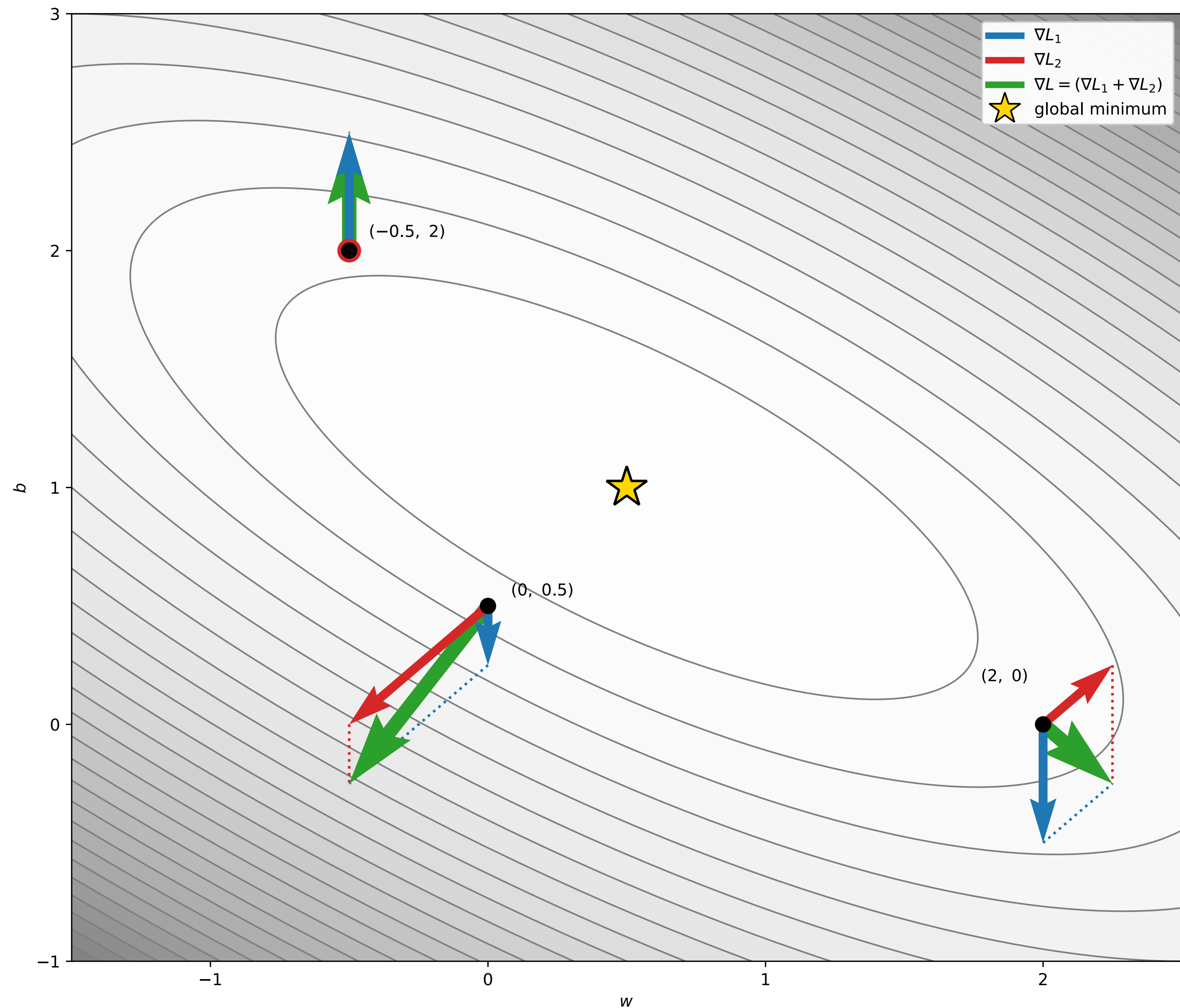
Scaling to big data

- When number of examples N is large, both gradient descent and exact method are slow
 - ▶ $O(Nk)$ per iteration vs. $O(Nk^2)$ total
- Need an algorithm with better dependence on N

Gradient is a sum

Not just that, but each term in the sum is iid

$$L(w, b) = \frac{1}{2}[(b - 1)^2 + (w + b - \frac{3}{2})^2]$$



One random example is enough

- Given a sum $L(\theta) = L_1(\theta) + \dots + L_N(\theta)$
- Factor out N to get an average:
 - ▶ $\frac{1}{N}L(\theta) = \frac{1}{N}L_1(\theta) + \dots + \frac{1}{N}L_N(\theta)$
- RHS has the form of an expectation:
 - ▶ sample an index k uniformly from $\{1 : N\}$
 - ▶ $\frac{1}{N}L(\theta) = \mathbb{E}_k(L_k(\theta))$
- Since gradient and expectation are both linear,
 - ▶ $\frac{1}{N} \nabla L(\theta) = \mathbb{E}_k(\nabla L_k(\theta))$
- That is: if we go in the direction suggested by *a single random* training example, then we *on average* go in the correct direction for the entire training set

Stochastic gradient descent

Algorithm: stochastic gradient descent w/ momentum

Inputs: loss function $L(\theta)$, initial value $\theta^{(1)}$, step size η , momentum β

$\bar{g} \leftarrow 0$

For $i \leftarrow 1, 2, \dots$:

if **converged** break

$k \leftarrow \text{Uniform}(\{1 : N\})$

$g \leftarrow \nabla L_k(\theta^{(i)}) \quad \leftarrow \text{for linear/ridge regression, } \frac{\lambda}{N}w + X^{(k)}(X^{(k)} \cdot w - Y^{(k)})$

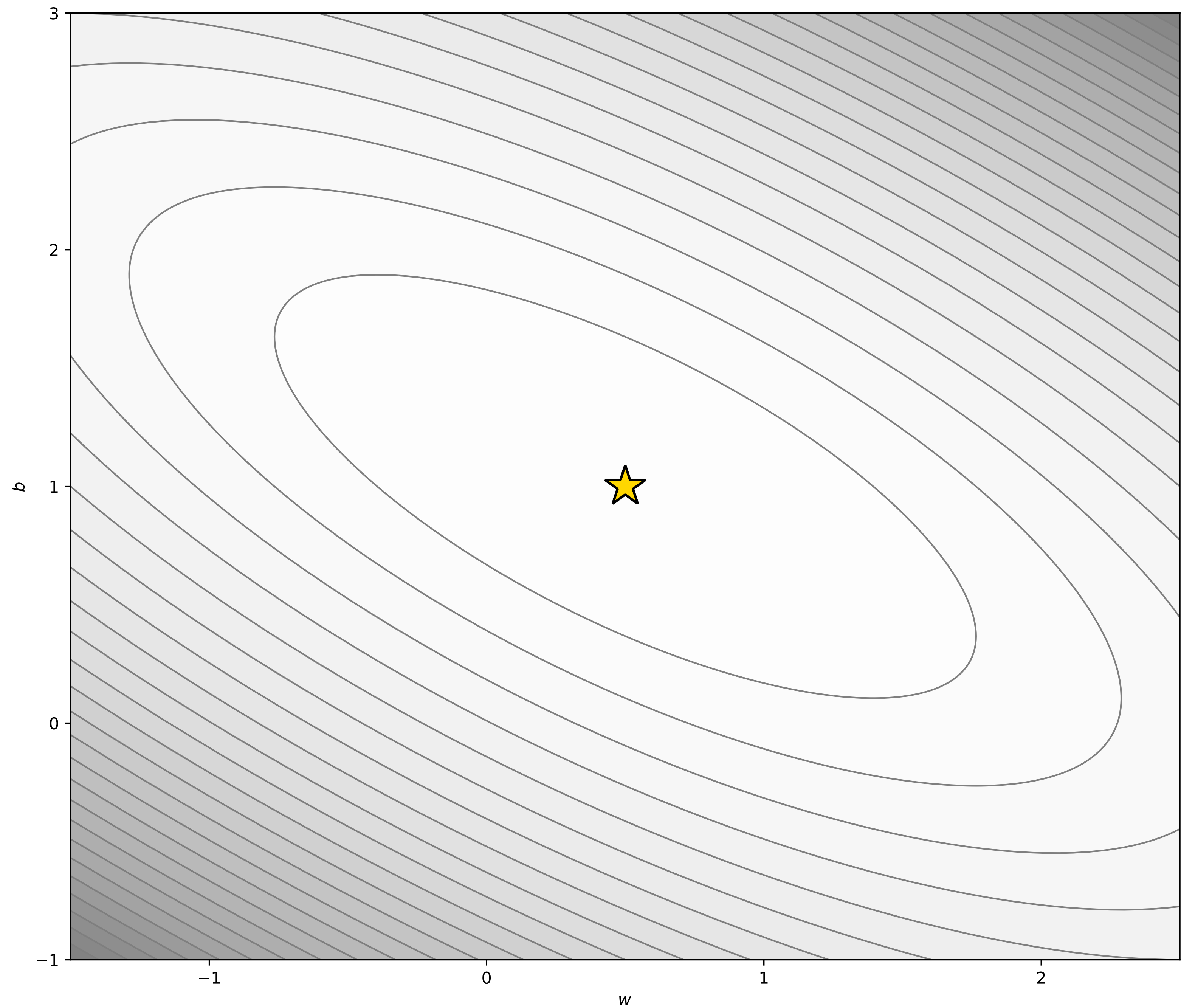
$\bar{g} \leftarrow \beta \bar{g} + (1 - \beta)g$

$\theta^{(i+1)} \leftarrow \theta^{(i)} - \eta \bar{g} \quad \leftarrow \text{we have absorbed a factor } N \text{ into learning rate}$
return $\theta^{(i)}$ (not a universal convention!)

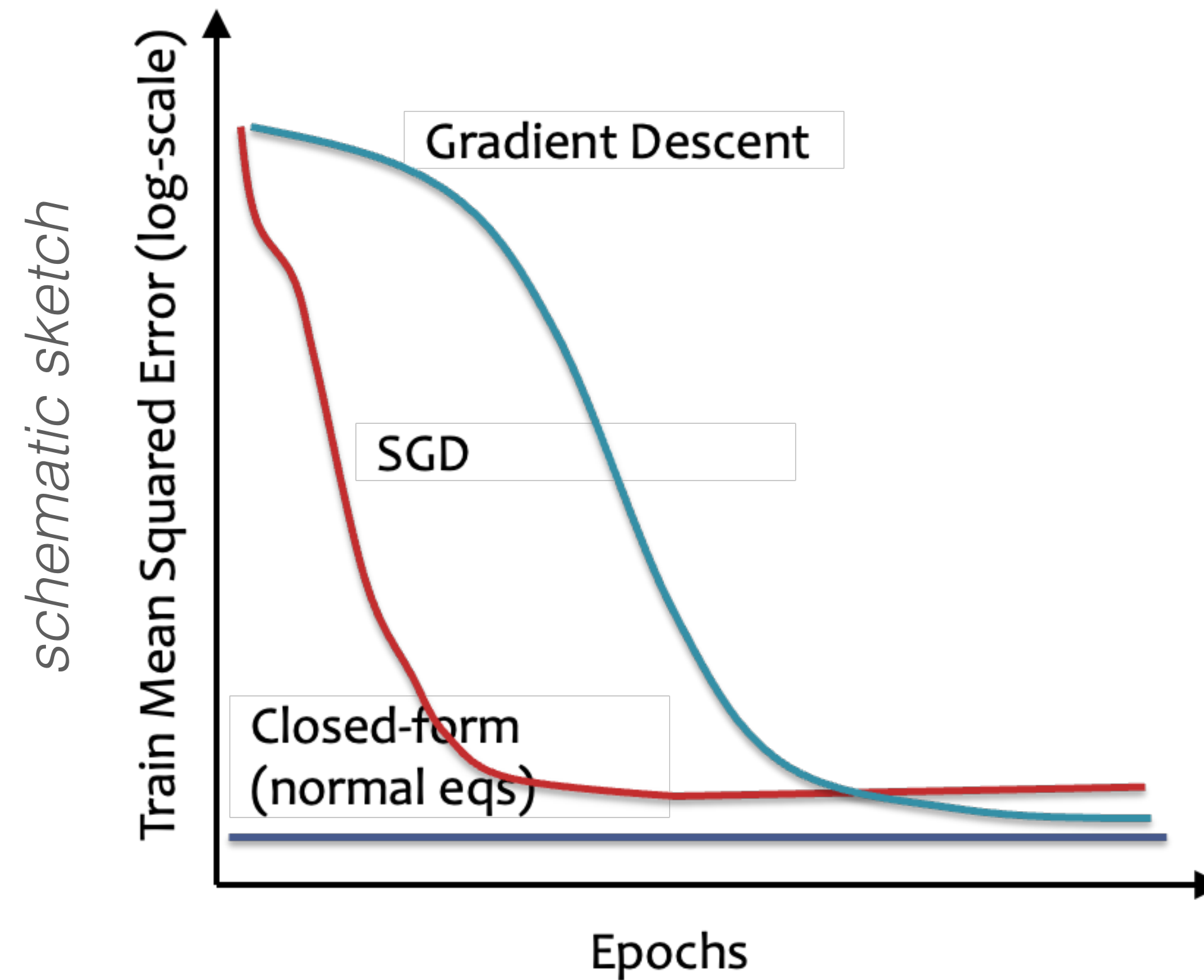
- If ridge $\lambda > 0$, common to split this term among all examples
 - ▶ pulls w back toward 0 — called **weight decay** *can exempt bias from decay*
- $\beta = 0$ gives SGD w/o momentum
- We typically write (and analyze) $k \leftarrow \text{Uniform}(\{1 : N\})$, but more commonly we shuffle and scan through entire training set (an **epoch**), then repeat — works noticeably better

GD vs. SGD

*Curvature now places a
sharper limit on learning rate:
both noise and stepping too
far can increase loss*



GD vs. SGD



- SGD can start improving very quickly: even before GD has completed its first epoch
- But SGD is limited by a *noise floor*: bounces around the optimum rather than smoothly approaching it
 - ▶ reducing the learning rate over time can help

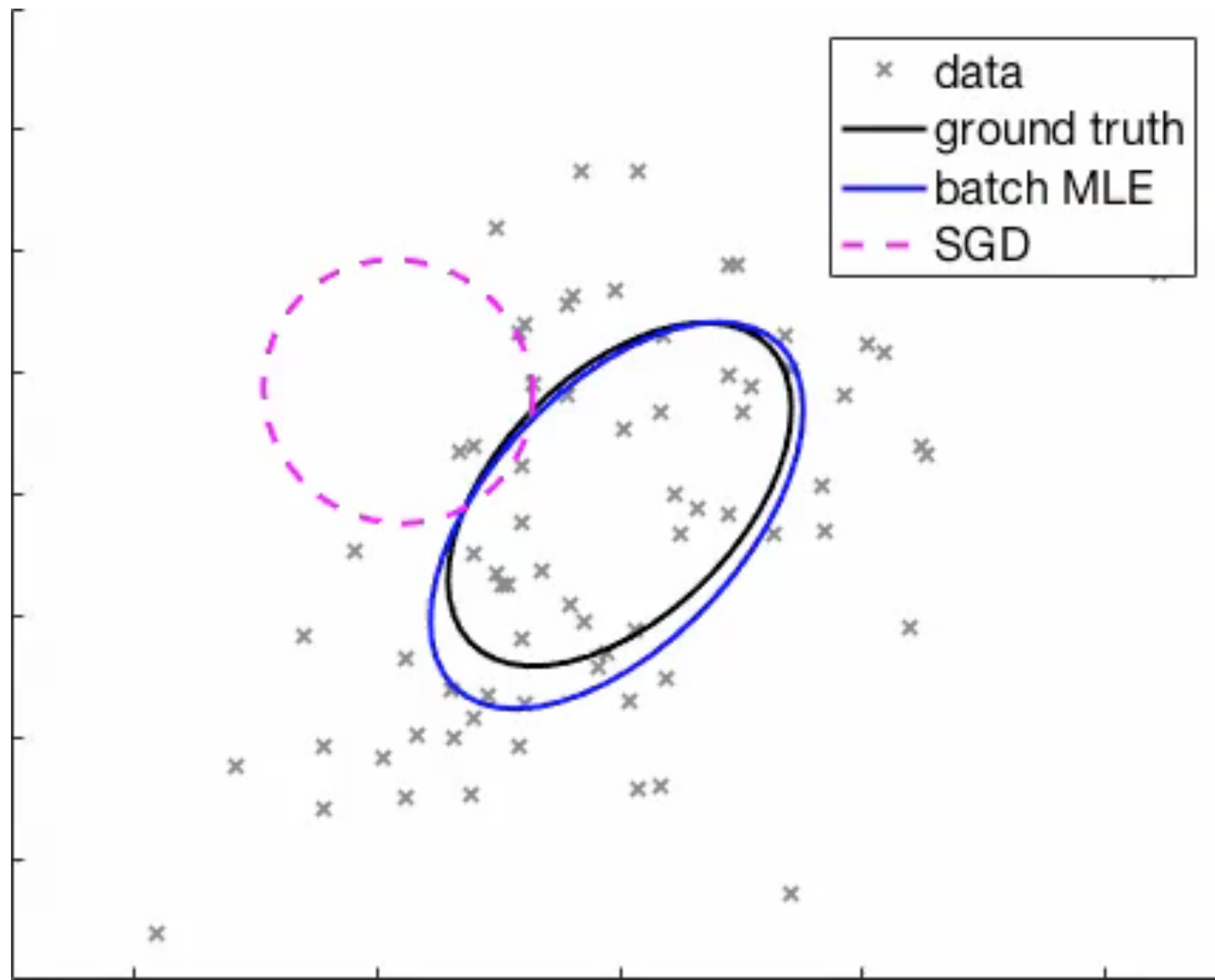
GD vs. SGD

Bounds for special case: strongly convex function, $\nabla^2 L \succeq \rho I$

Method	Steps to convergence	Cost per step
Gradient descent	$O(\ln \frac{1}{\epsilon})$	$O(Nk)$
SGD	$O(\frac{1}{\epsilon})$	$O(k)$

- Define convergence as $L(\theta^{(t)}) \leq L(\theta^*) + \epsilon$
- SGD takes more iterations (esp. for small ϵ), but each iteration is much faster
- At practical values of ϵ and N , SGD can win by a lot

SGD example



SGD phases

(not exact: soft transitions)

- Descent phase: gradient is large compared to sample noise
 - can happen if we're far from optimum
 - steady progress toward lower loss
 - larger learning rate is best (limited only by curvature of loss)
- Stochastic phase: gradient and noise are comparable magnitude
 - often happens as we get closer to optimum, or we might start in this phase
 - progress depends on keeping learning rate low enough to benefit from averaging
- Wandering phase: noise dominates gradient
 - little further progress in reducing loss, random walk in neighborhood of optimum
 - can go back to stochastic phase by dropping learning rate and/or increasing batch size

A problem

- We can parallelize GD by splitting the training set among processors (cluster machines or GPU cores)
- Doesn't work for SGD (why?)
- Naively, SGD can't run fast on a cluster or GPU

Minibatch SGD

Algorithm: minibatch SGD w/ momentum

Inputs: loss function $L(\theta)$, initial value $\theta^{(1)}$, step size η , momentum β

$\bar{g} \leftarrow 0$

For $i \leftarrow 1, 2, \dots$:

if **converged** break

$K \leftarrow \text{Uniform}(\{1 : N\}, B)$ $\leftarrow$ the set K (w/ B elements) is the minibatch

$g \leftarrow \frac{1}{B} \sum_{k \in K} \nabla L_k(\theta^{(i)})$ $\leftarrow$ same expectation but averages out noise

$\bar{g} \leftarrow \beta \bar{g} + (1 - \beta)g$

$\theta^{(i+1)} \leftarrow \theta^{(i)} - \eta \bar{g}$ $\leftarrow$ we still absorb a factor N into learning rate

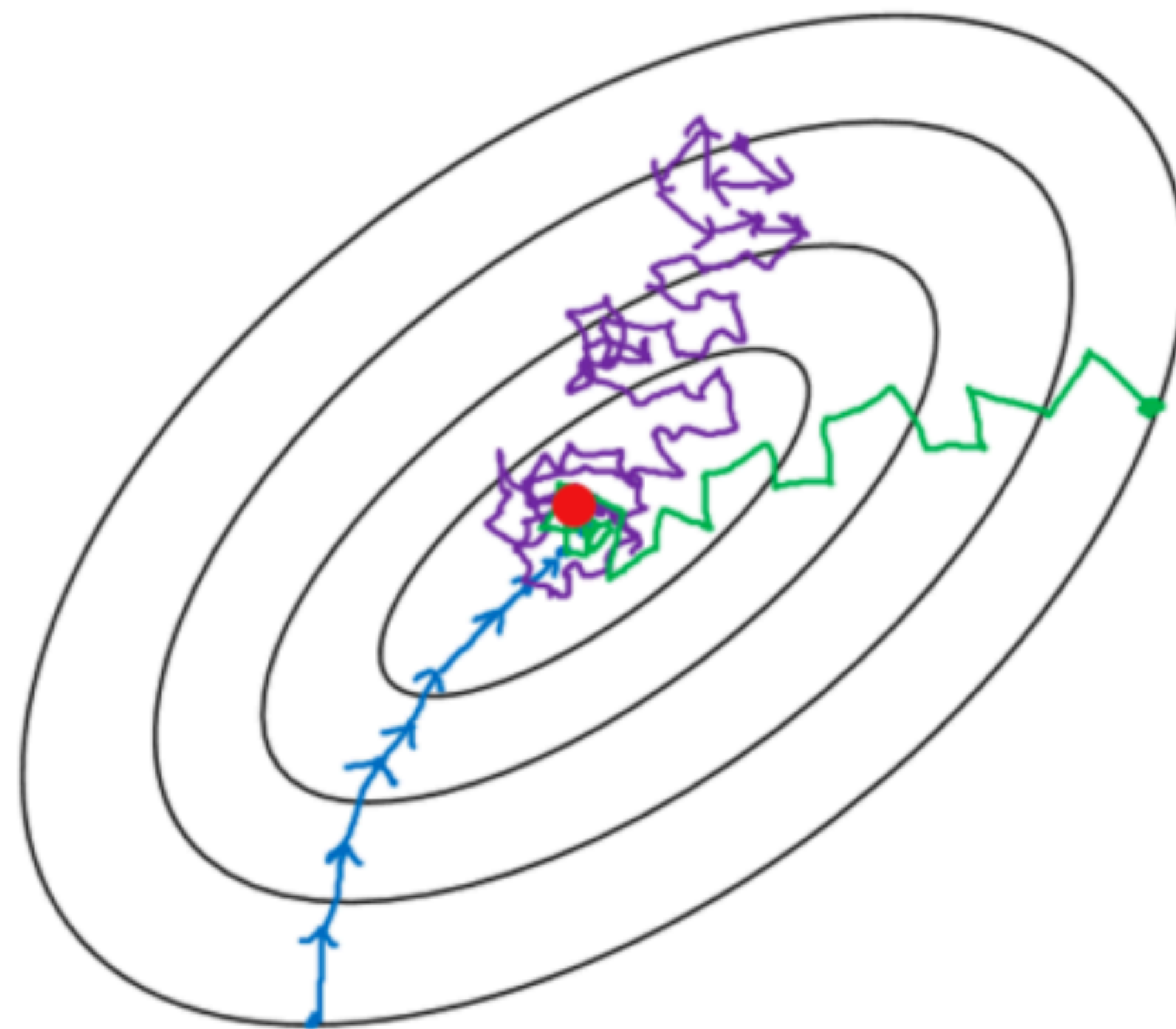
return $\theta^{(i)}$

- As before, shuffle and scan through data in each epoch
- A big enough minibatch can mostly-saturate a GPU
 - ▶ have to get fancier for a cluster since communication delays are much worse — e.g., asynchronous methods like parameter servers

Scaling batch size and learning rate

- For small learning rate and batch size, scaling $\eta \propto B$ leaves trajectory approximately unchanged
 - ▶ step direction becomes identical when $\eta \rightarrow 0$
- This scaling also leaves compute unchanged: two steps at η, B cost the same as one step at $2\eta, 2B$
- As we increase η we start to become curvature-limited
 - ▶ best η (for 1-step decrease in L) goes up slower than B
 - ▶ total work for a given improvement in L goes up
 - ▶ but bigger B means better parallelization, so wall-clock time can still go down
- Around *critical batch size*, scaling is $\eta \propto \sqrt{B}$
 - ▶ CBS: noise and deterministic curvature match effect sizes

Gradient descent variants



- $B = N$ gradient descent
- $B \ll N$ minibatch SGD
- $B = 1$ SGD

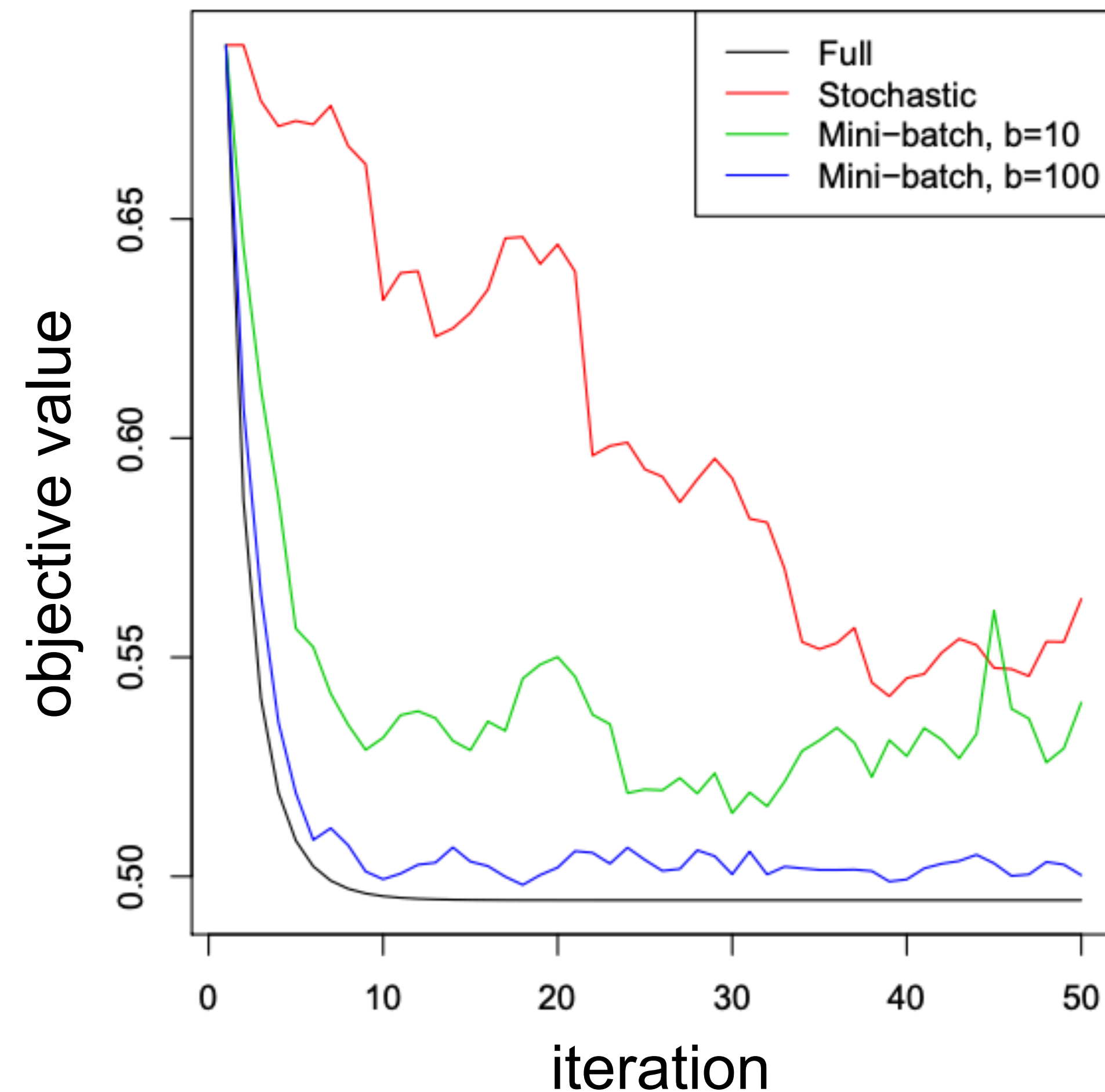
Smaller B takes more iterations, but
each iteration is cheaper

SGD vs. Mb-SGD vs. GD

Compare **number of iterations**
vs. **objective value**

- This ex: logistic regression with $n = 10,000, k = 20$
- Fixed step size

→ *Larger batches need fewer iterations, make convergence more stable*

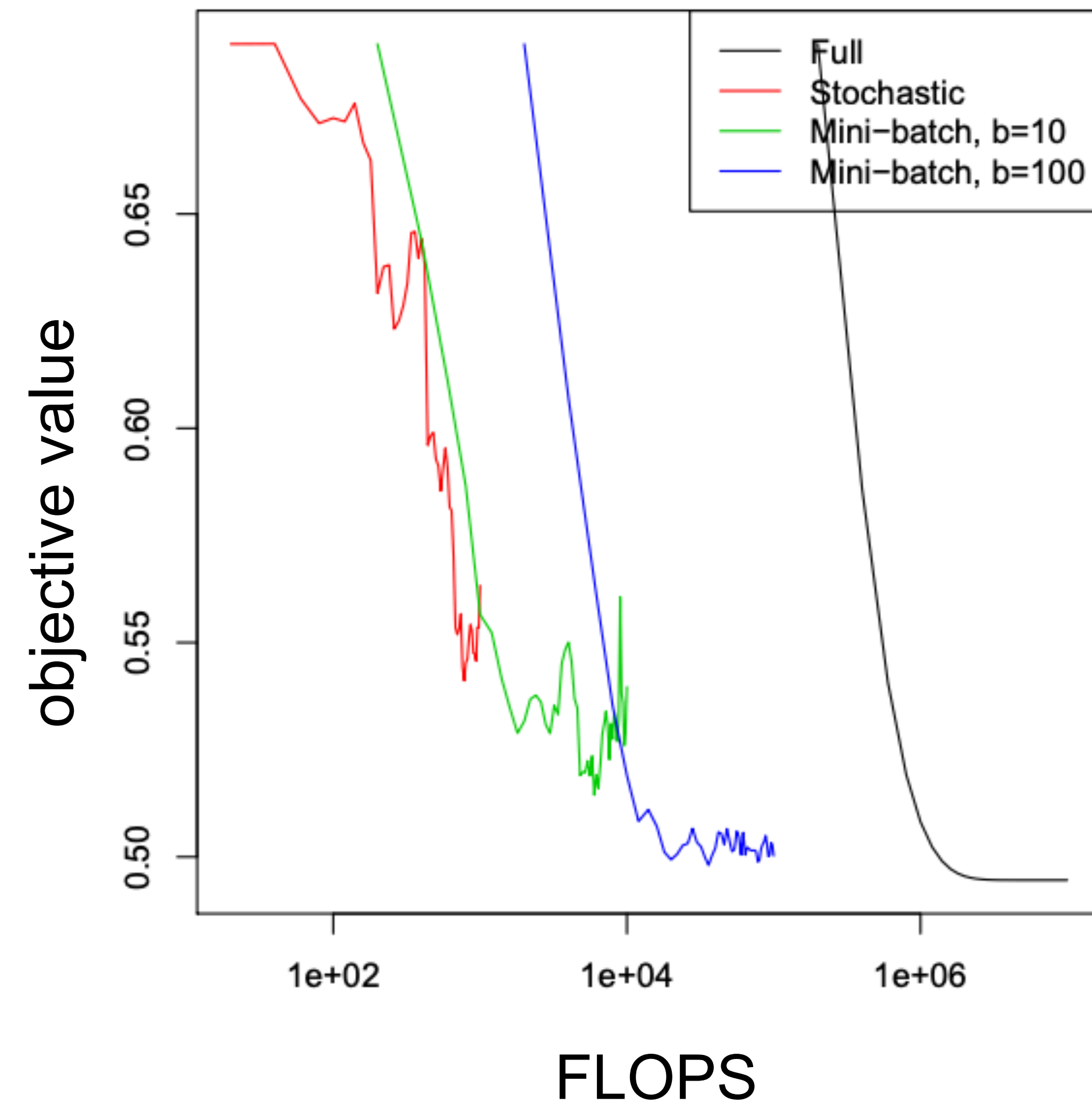


SGD vs. Mb-SGD vs. GD

Compare **FLOPs** vs. **objective value**

- FLOPs = floating point operations (compute cost)

→ *Although Mb-SGD converges more slowly than GD terms of iterations, it may converge more quickly in terms of FLOPs*

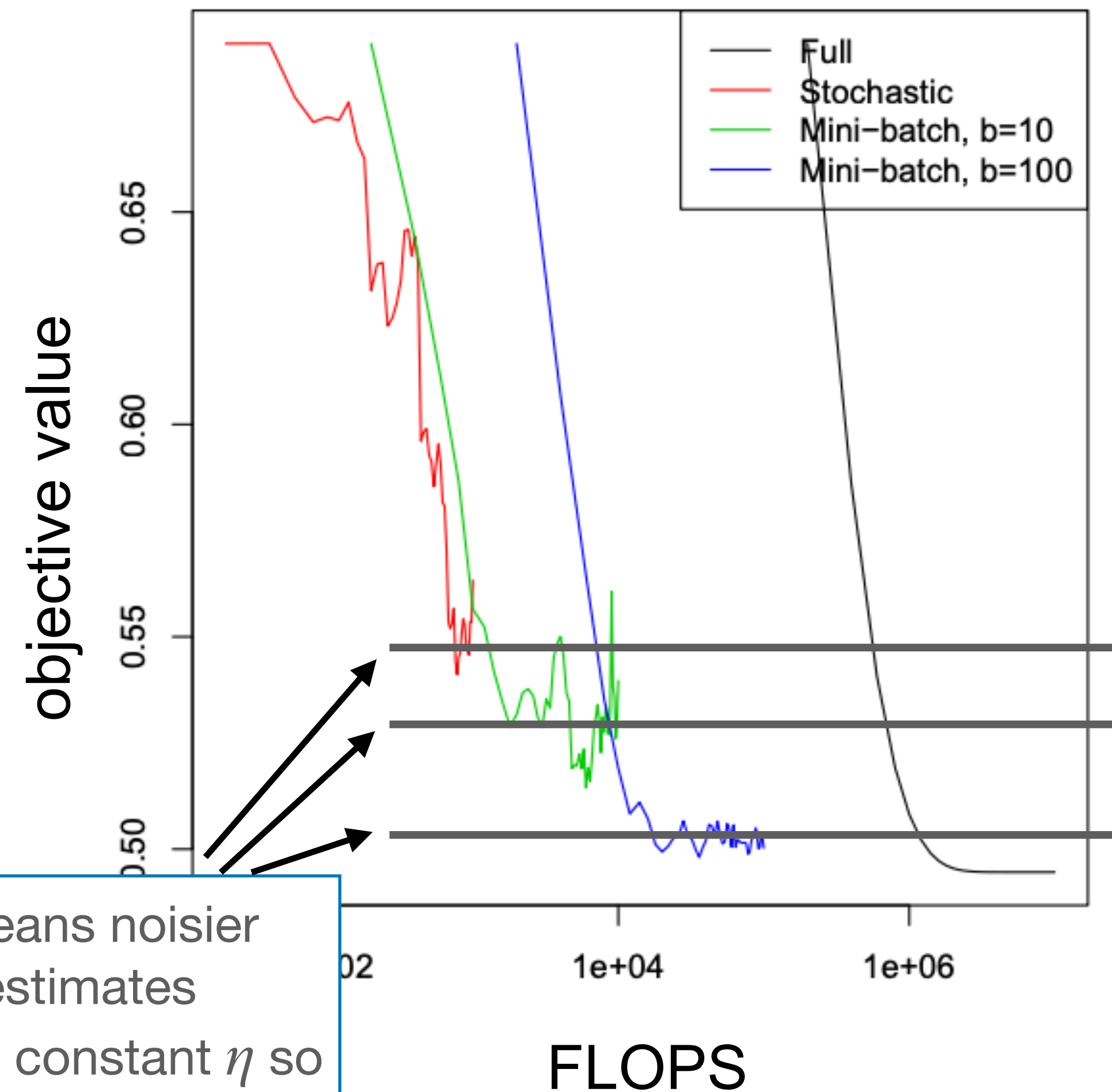


SGD vs. Mb-SGD vs. GD

Compare **FLOPs** vs. **objective value**

- FLOPs = floating point operations (compute cost)

→ *Although Mb-SGD converges more slowly than GD terms of iterations, it may converge more quickly in terms of FLOPs*



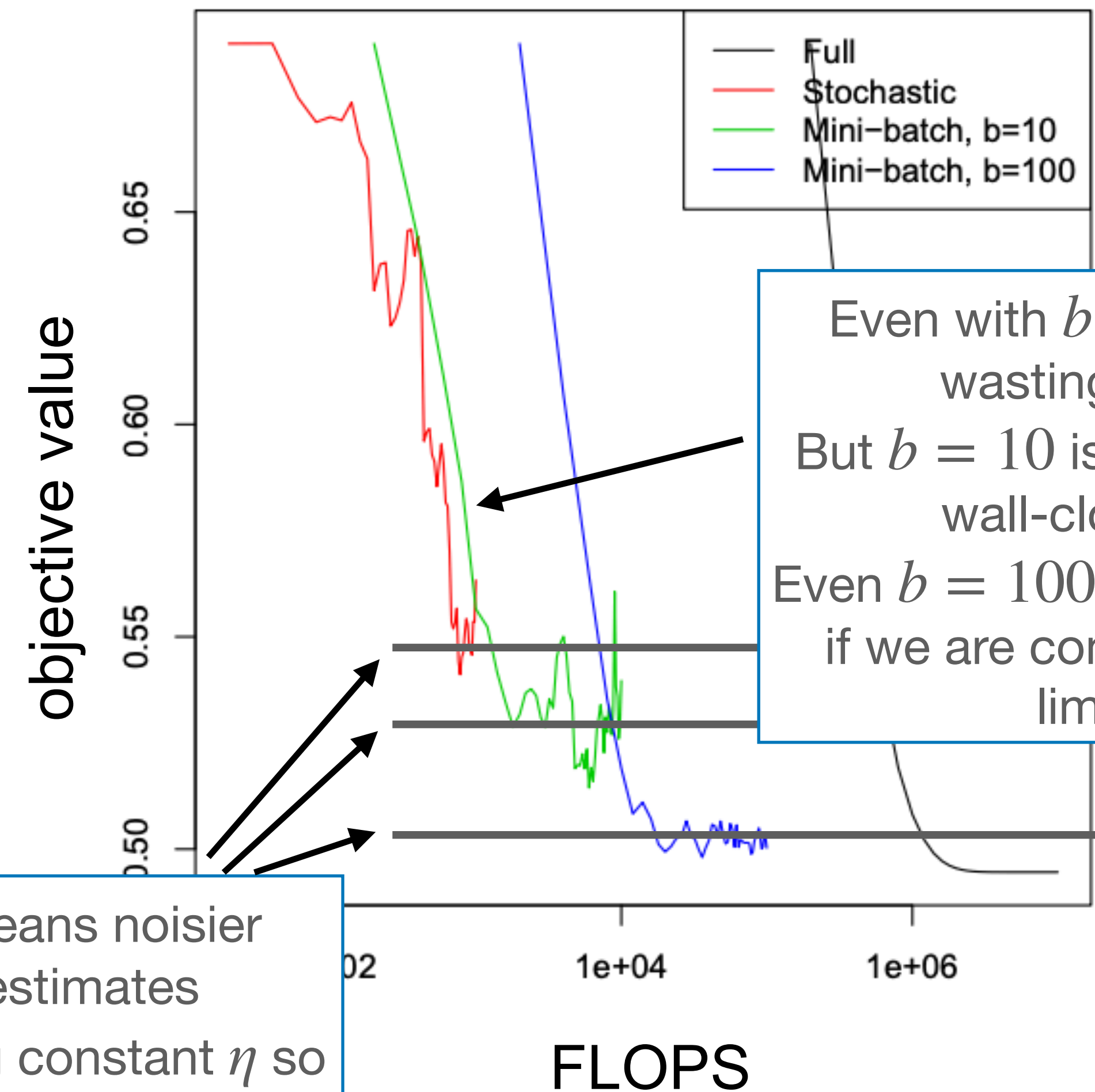
Smaller b means noisier gradient estimates
We are keeping constant η so we reach the random-walk stage at higher error

SGD vs. Mb-SGD vs. GD

Compare **FLOPs** vs. **objective value**

- FLOPs = floating point operations (compute cost)

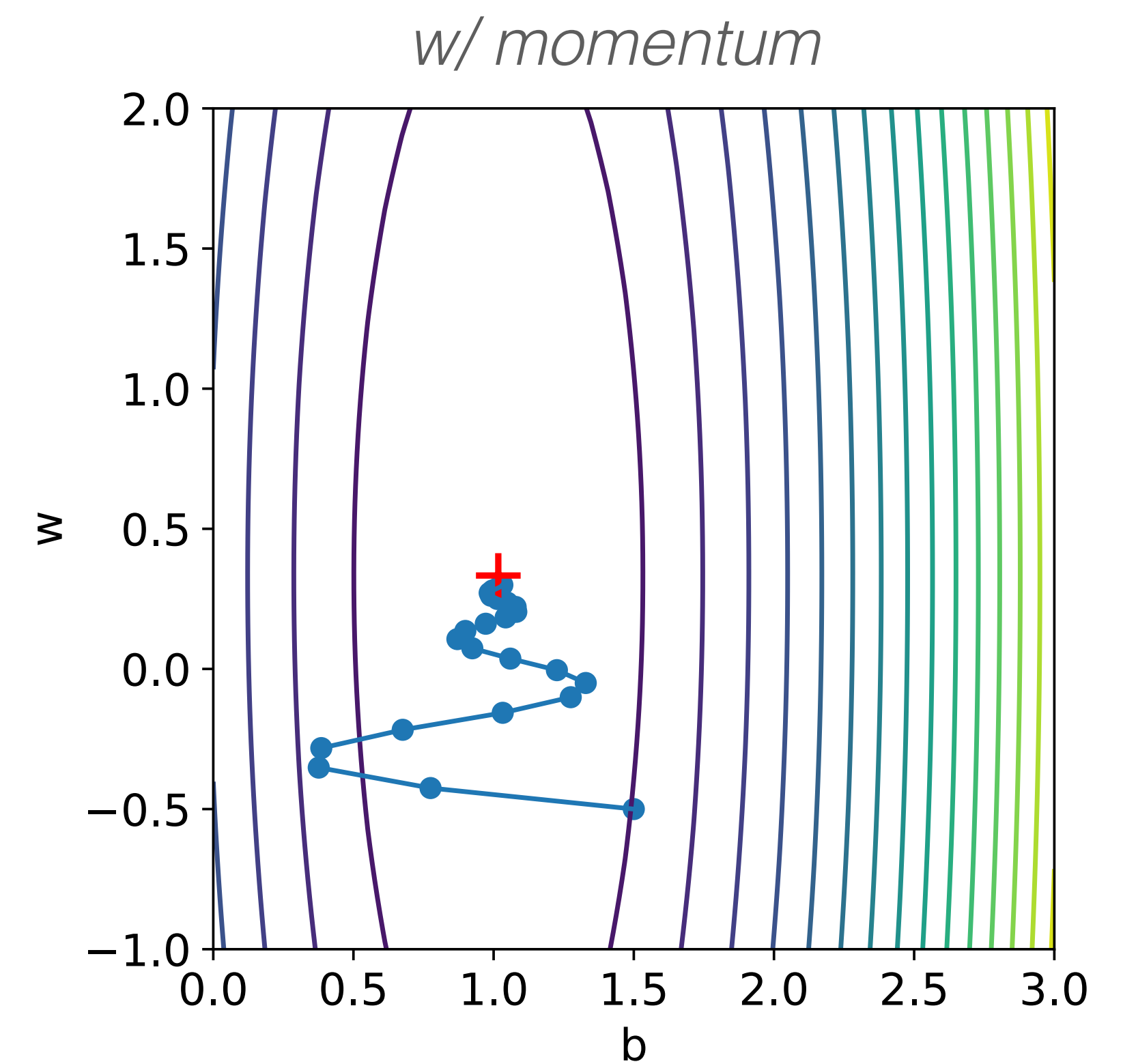
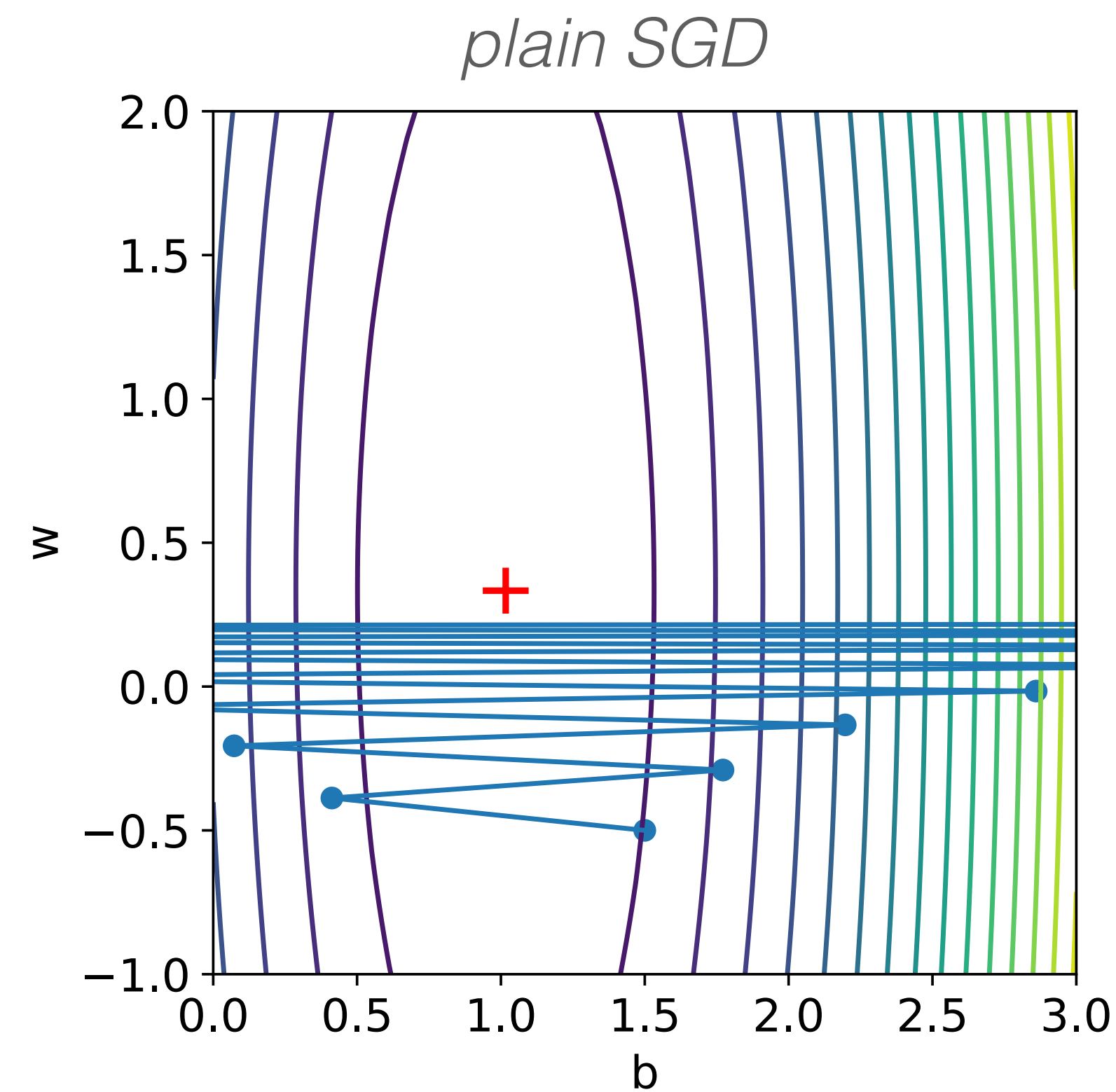
→ *Although Mb-SGD converges more slowly than GD terms of iterations, it may converge more quickly in terms of FLOPs*



Even with $b = 10$ we are wasting FLOPs
But $b = 10$ is likely faster in wall-clock time
Even $b = 100$ might be faster if we are communication-limited

Smaller b means noisier gradient estimates
We are keeping constant η so we reach the random-walk stage at higher error

Gradient variance



- Momentum helps with poor scaling / poor conditioning
- Can we do better?
- Idea: gradient has high variance in directions where we're oscillating
- Suggests tracking variance $\rightarrow$ Adam

Learning rate scales differently than SGD

Track the gradient variance: Adam

AdamW: handle weight decay separately

- Adam (“ADaptive Moment estimation”):

$$m_{t+1} = \beta_m m_t + (1 - \beta_m) g_t$$

gradient estimate g_t

$$v_{t+1} = \beta_v v_t + (1 - \beta_v) g_t^2$$

$(\cdot)^2$ is componentwise

$$\hat{m}_{t+1} = \frac{1}{1 - \beta_m^t} m_{t+1}, \quad \hat{v}_{t+1} = \frac{1}{1 - \beta_v^t} v_{t+1}$$

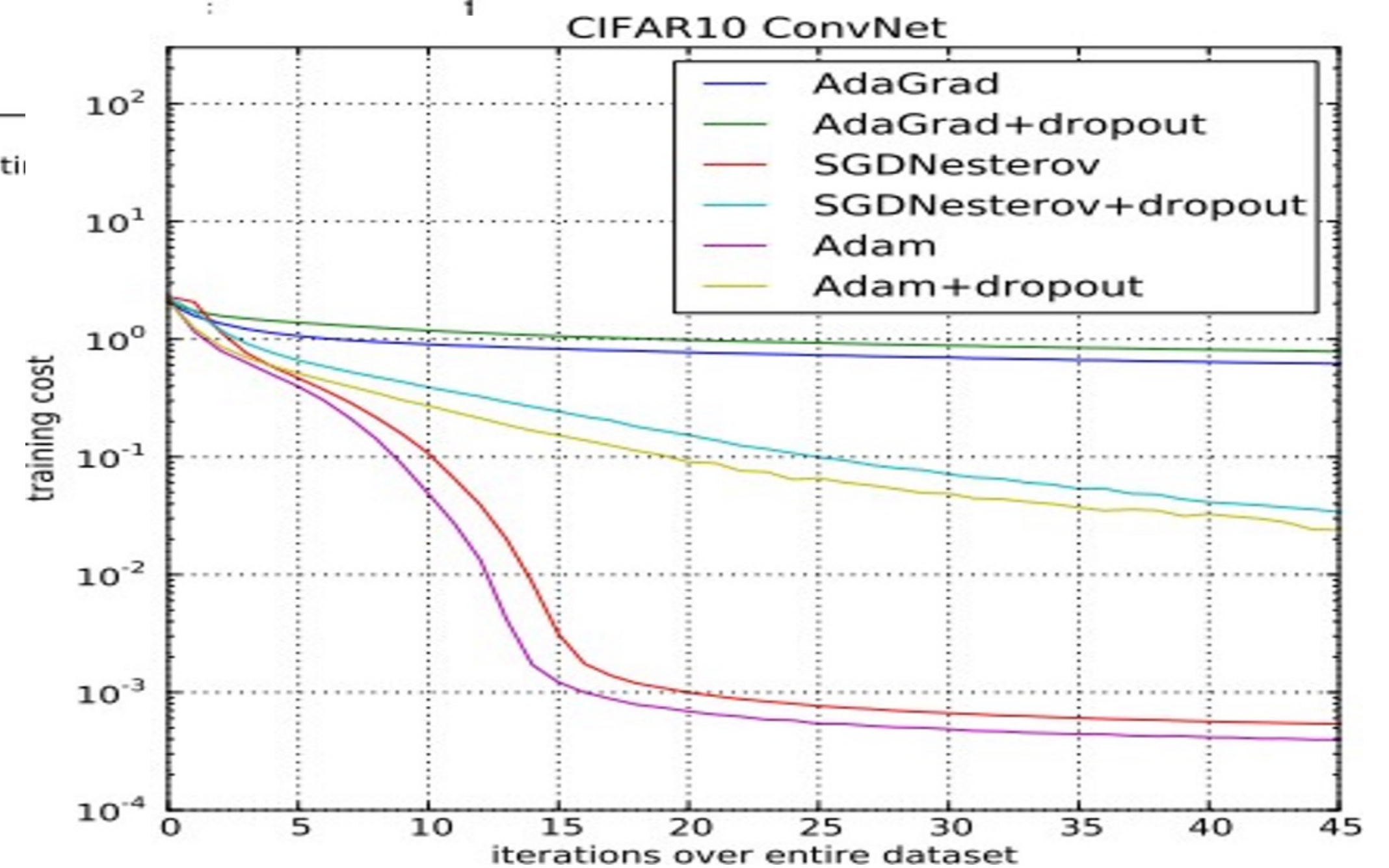
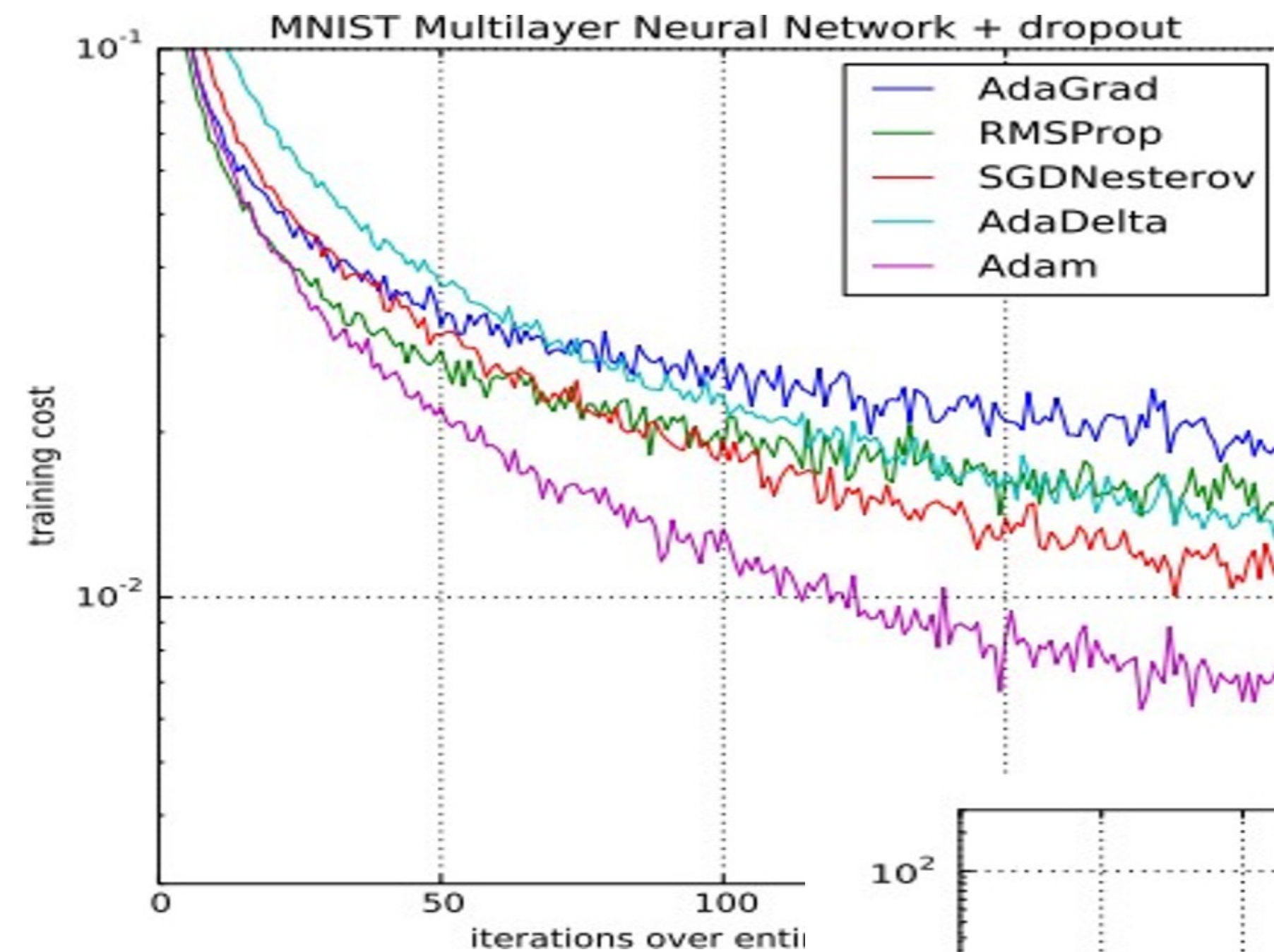
transient correction

$$\theta_{t+1} = \theta_t - \eta \frac{1}{\sqrt{\hat{v}_{t+1}} + \epsilon} \hat{m}_{t+1}$$

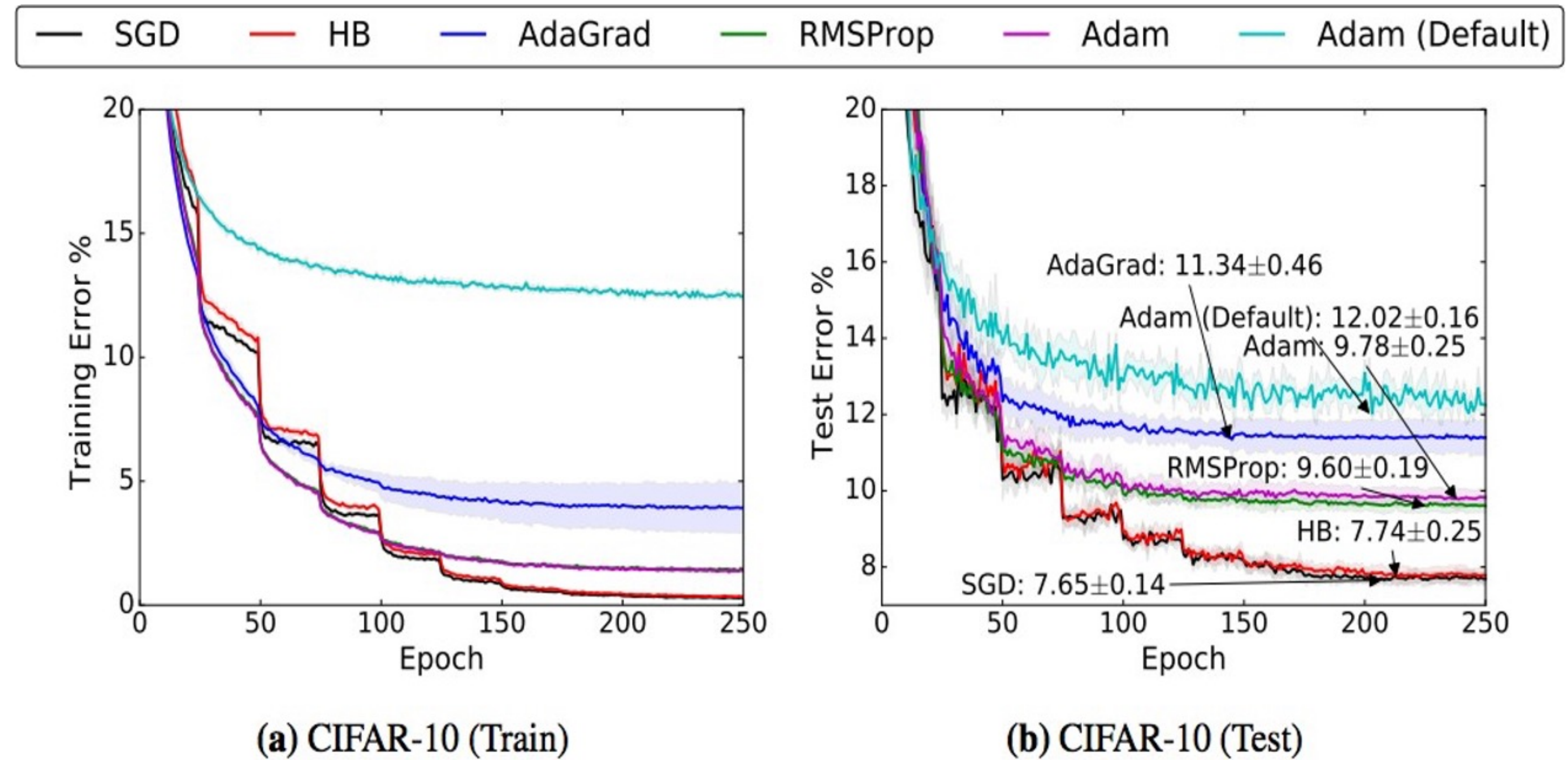
$\sqrt{\cdot}$ is componentwise

- Four hyperparameters: moving average $\beta_m, \beta_v \in (0,1)$, learning rate $\eta > 0$, curvature regularizer $\epsilon > 0$
- Warning! Not proven to converge (and in fact does not converge in some cases)
 - there are other methods that try to use gradient variance while guaranteeing better theoretical properties, but none uniformly better than Adam in practice

Adam performance



But!



- In some cases Adam may not improve over SGD or SGD+momentum — may even be noticeably worse
- Sensitive to hyperparameters (and there are 4 of them)
- Especially test error may suffer!
- Upshot: Adam is worth trying, esp. if SGD seems infeasibly slow, but is not a panacea

[figure credit: Wilson et al., 2017]

Learning objectives— convexity, gradient descent

- When would we choose gradient descent vs. exact for linear regression? How would we implement either (w/ access to NumPy and Torch)?
- What is a convex function? A convex set?
- How does convexity affect the set of local or global optima? How does it affect gradient descent?
- When is the optimal solution for linear regression well-determined, poorly-determined, or undetermined?
- What effect does a poorly-determined solution have on gradient descent? How can we mitigate this?
- How can we use SGD or minibatch SGD to scale to large datasets and take advantage of parallel compute?